UWBIoT API Reference Guide

Rev. 1 — 16 March 2026 User manual

1 UWBIoT API Reference

1.1 APIs : UWB IoT and SR250

1.1.1 UWB API Status Codes

*group***uwb_status**

Various Status cores returned by UWB APIs

Defines

UWBAPI_STATUS_LOW_POWER_ERROR

Status Codes for SR040.
Battery low error status

UWBAPI_STATUS_OK

Status Codes, as per UCI.
Command succeeded

UWBAPI_STATUS_REJECTED

Request is rejected

UWBAPI_STATUS_FAILED

Command Failed

UWBAPI_STATUS_NOT_INITIALIZED

API called without UCI being initialized

UWBAPI_STATUS_INVALID_PARAM

Invalid parameter provided

UWBAPI_STATUS_INVALID_RANGE

Invalid value range provided

UWBAPI_STATUS_SYNTAX_ERROR

UWBAPI_STATUS_SESSION_NOT_EXIST

Session wrt SESSION Handle Does not exist

UWBAPI_STATUS_SESSION_ACTIVE

Session active

UWBAPI_STATUS_MAX_SESSIONS_EXCEEDED

MAX Sessions exceeded

UWBAPI_STATUS_SESSION_NOT_CONFIGURED

Operation is started with out configuring required parameters for Session

UWBAPI_STATUS_SESSIONS_ONGOING

Sessions ongoing

UWBAPI_STATUS_MULTICAST_LIST_FULL

Indicates when multicast list is full during one to many ranging

UWBAPI_STATUS_ERROR_SESSION_INVALID_SLOT_ALLOCAION

Slot allocation for phases is not correct

UWBAPI_STATUS_ERROR_SESSION_INVALID_SLOT_DURATION

Slot duration of the secondary is not integer multiple of the slot duration of primary session

UWBAPI_STATUS_OK_NEGATIVE_DISTANCE_REPORT

Negative distance was reported

UWBAPI_STATUS_ESE_RESET

ESE Rest happened during command processing.

UWBAPI_STATUS_DATA_TRANSFER_ERROR

Unrecoverable data transfer error

UWBAPI_STATUS_NO_CREDIT_AVAILABLE

Credit not available for Data Packet

UWBAPI_STATUS_ERROR_ROUND_INDEX_NOT_ACTIVATED

Given round index couldn't be activated

UWBAPI_STATUS_ERROR_NUMBER_OF_ACTIVE_RANGING_ROUNDS_EXCEEDED

Given round exceeds the possible number of ranging rounds

UWBAPI_STATUS_ERROR_ROUND_INDEX_NOT_SET_AS_INITIATOR

The role for the configured ranging round index is not Initiator and therefore RDM list cannot be set

UWBAPI_STATUS_DLTDOA_DEVICE_ADDRESS_NOT_MATCHING_IN_REPLY_TIMELIST

Device address not matching

UWBAPI_STATUS_BUFFER_OVERFLOW

Buffer overflow

UWBAPI_STATUS_PBF_PKT_SENT

Status PBF=1 CMD SENT

UWBAPI_STATUS_HPD_WAKEUP

Device is woken up from HPD

UWBAPI_STATUS_TIMEOUT

Command failed with time out

UWBAPI_STATUS_ESE_ERROR

SE Error

UWBAPI_STATUS_SUSPEND

Ranging suspended

UWBAPI_STATUS_SESSION_RSN_RADAR_RFRI_INVALID

Ranging interval/slot duration incorrect

UWBAPI_STATUS_UNKNOWN

It is not known whether the intended operation was failed or successful.

UWBAPI_STATUS_INVALID_RESPONDER_SLOT_INDEX

All information provided in this document is subject to legal disclaimers.

© 2026 NXP B.V. All rights reserved.

Responder slot index is invalid in the requested configuration

UWBAPI_STATUS_SLOT_LEN_NOT_SUPPORTED

Slot length is not supported in the requested configuration

UWBAPI_STATUS_INVALID_SLOT_PER_RR

Number of slots is invalid in the requested configuration

UWBAPI_STATUS_INVALID_STS_IDX

STS index is not valid.

UWBAPI_STATUS_RESPONDER_NOT_PARTICIPATING_IN_MEASUREMENT

Responder is in the listen only mode. Responder does not send a range frame

UWBAPI_UCI_DEV_INIT

Device Status codes, as per UCI.
Device State - INIT

UWBAPI_UCI_DEV_READY

Device State - READY

UWBAPI_UCI_DEV_ACTIVE

Device State - ACTIVE

UWBAPI_UCI_DEV_ERROR

Device State - ERROR

UWBAPI_SESSION_INIT_SUCCESS

Session State codes, as per UCI.
Session State - Session Init Success

UWBAPI_SESSION_INIT_SUCCESS

Session State codes, as per UCI.
Session State - Session Init Success

UWBAPI_SESSION_DEINIT_SUCCESS

Session State - Session DeInit Success

UWBAPI_SESSION_ACTIVATED

Session State - Session Activated

UWBAPI_SESSION_IDLE

Session State - Session Idle

UWBAPI_SESSION_ERROR

Session State - Error

UWBAPI_STATUS_OK_MULTICAST_LIST_UPDATE

Multicast update status codes, as per UCI.
The multicast list is updated (Add/Delete) successfully for the given Controlee.

UWBAPI_STATUS_ERROR_MULTICAST_LIST_FULL

The multicast is full

UWBAPI_STATUS_ERROR_KEY_FETCH_FAIL

Session Key fetch from Secure Component is failed

UWBAPI_STATUS_ERROR_SUB_SESSION_ID_NOT_FOUND

All information provided in this document is subject to legal disclaimers.
© 2026 NXP B.V. All rights reserved.

Sub-Session ID is not found in Secure Component

UWBAPI_STATUS_ERROR_SUB_SESSION_KEY_NOT_FOUND

Sub-Session Key is not found in Secure Component

UWBAPI_STATUS_ERROR_SUB_SESSION_KEY_NOT_APPLICABLE

Sub-Session Key is configured with STS config is other than 0x04 (Provisioned STS for Responder specific Sub-session Key)

UWBAPI_STATUS_ERROR_SESSION_KEY_NOT_FOUND

Sub-Session Key is configured but SESSION_KEY App configuration parameter is not programmed

UWBAPI_STATUS_ERROR_ADDRESS_NOT_FOUND

its short address is not found while deleting its entry from multicast list

UWBAPI_STATUS_ERROR_ADDRESS_ALREADY_PRESENT

its short address is already present in the multicast list

UWBAPI_DATA_TRANSFER_STATUS_REPETITION_OK

Data Transfer related status codes.

Data transmission is ongoing.

UWBAPI_DATA_TRANSFER_STATUS_OK

Data transmission is completed.

UWBAPI_DATA_TRANSFER_STATUS_ERROR

Application Data could not be sent due to an unrecoverable error

UWBAPI_DATA_TRANSFER_STATUS_NO_CREDIT_AVAILABLE

DATA_MESSAGE_SND is not accepted as no credit is available

UWBAPI_DATA_TRANSFER_STATUS_REJECTED

DATA_MESSAGE_SND packet sent in wrong state or Application Data Size exceeds the maximum size that can be sent in one Ranging Round.

UWBAPI_DATA_TRANSFER_STATUS_TYPE_NOT_SUPPORTED

Data transfer is not supported for given session type.

UWBAPI_DATA_TRANSFER_DATA_TRANSFER_IS_ONGOING

Application Data is being transmitted and the number of configured DATA_REPETITION_COUNT transmissions is not yet completed

UWBAPI_DATA_TRANSFER_STATUS_INVALID_FORMAT

The format of the command DATA_MESSAGE_SND associated with this notification is incorrect (e.g, a parameter is missing, a parameter value is invalid).

UWB_DATA_TRANSFER_STATUS

ULTDOA_MESSAGE_CONTROL_MASK

UL-TDoA related definitions.

UL-TDoA Message Control Mask

ULTDOA_DEVICE_ID_16BIT_VALUE

16-bit UL-TDoA Device ID

ULTDOA_DEVICE_ID_32BIT_VALUE

32-bit UL-TDoA Device ID

All information provided in this document is subject to legal disclaimers.

© 2026 NXP B.V. All rights reserved.

ULTDOA_DEVICE_ID_64BIT_VALUE

64-bit UL-TDoA Device ID

ULTDOA_DEVICE_ID_MASK

MASK for above values.

ULTDOA_40BIT_TX_TIMESTAMP_MASK

40-bit TX timestamp

ULTDOA_64BIT_TX_TIMESTAMP_MASK

64-bit TX timestamp

ULTDOA_64BIT_RX_TIMESTAMP_MASK

64-bit RX timestamp

ULTDOA_16BIT_IN_BYTES

Length in bytes for 16-bits

ULTDOA_32BIT_IN_BYTES

Length in bytes for 32-bits

ULTDOA_40BIT_IN_BYTES

Length in bytes for 40-bits

ULTDOA_64BIT_IN_BYTES

Length in bytes for 64-bits

AUTH_TAG_IN_16BYTES

Length in Bytes for 128-bits

MAX_NUM_ANTENNA_PAIR

Max number of supported rx antenna ID/pair

FOV_SPECIFIC_DATA_TYPE

FoV specific Vendor Data type

NXP_SPECIFIC_DATA_TYPE_WITH_MSG_CNTRL

NXP Specific Data with Message Control Vendor Data type

CRYPTO_TAG_LEN

Cryptograpic tag length

GET_ARRAY_SIZE(*array*)

Macro to get the Array size

IS_CSA_SESSION(*sessionHandle*)

IS_CCC_SESSION(*sessionHandle*)

Enums

enum **eUWBD_DEVICE_STATUS_t**

Values:

enumerator **UWBD_STATUS_INIT**

enumerator **UWBD_STATUS_READY**

enumerator **UWBD_STATUS_ACTIVE**

All information provided in this document is subject to legal disclaimers.

© 2026 NXP B.V. All rights reserved.

enumerator **UWBD_STATUS_HDP_WAKEUP**

enumerator **UWBD_STATUS_UNKNOWN**

enumerator **UWBD_STATUS_ERROR**

enum **eSESSION_STATUS_t**

Values:

enumerator **UWB_SESSION_INITIALIZED**

enumerator **UWB_SESSION_DEINITIALIZED**

enumerator **UWB_SESSION_ACTIVE**

enumerator **UWB_SESSION_IDLE**

enumerator **UWB_SESSION_ERROR**

1.1.2 UWB API Consts

*group***uwb_const**

Some helper constants for UWB APIs

1.1.3 UWB API Data Types

*group***uwb_types**

Various structrues and data types used by UWB

Defines

MODELID_RFU

CLOCK_DRIFT

kUWB_RangingMethod_TDoA

kUWB_RangingMethod_SS_TWR

kUWB_RangingMethod_DS_TWR

kUWB_RangingMethod_SS_TWR_ND

kUWB_RangingMethod_DS_TWR_ND

kUWB_RangingMethod_HUS

tUwbApi_AppDataCallback

Deprecated

IS_DATA_TRANSFER_CONNECTION_ORIENTED(*llmSelector*)

Connection Oriented Modes

IS_DATA_TRANSFER_CONNECTION_LESS(*llmSelector*)

Connection Less Modes

Typedefs

typedef uint8_t **tUWBAPI_STATUS**

Types used by Uwb APIs.

All information provided in this document is subject to legal disclaimers.

© 2026 NXP B.V. All rights reserved.

Status used by UWB APIs.

typedef UWB_RangingRoundUsage_t **UWB_RangingMethod_t**

typedef void **tUwbApi_AppCallback**(*eNotificationType opType, void *pData, size_t len*)

Generic callback function registered by Application to receive data from middleware. This is a common callback for All session types.

1. Ranging
2. App Data Transfer
3. PER Exchange
4. Generic Error Notification
5. Device Reset Notification
6. RFrame Data Notification
7. Recovery Notification
8. Scheduler status Notification

This data is provided by the stack, to the user. The user should implement this callback if user wants to receive the NTF information.

- If the callback function pointer is NULL, the user will not receive the notifications.

Param opType

Type of Notification

Param pData

Data received

Param len

Data length

typedef void **tUwbApi_TMLDataCallback**(*uint8_t *recvData, uint16_t recvDataLen*)

TML callback function registered by Application to receive data packet from UWB If the callback function pointer is NULL, the user will not receive the data packet from TML.

Param recvData

pointer to received data

Param recvDataLen

len of data received

Enums

enum **eDataTxPhaseCfg_NftCode**

Data Transfer Phase configuration notification status codes.

These error codes to be returned by the UWBS when the DTPML configuration provided by the Host are applied.

Note:

Notification specific return codes.

Values:

enumerator **DTPCM_STATUS_OK_DTPCM_CONFIG_SUCCESS**

Reported when DTPCM is configured for given MAC Address

enumerator **DTPCM_STATUS_ERROR_DUPLICATE_SLOT_ASSIGMENT**

Reported when configured slot assignments is inconsistent, i.e., one slot is assigned to more than one FiRA device

enumerator **DTPCM_STATUS_ERROR_DTPML_OVERSIZE**

Reported when configured slot assignments cannot fit within one DTPCM.

enum **eDevResetMode**

All information provided in this document is subject to legal disclaimers. © 2026 NXP B.V. All rights reserved.

RESET Modes.
Values:

enumerator **UWBD_RESET**

UWB Device Reset

enumerator **RFU**

RFU

enum **eSessionType**

UWBD Session Type.
Values:

enumerator **UWBD_RANGING_SESSION**

Ranging Session

enumerator **UWBD_RANGING_WITH_INBAND_DATA_TRANSFER**

Inband data transfer

enumerator **UWBD_DATA_TRANSFER**

Data Transfer Session

enumerator **UWBD_RANGING_ONLY_PHASE**

Ranging Only Phase

enumerator **UWBD_INBAND_DATA_PHASE**

Inband Data Phase

enumerator **UWBD_RANGING_WITH_DATA_PHASE**

Ranging With Data Phase

enumerator **UWBD_HUS_SESSION**

HUS Primary Session

enumerator **UWBD_CCC_SESSION**

CCC Session

enumerator **UWBD_CSA_SESSION**

CSA Session

enumerator **UWBD_RFTEST**

Test Session

enumerator **UWBD_RADAR_TRANSFER**

Radar Trasnfer Session

enumerator **UWBD_RFU**

RFU

enum **eLinkLayerModes**

Modes for setting LINK_LAYER_MODE app config, supported in UWB API layer.
Values:

enumerator **Link_Layer_Mode_Bypass**

Bypass mode.

enumerator **Link_Layer_Mode_LogicalLink**

All information provided in this document is subject to legal disclaimers.
© 2026 NXP B.V. All rights reserved.

Logical Link Mode (default)

enum **eNotificationType**

UWBD notification Type.
Values:

enumerator **UWBD_RANGING_DATA**

Ranging Data Notification

enumerator **UWBD_DATA_TRANSMIT_NTF**

Data Transmit Notification

enumerator **UWBD_PER_SEND**

PER Packet Sent Notification

enumerator **UWBD_PER_RCV**

PER receive operation completed notification

enumerator **UWBD_SR_RX_RCV**

TEST_SR_RX_NTF receive operation completed notification

enumerator **UWBD_GENERIC_ERROR_NTF**

Generic Error Notification

enumerator **UWBD_DEVICE_RESET**

Upon Receiving Device Reset, Application needs to clear all session context and re-initiate all the sessions

enumerator **UWBD_RFRAME_DATA**

RFRAME Data Notification

enumerator **UWBD_DBG_DPD_INFO_NTF**

DPD timings profiling notification

enumerator **UWBD_CIR_PULL_DATA_NTF**

CIR Pull Data notification

enumerator **UWBD_RECOVERY_NTF**

Upon receiving Recovery Notification, Application has to invoke Recovery API non main/application thread context.

enumerator **UWBD_SCHEDULER_STATUS_NTF**

UWBS shall send SCHED_STATUS_NTF notification when scheduler meets either SESSION_SCHEDULER_ATTEMPTS or SESSION_SYNC_ATTEMPTS configuration criteria.

enumerator **UWBD_SESSION_DATA**

Session Data Notification

enumerator **UWBD_MULTICAST_LIST_NTF**

Multicast list notification

enumerator **UWBD_OVER_TEMP_REACHED**

Over Temperature reached notification

enumerator **UWBD_BLINK_DATA_TX_NTF**

Blink data tx notification

enumerator **UWBD_DATA_TRANSFER_PHASE_CONFIG_NTF**

Data Tx Phase Configuration Notification

All information provided in this document is subject to legal disclaimers.
© 2026 NXP B.V. All rights reserved.

enumerator **UWBD_ACTION_APP_CLEANUP**

TEST RX notification Perform Application Clean Up & Restart upon receiving this notification

enumerator **UWBD_TEST_MODE_LOOP_BACK_NTF**

Loopback Status Data

enumerator **UWB_TEST_PHY_LOG_NTF**

PHY LOG NTF

enumerator **UWB_TEST_EXT_PSDU_LOG_NTF**

EXT PSDU LOG NTF

enumerator **UWBD_TEST_RX_RCV**

TEST RX notification

enumerator **UWBD_DATA_RECV_NTF**

Data receive

enumerator **UWBD_CIR_DATA_NTF**

CIR notification data

enumerator **UWBD_DATA_LOGGER_NTF**

Data logger ntf

enumerator **UWBD_PSDU_DATA_NTF**

PSDU log ntf

enumerator **UWBD_RANGING_TIMESTAMP_NTF**

RANGING timestamp ntf

enumerator **UWBD_COMMAND_TIMESTAMP_NTF**

COMMAND timestamp ntf

enumerator **UWBD_DATA_RCV_NTF**

Data receive notification

enumerator **UWBD_CREDIT_RCV_NTF**

Data Credit Notification

enumerator **UWBD_RADAR_RCV_NTF**

Radar rcv ntf

enumerator **UWBD_TEST_RADAR_ISO_NTF**

Test radar Isolation ntf

enumerator **UWBD_PRESENCE_DETECTION_NTF**

Presence detection ntf

enumerator **UWBD_WIFI_WLAN_IND_NTF**

enumerator **UWB_WLAN_COEX_MAX_GRANT_DURATION_EXCEEDED_WRN_NTF**

Max Active Grant Duration Exceeded Warning NTF

enumerator **WLAN_UWB_IND_NTF**

WiFi CoEx IND Error Notification

enumerator **UWBD_RANGING_CCC_DATA**

Ranging CCC/CSA Data Notification

All information provided in this document is subject to legal disclaimers.
© 2026 NXP B.V. All rights reserved.

enumerator **UWBD_BINDING_STATUS_NTF**

Event for Binding Status Notification

enumerator **UWBD_LL_CREATE_NTF**

Event for Logical Link Create Notification

enumerator **UWBD_LL_UWBS_CLOSE_NTF**

Event for Logical Link Close Notification

enumerator **UWBD_LL_UWBS_CREATE_NTF**

Event for Logical Link create UWBS Notification

enumerator **UWBD_LOGICAL_LINK_DATA_RCV_NTF**

Logical Link Data receive notification

enumerator **UWBD_SESSION_ROLE_CHANGE_RCV_NTF**

enumerator **UWBD_SESSION_ERROR_TIMEOUT_NTF**

Error Scenario TIMEOUT ntf

enumerator **UWBD_INVALID_NTF_EVT**

Invalid notification event

enum **eCccRangingStatus**

enum lists out the CCC ranging notification information status. For the CCC session, the lower nibble represents the ranging status of the controller and the higher nibble represents the ranging status of the controlee. For other sessions, this contains only the status of the controllee.

Values:

enumerator **CCC_STATUS_SUCCESS**

enumerator **CCC_STATUS_TRANSACTION_OVFL**

enumerator **CCC_STATUS_TRANSACTION_EXPD**

enumerator **CCC_STATUS_INCORRECT_FRAME**

enumerator **CCC_STATUS_RESP_LISTEN**

enumerator **CCC_STATUS_CTRL_MSG_LOST**

enum **eCCC_HopModes_t**

Enumerator lists out the Hop modes for CCC Controller.

Note:

This config is applicable only for Controller and ignored in case of Controlee.

Values:

enumerator **kUWB_CCC_HopMode_Disable**

enumerator **kUWB_CCC_HopMode_Cont_AES**

enumerator **kUWB_CCC_HopMode_Cont_def**

enumerator **kUWB_CCC_HopMode_Adapt_AES**

enumerator **kUWB_CCC_HopMode_Adapt_def**

enumerator **kUWB_NXP_HopMode_Adapt**

enum **eUWB_ProfileID_t**

Enumeration lists out the Vendor specific Profile ID.

All information provided in this document is subject to legal disclaimers.
© 2026 NXP B.V. All rights reserved.

Values:

enumerator **kUWB_Profile_1**

enum **demo_chip_type_t**

Enumeration lists out the chip specific type used.
Values:

enumerator **kChipType_NA**

enumerator **kChipType_SR150**

enumerator **kChipType_SR040**

enumerator **kChipType_SR250**

enum **demo_board_type_t**

Enumeration lists out the board specific type used.
Values:

enumerator **kBoardType_NA**

enumerator **kBoardType_Shield**

enumerator **kBoardType_FinderV3**

enumerator **kBoardType_Virgo**

enum **PreferedUpdateRate_t**

Enumeration lists out the Preferred specific rate type used.
Values:

enumerator **kUpdateRate_Automatic**

enumerator **kUpdateRate_Infrequent**

enumerator **kUpdateRate_UserInteractive**

enum **UWB_ChannelNumber_t**

Enumeration lists out the channel numbers used.
Values:

enumerator **kUWB_ChannelNumber_5**

enumerator **kUWB_ChannelNumber_6**

enumerator **kUWB_ChannelNumber_8**

enumerator **kUWB_ChannelNumber_9**

enum **UWB_MacFcsType_t**

Enumeration lists out the MAC FCS type used.
Values:

enumerator **kUWB_MacFcsType_CRC16**

enumerator **kUWB_MacFcsType_CRC32**

enum **UWB_SfdId_t**

Enumeration lists out the SFD ID's used.
Values:

enumerator **kUWB_SfdId_BPRF_0**

All information provided in this document is subject to legal disclaimers.
© 2026 NXP B.V. All rights reserved.

enumerator **kUWB_SfdId_BPRF_2**

enumerator **kUWB_SfdId_HPRF_1**

enumerator **kUWB_SfdId_HPRF_2**

enumerator **kUWB_SfdId_HPRF_3**

enum **UWB_PreambleIndxCode_t**

Enumeration lists out the Preamble Index codes used.
Values:

enumerator **kUWB_PreambleIndxCode_BPRF_09**

enumerator **kUWB_PreambleIndxCode_BPRF_10**

enumerator **kUWB_PreambleIndxCode_BPRF_11**

enumerator **kUWB_PreambleIndxCode_BPRF_12**

enumerator **kUWB_PreambleIndxCode_HPRF_25**

enumerator **kUWB_PreambleIndxCode_HPRF_26**

enumerator **kUWB_PreambleIndxCode_HPRF_27**

enumerator **kUWB_PreambleIndxCode_HPRF_28**

enumerator **kUWB_PreambleIndxCode_HPRF_29**

enumerator **kUWB_PreambleIndxCode_HPRF_30**

enumerator **kUWB_PreambleIndxCode_HPRF_31**

enumerator **kUWB_PreambleIndxCode_HPRF_32**

enum **UWB_PreambleDuration_t**

Enumeration lists out the Preamble duration used.
Values:

enumerator **kUWB_PreambleDuration_32Symbols**

enumerator **kUWB_PreambleDuration_64Symbols**

enum **UWB_Session_InfoNtf_t**

Enumeration lists out the Ranging Data Notifications used.
Values:

enumerator **kUWB_DisableSession_Info_Ntf**

enumerator **kUWB_EnableSession_Info_Ntf**

enumerator **kUWB_SessionInfo_Ntf_Proximity**

enumerator **kUWB_SessionInfo_Ntf_AOABounds**

enumerator **kUWB_SessionInfo_Ntf_AOABounds_n_Proximity**

enumerator **kUWB_SessionInfo_Ntf_Enter_Leave_Proximity**

enumerator **kUWB_SessionInfo_Ntf_Enter_Leave_AOABounds**

enumerator **kUWB_SessionInfo_Ntf_Enter_Leave_AOABounds_n_Proximity**

enum **UWB_PrfMode_t**

All information provided in this document is subject to legal disclaimers.
© 2026 NXP B.V. All rights reserved.

Enumeration lists out the PRF modes used.
Values:

enumerator **kUWB_PrfMode_62_4MHz**

BPRF

enumerator **kUWB_PrfMode_124_8MHz**

HPRF

enumerator **kUWB_PrfMode_249_6MHz**

HPRF mode with data rate 27.2 and 31.2 Mbps

enum **UWB_PsduDataRate_t**

Enumeration lists out the PSDU data rates used.
Values:

enumerator **kUWB_PsduDataRate_6_81Mbps**

enumerator **kUWB_PsduDataRate_7_80Mbps**

enumerator **kUWB_PsduDataRate_27_2Mbps**

enumerator **kUWB_PsduDataRate_31_2Mbps**

enumerator **kUWB_PsduDataRate_850Kbps**

enum **UWB_RfFrameConfig_t**

Enumeration lists out the RFrame Config sts values.
Values:

enumerator **kUWB_RfFrameConfig_No_Sts**

enumerator **kUWB_RfFrameConfig_SP0**

enumerator **kUWB_RfFrameConfig_Sfd_Sts**

enumerator **kUWB_RfFrameConfig_SP1**

enumerator **kUWB_RfFrameConfig_Psdu_Sts**

enumerator **kUWB_RfFrameConfig_Sfd_Sts_NoPhr_NoPsdu**

enumerator **kUWB_RfFrameConfig_SP3**

enum **UWB_DeviceRole_t**

Enumeration lists out the Device role values.
Values:

enumerator **kUWB_DeviceRole_Responder**

enumerator **kUWB_DeviceRole_Initiator**

enumerator **kUWB_DeviceRole_UT_Sync_Anchor**

enumerator **kUWB_DeviceRole_UT_Anchor**

enumerator **kUWB_DeviceRole_UT_Tag**

enumerator **kUWB_DeviceRole_Advertiser**

enumerator **kUWB_DeviceRole_Observer**

enumerator **kUWB_DeviceRole_DlTDoA_Anchor**

All information provided in this document is subject to legal disclaimers.
© 2026 NXP B.V. All rights reserved.

enumerator **kUWB_DeviceRole_DlTDoA_Tag**

enum **UWB_MultiNodeMode_t**

Enumeration lists out the Multicast mode values.
Values:

enumerator **kUWB_MultiNodeMode_UniCast**

enumerator **kUWB_MultiNodeMode_OnetoMany**

enumerator **kUWB_MultiNodeMode_ManytoMany**

enum **UWB_DeviceType_t**

Enumeration lists out the Device type values.
Values:

enumerator **kUWB_DeviceType_Controlee**

enumerator **kUWB_DeviceType_Controller**

enumerator **kUWB_DeviceType_CCC_Controller**

enumerator **kUWB_DeviceType_CCC_Controlee**

enum **UWB_RangingRoundUsage_t**

Enumeration lists out the Ranging Round Usage values.
Values:

enumerator **kUWB_RangingRoundUsage_TDoA**

enumerator **kUWB_RangingRoundUsage_SS_TWR**

enumerator **kUWB_RangingRoundUsage_DS_TWR**

enumerator **kUWB_RangingRoundUsage_SS_TWR_nd**

enumerator **kUWB_RangingRoundUsage_DS_TWR_nd**

enumerator **kUWB_RangingRoundUsage_DL_TDOA**

enumerator **kUWB_RangingRoundUsage_OWR_AOA**

enumerator **kUWB_RangingRoundUsage_eSS_TWR**

enumerator **kUWB_RangingRoundUsage_aDS_TWR**

enumerator **kUWB_RangingRoundUsage_DTx**

enumerator **kUWB_RangingRoundUsage_HUS**

enum **UWB_Radar_PulseShape_t**

Enumeration lists out the Radar TX Pulse Shape IDs.
Valid pulse shapes are restricted by firmware. Any value outside this enum will be rejected.
Values:

enumerator **kUWB_Radar_PulseShape_41**

Pulse Shape 41

enumerator **kUWB_Radar_PulseShape_50**

Pulse Shape 50

enumerator **kUWB_Radar_PulseShape_65**

Pulse Shape 65

All information provided in this document is subject to legal disclaimers.
© 2026 NXP B.V. All rights reserved.

enum **UWB_AOA_Result_Req_t**

Enumeration lists out the AOA_RESULT_REQ values.
Values:

enumerator **kUWB_AOA_Result_Req_Disable**

enumerator **kUWB_AOA_Result_Req_Enable**

enumerator **kUWB_AOA_Result_Req_Azimuth**

enumerator **kUWB_AOA_Result_Req_Elevation**

enum **UWB_StsConfig_t**

Enumeration lists out the STS Config values.
Values:

enumerator **kUWB_StsConfig_StaticSts**

enumerator **kUWB_StsConfig_DynamicSts**

enumerator **kUWB_StsConfig_DynamicSts_Ctrlee_key**

enumerator **kUWB_StsConfig_ProvisionSts**

enumerator **kUWB_StsConfig_ProvisionSts_Ctrlee_key**

enum **UWB_MacAddressMode_t**

Enumeration lists out the MAC address mode.
Values:

enumerator **kUWB_MacAddressMode_2bytes**

enumerator **kUWB_MacAddressMode_8bytes**

enum **UWB_ScheduledMode_t**

Enumeration lists out the scheduled Mode values.
Values:

enumerator **kUWB_ScheduledMode_ContentionBased**

Contention based Ranging Scheduling

enumerator **kUWB_ScheduledMode_TimeScheduled**

Time based Ranging Scheduling

enumerator **kUWB_ScheduledMode_HybridBased**

Hybrid based Scheduling

enum **UWB_RadarMode_t**

Enumeration lists out the Radar Mode values.

- 0x01: Medium distance, e.g. used for vital sign detection;
- 0x02: Close distance, e.g. used for hand gesture recognition
- 0x03: Far distance, e.g. used for presence detection
- 0x04 - RFU
- 0x05: Medium distance, e.g. used for static object support
- 0x06: Far distance, e.g. used for static object support

Values:

enumerator **kUWB_RadarMode_Medium_Distance**

enumerator **kUWB_RadarMode_Close_Distance**

All information provided in this document is subject to legal disclaimers.
© 2026 NXP B.V. All rights reserved.

enumerator **kUWB_RadarMode_Far_Distance**

enumerator **kUWB_RadarMode_Static_Medium_Distance**

enumerator **kUWB_RadarMode_Static_Far_Distance**

enumerator **kUWB_RadarMode_Test_Isolation**

enum **UWB_MacAddPhaseTypeCtrl_t**

Enumeration lists out the control values of PhaseList.
b0 : MAC addressing mode of the Controller of the Phase b1 : Phase type
Values:

enumerator **kUWB_CtrlShortMacAdressAndPhaseCfp**

enumerator **kUWB_CtrlExtMacAdressAndPhaseCfp**

enumerator **kUWB_CtrlShortMacAdressAndPhaseCap**

enumerator **kUWB_CtrlExtMacAdressAndPhaseCap**

enum **phSlotBitmap_t**

Enumeration to Set the Slot Bitmap.

- [b3-b1]
  - 0 = 8 ranging slots
  - 1 = 16 ranging slots
  - 2 = 32 ranging slots
  - 3 = 64 ranging slots
  - 4 = 128 ranging slots
  - 5 = 256 ranging slots
  - 6-7 = RFU

Values:

enumerator **ranging_slots_8**

enumerator **ranging_slots_16**

enumerator **ranging_slots_32**

enumerator **ranging_slots_64**

enumerator **ranging_slots_128**

enumerator **ranging_slots_256**

enum **eMulticastControllerActions_t**

Enumeration to list out the Actions of the Controller's Multicast List.
Values:

enumerator **kUWB_AddControlee**

enumerator **kUWB_DelControlee**

enumerator **KUWB_Add16BSubSessionKey**

enumerator **KUWB_Add32BSubSessionKey**

enum **eSelectLogicalLinkModes_t**

Enumerator lists out the types of Logical Link Modes.
Values:

enumerator **kUwb_ConnectionLessNonSecure**

All information provided in this document is subject to legal disclaimers.
© 2026 NXP B.V. All rights reserved.

enumerator **kUwb_ConnectionLessSecure**

enumerator **kUwb_ConnectionOrientedNonSecure**

enumerator **kUwb_ConnectionOrientedSecure**

enumerator **kUwb_ConnectionLessUWBS**

enumerator **kUwb_ConnectionOrientedUWBS**

enumerator **kUwb_ProprietarySecure**

enumerator **kUwb_ProprietarySecureUWBS**

enum **rLIGetParam_controlField_t**

Enemeration Lists out the Logical Link get param Control field values.
Values:

enumerator **LL_GET_PARAM_CF_SDU_SIZE**

Maximum LL SDU size

enumerator **LL_GET_PARAM_CF_PDU_SIZE**

Maximum LL PDU size

enumerator **LL_GET_PARAM_CF_TxW**

Transmit Window Size, TxW

enumerator **LL_GET_PARAM_CF_RxW**

Receive Window Size, RxW

enumerator **LL_GET_PARAM_CF_REP_CNT_MAX**

Repetition count Max

enumerator **LL_GET_PARAM_CF_LINK_TO**

Link TO

enumerator **LL_GET_PARAM_CF_PORT**

PORT

enum **eLICreate_StatusCodes_t**

Enumeration Lists out the Logical Link create status codes.
Values:

enumerator **LOGICAL_LINK_ACCEPTED**

enumerator **LOGICAL_LINK_REJECTED**

enumerator **LOGICAL_CO_LINK_CONNECTED**

enumerator **LOGICAL_LINK_ERROR**

enum **eLIClose_statusCodes_t**

Enumeration Lists out the Logical Link Close Status Codes.
Values:

enumerator **kUWB_LIRemoteTerm**

enumerator **kUWB_LITimeOut**

enumerator **kUWB_LITransmissionError**

enumerator **kUWB_LITermSE**

All information provided in this document is subject to legal disclaimers.
© 2026 NXP B.V. All rights reserved.

enumerator **kUWB_LlTermUnknown**

enumerator **kUWB_LlHostTerm**

enum **eUwb_CapsDeviceType_t**

Enumeration lists out the capability device type values.
Values:

enumerator **kUWB_CapsDeviceType_Controller**

FiRa Controller

enumerator **kUWB_CapsDeviceType_Controlee**

FiRa Controlee

enumerator **kUWB_CapsDeviceType_CCC_Controller**

CCC Controller

enumerator **kUWB_CapsDeviceType_CCC_Controlee**

CCC Controlee

enumerator **kUWB_CapsDeviceType_Aliro_User**

Aliro User

enumerator **kUWB_CapsDeviceType_Aliro_Reader**

Aliro Reader

struct **phUwbSessionData_t**

#include <UwbApi_Types.h>
Structure for storing Session Data.

Public Members

uint32_t **sessionHandle**

Session Handle

uint8_t **session_type**

Session Type

uint8_t **session_state**

Session State

struct **phUwbSessionsContext_t**

#include <UwbApi_Types.h>
Structure for storing Session Context.

Public Members

uint8_t **status**

Status

uint8_t **sessioncnt**

[Input/Output] Session Count

phUwbSessionData_t ***pUwbSessionData**

Pointer to Session Data

struct **phRangingMesr_t**

#include <UwbApi_Types.h>

All information provided in this document is subject to legal disclaimers.
© 2026 NXP B.V. All rights reserved.

Structure lists out the ranging measurement information. Ranging measurements array for TWR.

Public Members

uint8_t **mac_addr**[EXT_MAC_ADDR_LEN]

Mac address of the participating device, addr can be of short 2 byte or extended 8 byte modes

uint8_t **status**

Status

uint8_t **nLos**

Indicates if the ranging measurement was in Line of sight or non-line of sight

uint8_t **aoa_azimuth_FOM**

AOA Azimuth FOM

uint8_t **aoa_elevation_FOM**

AOA elevation FOM

uint8_t **aoa_dest_azimuth_FOM**

AOA destination azimuth FOM

uint8_t **aoa_dest_elevation_FOM**

AOA destination elevation FOM

uint8_t **slot_index**

Status to the slot number within the ranging round In time schedule mode, in case of a failure, this field indicates the slot number within the ranging round where the failure has occurred In contention-based ranging mode, this field can be used to indicate the slot selected by the controlee device. If the Slot Index field is not used, then it shall be set to 0.

uint8_t **rssi**

Rssi

uint16_t **distance**

Distance in centimeters

int16_t **aoa_azimuth**

AOA Azimuth

int16_t **aoa_elevation**

AOA Elevation

int16_t **aoa_dest_azimuth**

AOA destination azimuth

int16_t **aoa_dest_elevation**

AOA destination elevation

struct **phRxInfoMesr_t**

#include <UwbApi_Types.h>
Structure lists out the Vendor speicifc Rx Antenna Info for AoA Measurements.

Public Members

uint8_t **rxMode**

RX mode

uint8_t **num_of_rx_antennaRxInfo**

All information provided in this document is subject to legal disclaimers.
© 2026 NXP B.V. All rights reserved.

Number of RX antenna to follow

uint8_t **rx_antennaIdRxInfo**[8]

RX antenna pair

struct **phRxInfoDebugNtf_t**

#include <UwbApi_Types.h>
Structure lists out the Vendor speicifc Rx Antenna Info For Debug Notifications.

Public Members

uint8_t **rxModeDebugNtf**

If message control with b1 of octet[2] = 0

uint8_t **numOfRframes**

If message control with b1 of octet[2] = 1

uint8_t **num_of_rx_antennaDebugNtf**

Number of RX antenna to follow

uint8_t **rx_antennaIdDebugNtf**[8]

RX antenna pair

struct **phAoaPdoaMesr_t**

#include <UwbApi_Types.h>
Structure lists out the Vendor speicifc information for AoA / PDoA measurements per RX.

Public Members

int16_t **angleOfArrival**

Angle of arrival

int16_t **pdoa**

Phase difference of arrival

uint16_t **pdoaIndex**

Phase difference of arrival index in the whole CIR

uint8_t **aoaFovFlag**

This parameter indicates the presence or absence of the peer device in FoV for user configured Horizontal Rx Antenna Pair as defined in 'AZIMUTH_FIELD_OF_VIEW' 0x00: Peer device is not present in FoV 0x01: Peer device is present in FoV This field would be displayed only when NXP Specific Data Type is 0xA0

struct **phSnrPathIndexMesr_t**

#include <UwbApi_Types.h>
Structure lists out the Vendor speicifc Rx Antenna information SNRFirst / SNRMain /FirstIndex : Main Index measurements per RX entry.

Public Members

uint8_t **rxSnrFirstPath**

Signal-to-Noise (SNR) of the First Path in dB

uint8_t **rxSnrMainPath**

Signal-to-Noise (SNR) of the Main Path in dB

int16_t **rx_FirstPathIndex**

First path index in the whole CIR

All information provided in this document is subject to legal disclaimers. © 2026 NXP B.V. All rights reserved.

int16_t **rx_MainPathIndex**

Main path index in the whole CIR

int16_t **snrTotal**

SNR Total of configured RX in Q8.8 dB

int16_t **rssi**

Received Signal Strength Indicator of RX in dB

struct **phTwoWayRangingVsMesr_t**

#include <UwbApi_Types.h>
Structure lists out the TWR Vendor speicifc measurements for each responder.

Public Members

phAoaPdoaMesr_t **aoaPdoaMesr_twr**[8]

AoA / PDoA measurements per RX

phSnrPathIndexMesr_t **snrPathIndexMesr_twr**[8]

SNRFirst / SNRMain / FirstIndex / Main Index measurements per RX

uint16_t **distance_2**

Range or distance between the device and target Set to 0 for Initiator: distance-2 is reported using the RFM pair on responder only in AOA_RFM mode

uint32_t **distance_mm**

Distance between the device and target (resolution of 0.01 cm)

struct **phTwoWayRangingVsData_t**

#include <UwbApi_Types.h>
Structure lists out the Vendor speicifc information for TWR.

Public Members

uint8_t **wifiCoExStatus**

Vendor Specific Data WiFi co-existence status

uint8_t **txAntennaInfo**

Tx Antenna Info
Note:
This field shall be shared only when TX_ANT_INFO_EN is enabled

phRxInfoMesr_t **rxInfoMesr_twr**

Rx Antenna Info for AoA Measurements

phRxInfoDebugNtf_t **rxInfoDebugNtf_twr**

Rx Antenna Info For Debug Notifications

phTwoWayRangingVsMesr_t **vsMesr**[12]

twr vs measurement for each responders

bool **isTxAntInfoPreset**

Check if TX antenna info is present in message_control

bool **isDistanceMmPresent**

Check if Distance_mm is present in message_control

bool **isRxAntDebugInfoPresent**

All information provided in this document is subject to legal disclaimers.
© 2026 NXP B.V. All rights reserved.

Check if Rx antenna Debug Info reported for all RFrames

struct **phOneWayRangingVsMesr_t**

#include <UwbApi_Types.h>
Structure lists out the OWR Vendor speicifc measurements for each responder.

Public Members

phAoaPdoaMesr_t **aoaPdoaMesr_owr**[8]

AoA / PDoA measurements per RX

int16_t **rssi**

RSSI in dB

struct **phOneWayRangingVsData_t**

#include <UwbApi_Types.h>
Structure lists out the Vendor speicifc information for OWR.

Public Members

phRxInfoMesr_t **rxInfoMesr_owr**

Vendor Specific Data Rx Antenna Info for AoA Measurements

phOneWayRangingVsMesr_t **vsMesr**[MAX_NUM_OF_TDOA_MEASURES]

OWR vs measurment for each responders

struct **phRangingMesrTdoa_t**

#include <UwbApi_Types.h>
Structure lists out the TDoA ranging measurement information for one way ranging.

Public Members

uint8_t **mac_addr**[EXT_MAC_ADDR_LEN]

Mac address of the participating device, addr can be of short 2 byte or extended 8 byte modes

uint8_t **status**

Status

uint8_t **message_control**

This signifies the presence of Device ID, Rx and Tx timestamps

uint8_t **frame_type**

Type of frame

uint8_t **nLos**

Indicates if the ranging measurement was in Line of sight or non-line of sight

int16_t **aoa_azimuth**

AOA Azimuth

uint8_t **aoa_azimuth_FOM**

AOA Azimuth FOM

int16_t **aoa_elevation**

AOA elevation

uint8_t **aoa_elevation_FOM**

AOA elevation FOM

All information provided in this document is subject to legal disclaimers.
© 2026 NXP B.V. All rights reserved.

uint32_t **frame_number**

Number received in the Payload of the blink UTM from the UT-Tag or UTM from the UT-Synchronization Anchor.

uint8_t **rx_timestamp**[(0x08)]

Local RX timestamp of the received UWB RFRAME as measured by the device that is sending this SESSION_INFO_NTF. The unit is 2^(-7) of the 499.2 MHz chipping period, which is approximately 15.65ps.

uint8_t **ul_tdoa_device_id**[(0x08)]

UL-TDoA Device ID of the sender of the received UTM as listed in the Blink UTM from the UT-Tag or the Synchronization UTM from the UT-Synchronization Anchor.

uint8_t **tx_timestamp**[(0x08)]

TX timestamp of the UWB RFRAME as listed in the Blink UTM from the UT-Tag or the Synchronization UTM from UT-Synchronization Anchor. The unit is 2^(-7) of the 499.2 MHz chipping period, which is approximately 15.65ps.

struct **phTdoaRangingVsData_t**

#include <UwbApi_Types.h>

Structure lists out the Vendor speicifc information for tdoa.

Public Members

uint8_t **vendorExtLength**

Vendor Specific Data Vendor extension length for TDOA Measurements

int16_t **rssi_rx1**

RSSI RX1

int16_t **rssi_rx2**

RSSI RX2

uint8_t **noOfPdoaMeasures**

No of pdoA measurement

int16_t **pdoa**[8]

Estimation of phase difference for antenna pair N

uint16_t **pdoaIndex**[8]

CIR Index estimate at which pdoa has been detected

uint16_t **message_control_extension**

Msg Control bitfiled for specific extensions If bit0 is 1 then it will be RX3 extensions, others RFU

uint8_t **rx3_ext_info_len**

Number of bytes for RX3 related extensions

int16_t **rssi_rx3**

RSSI RX3

union **RANGING_MEAS**

#include <UwbApi_Types.h>

Union for TWR ranging and TDoA ranging measurement information.

Public Members

phRangingMesr_t **range_meas_twr**[12]

Ranging measurements array

All information provided in this document is subject to legal disclaimers.
© 2026 NXP B.V. All rights reserved.

phRangingMesrTdoa_t **range_meas_tdoa**[MAX_NUM_OF_TDOA_MEASURES]

Ranging measurements TDoA array One Way Ranging Ntf

union **VENDORSPECIFIC_MEAS**

#include <UwbApi_Types.h>
Union for vendor specific information of TWR ranging and TDoA ranging.

Public Members

phTwoWayRangingVsData_t **twr**

Ranging measurements array

phOneWayRangingVsData_t **owr_aoa**

One Way Ranging Ntf

phTdoaRangingVsData_t **tdoa**

One Way Ranging TDoA Ntf

struct **phRangingData_t**

#include <UwbApi_Types.h>
Structure lists out the ranging notification information.

Public Members

uint8_t **rcr_indication**

RCR Indication

uint8_t **ranging_measure_type**

Ranging Measurement Type
- 0x00 = One-Way Ranging Measurement (OWR)
- 0x01 = Two-Way Ranging Measurement (SS-TWR, DS-TWR)
- 0x02 = DL-TDoA Ranging Measurement v1 (OWR DL-TDOA)
- 0x03 = OWR for AoA Measurement
- 0x04 = Reserved for CCC
- 0x05 = Reserved for CCC
- 0x06 = DL-TDoA Ranging Measurement_ Vv2 (OWR DL-TDoA)
- 0x07-0xFF = RFU

Note:
OWR for AoA Measurement is allowed when RANGING_ROUND_USAGE is set to “OWR for AoA Measurement” and DEVICE_ROLE is set to “Observer”

uint8_t **mac_addr_mode_indicator**

Mac addr mode indicator,
- 0: short 2 byte,
- 1: extended 8 byte mode

uint8_t **no_of_measurements**

Number of ranging measurements

uint32_t **seq_ctr**

keep track of the notifications

uint32_t **sessionHandle**

Session Handle of the ranging round

uint32_t **curr_range_interval**

All information provided in this document is subject to legal disclaimers.
© 2026 NXP B.V. All rights reserved.

Current ranging interval setting in milli seconds

uint32_t **sessionHandle_of_primary_session**

Session Handle of Primary Session

RANGING_MEAS **ranging_meas**

Ranging measures array

uint8_t **vs_data_type**

Vendor specific data type

uint16_t **vs_length**

Vendor specific data Length

VENDORSPECIFIC_MEAS **vs_data**

Vendor specific data

uint32_t **message_control**

Message control

uint8_t **wifiCoExStatusCode**

Status code for WLAN during ranging RR

uint32_t **antenna_pairInfo**

Antenna Rx Configuration information used in current ranging round

uint8_t **authInfoPrsen**

Indicator for presence of Authentication Tag

uint8_t **authenticationTag**[(0x10)]

Authentication Tag

struct **phNewRoleChangeNtf_t**

#include <UwbApi_Types.h>
Structure lists out the ranging notification information.

Public Members

uint32_t **sessionHandle**

Session Handle of the ranging round

uint8_t **new_role**

New Role 0x00 - Initiator 0x01 - Responder

struct **phCsaPdoaMeasurements_t**

#include <UwbApi_Types.h>
Structure lists out the PDoA Measurements for CCC/CSA Range Data Ntf .

Public Members

int16_t **pdoa**

Estimation of phase difference in degrees from antenna pair [1…N]

uint16_t **pdoaIndex**

CIR index estimate at which PDoA [1…N] has been detected

struct **phCsaAntennaPairInfo_t**

#include <UwbApi_Types.h>

All information provided in this document is subject to legal disclaimers.
© 2026 NXP B.V. All rights reserved.

Structure lists out the Antenna Pair Info for CCC/CSA Range Data Ntf .

Public Members

uint8_t **configMode**

Antenna Rx Configuration information used in current ranging round Configuration Mode as per ANTENNAS_CONFIGURATION_RX supported configuration modes are 0 ,1, 5 and 6 0 : Configuration Mode 0: ToF Only Mode 1 : Configuration Mode 1: Configuration Mode for 3D/2D AoA, test/loopback session and implicit ToA mode usecase 5 : Configuration Mode 5: Configuration Mode for CSA ToA mode use case 6 : Configuration Mode 6: Configuration Mode for CSA AoA mode use case

uint8_t **antPairId1**

Config mode(s) 0: Antenna ID 1 as per ANTENNA_RX_IDX_DEFINE Config mode(s) 1: Antenna Pair ID as per ANTENNAS_RX_PAIR_DEFINE Config mode(s) 5: Antenna ID as per ANTENNA_RX_IDX_DEFINE mapped to current active RR Config mode(s) 6: Antenna Pair ID as per ANTENNAS_RX_PAIR_DEFINE mapped to current active RR

uint8_t **antPairId2**

Config mode(s) 0: Antenna ID 2 as per ANTENNA_RX_IDX_DEFINE Config mode(s) 1: Antenna Pair ID as per ANTENNAS_RX_PAIR_DEFINE Config mode(s) 5: Antenna ID as per ANTENNA_RX_IDX_DEFINE mapped to current active RR Config mode(s) 6: Antenna Pair ID as per ANTENNAS_RX_PAIR_DEFINE mapped to current active RR

uint8_t **rfu**

RFU

struct **phCsaRssiMeasurements_t**

#include <UwbApi_Types.h>

Structure lists out the RSSI Measurements for CCC/CSA Range Data Ntf . Helios1 (N=2 and M=2): { RSSI_RX1 -2 Octets RSSI_RX2 -2 Octets } Repeat the same for FINAL frame.

Helios2 (N=2 and M=3): { RSSI_RX1 -2 Octets RSSI_RX2 -2 Octets RSSI_RX3 -2 Octets } – Repeat the same for FINAL frame

Public Members

int16_t **rssi_rx1**

RSSI Measurement from RX1 It is signed value expressed in Q8.8 format

int16_t **rssi_rx2**

RSSI Measurement from RX2 It is signed value expressed in Q8.8 format

int16_t **rssi_rx3**

RSSI Measurement from RX3 It is signed value expressed in Q8.8 format

struct **phCsaSnrMeasurements_t**

#include <UwbApi_Types.h>

Structure lists out the SNR Measurements for CCC/CSA Range Data Ntf .

Public Members

uint8_t **slotIndexAndAntennaMap**

Slot index and Antenna mapping details bits[5:0] - RFRAME slot index bits[7:6] for H1 bit[6] - RFU bit[7] - 0: RX1, 1: RX2 for H1 bits[7:6] : For H2

- 0b00: RXC
- 0b01: RXB
- 0b10: RFU
- 0b11: RXA

uint8_t **snrFirstPath**

All information provided in this document is subject to legal disclaimers. © 2026 NXP B.V. All rights reserved.

Signal-to-Noise Ratio (SNR) of the First path and it is reported as unsigned value in dB

uint8_t **snrMainPath**

Signal-to-Noise Ratio (SNR) of the main path and it is reported as unsigned value in dB

uint16_t **snrTotal**

Signal-to-Noise Ratio (SNR) of configured RX

struct **phCccRangingData_t**

#include <UwbApi_Types.h>

Structure lists out the CCC ranging notification information.

Public Members

uint32_t **sessionHandle**

Session Handle of the CCC ranging round

uint8_t **rangingStatus**

Ranging Status of the CCC Session, refer eCccRangingStatus

uint32_t **stsIndex**

STS Index

uint16_t **rangingRoundIndex**

Ranging Round Index

uint16_t **distance**

Distance between Initiator and Anchor in cm

uint8_t **uncertanityAnchorFom**

Ranging timestamp uncertainty of controllee

uint8_t **uncertanityInitiatorFom**

Ranging timestamp uncertainty of controller

uint8_t **ccmTag**[8]

CCM Tag

int16_t **aoa_azimuth**

AOA Azimuth

uint8_t **aoa_azimuth_FOM**

AOA Azimuth FOM

int16_t **aoa_elevation**

AOA Elevation

uint8_t **aoa_elevation_FOM**

AOA elevation FOM

phCccAntennaPairInfo_t **antenna_pair_info**

Antenna Pair Info

uint8_t **noOfPdoaMeasures**

Number of PDoA Measurements

phCccPdoaMeasurements_t **pdoaMeasurements**[255]

PDoA Measurements

All information provided in this document is subject to legal disclaimers.

© 2026 NXP B.V. All rights reserved.

uint8_t **noOfRssiMeasurements**

Number of RFrames used for RSSI measurements This field shall be set 0 when SWAP_ANTENNA_PAIR_3D_AOA parameter is set to 0x00

phCccRssiMeasurements_t **rssiMeasurements**[2U]

RSSI Measurements

uint8_t **noOfSnrMeasurements**

Number of SNR Measurements This field shall be set 0 and SNR Measurement field will not be present.

phCccSnrMeasurements_t **snrMeasurements**[6U]

SNR Measurements

struct **phCsaRangingData_t**

#include <UwbApi_Types.h>
Structure lists out the CSA ranging notification information.

Public Members

uint32_t **sessionHandle**

Session Handle of the CSA ranging round

uint8_t **rangingStatus**

Ranging Status of the CSA Session, refer eCccRangingStatus

uint32_t **stsIndex**

STS Index

uint16_t **rangingRoundIndex**

Ranging Round Index

uint16_t **blockIndex**

Index of the Ranging block for which the following measurement result applies

uint16_t **distance**

Distance between Initiator and Anchor in cm

uint8_t **uncertanityAnchorFom**

Ranging timestamp uncertainty of controlee

uint8_t **uncertanityInitiatorFom**

Ranging timestamp uncertainty of controller

uint8_t **ccmTag**[8]

CCM Tag

int16_t **aoa_azimuth**

AOA Azimuth

uint8_t **aoa_azimuth_FOM**

AOA Azimuth FOM

int16_t **aoa_elevation**

AOA Elevation

uint8_t **aoa_elevation_FOM**

AOA elevation FOM

All information provided in this document is subject to legal disclaimers.
© 2026 NXP B.V. All rights reserved.

phCsaAntennaPairInfo_t **antenna_pair_info**

Antenna Pair Info

uint8_t **noOfPdoaMeasures**

Number of PDoA Measurements

phCsaPdoaMeasurements_t **pdoaMeasurements**[255]

PDoA Measurements

uint8_t **noOfRssiMeasurements**

Number of RFrames used for RSSI measurements This field shall be set 0 when SWAP_ANTENNA_PAIR_3D_AOA parameter is set to 0x00

phCsaRssiMeasurements_t **rssiMeasurements**[2U]

RSSI Measurements

uint8_t **noOfSnrMeasurements**

Number of SNR Measurements This field shall be set 0 and SNR Measurement field will not be present.

phCsaSnrMeasurements_t **snrMeasurements**[6U]

SNR Measurements

struct **phCccRangingParams_t**

#include <UwbApi_Types.h>
Structure lists out the CCC Mandatory App Configs.

Public Members

uint8_t **deviceType**

Device Type, 0x00: Controlee, 0x01: Controller, 0x02: Advertiser, 0x03: Observer

uint8_t **noOfControlees**

Number of Controlees(N), 1<=N<=8 (Default is 1)

uint8_t **responderSlotIndex**

This parameter is used to choose responder index in two way ranging.
Note:
It is not applicable to controller.

uint16_t **slotDuration**

Unsigned integer that specifies duration of a ranging slot in the unit of RSTU

uint8_t **hoppingMode**

This parameter is used to enable/disable the hopping. eCCC_HopModes_t

struct **phRangingParams_t**

#include <UwbApi_Types.h>
Structure lists out the mandatory configurations to be set for ranging.

Public Members

uint8_t **deviceRole**

Device Role
- kUWB_DeviceRole_Responder = 0,
- kUWB_DeviceRole_Initiator = 1,
- kUWB_DeviceRole_UT_Sync_Anchor = 2,
- kUWB_DeviceRole_UT_Anchor = 3,

All information provided in this document is subject to legal disclaimers. © 2026 NXP B.V. All rights reserved.

- kUWB_DeviceRole_UT_Tag = 4,
- kUWB_DeviceRole_Advertiser = 5,
- kUWB_DeviceRole_Observer = 6,
- kUWB_DeviceRole_DlTDoA_Anchor = 7,
- kUWB_DeviceRole_DlTDoA_Tag = 8,

uint8_t **multiNodeMode**

Multi Node Mode,
- 0x00: O2O(One to one),
- 0x01: O2M(One to Many),
- 0x02 - 0xFF: Reserved

uint8_t **macAddrMode**

Device Mac Address mode 0:2 bytes,1:8 bytes mac addr with 2 bytes in header, 2: 8 bytes in mac addr and header

uint8_t **deviceMacAddr**[EXT_MAC_ADDR_LEN]

Device Mac Address, 2 bytes or 8 bytes addr is supported.

uint8_t **deviceType**

Device Type
- 0x00: Controlee
- 0x01: Controller
- 0x02: Advertiser
- 0x03: Observer

uint8_t **rangingRoundUsage**

Ranging Round Usage

uint8_t **scheduledMode**

Scheduled Mode

struct **phTxPulseShapeConfig_t**

#include <UwbApi_Types.h>
Structure lists out the preamble pulse shape config.

Public Members

uint8_t **shape_id**

Preamble pulse shape id

uint8_t **payload_tx_shape_id**

Payload tx pulse shape id

uint8_t **sts_shape_id**

STS Tx pulse shape id

uint8_t **dac_stage_cofig**

DAC Stage config

struct **AccessoryUwbConfigDataContent_t**

#include <UwbApi_Types.h>
Structure lists out the configurations to be get from Accessory Config Data.

Public Members

uint8_t **length**

All information provided in this document is subject to legal disclaimers. © 2026 NXP B.V. All rights reserved.

Total length

uint8_t **uwb_spec_ver_major**[2]

Specification Major version

uint8_t **uwb_spec_ver_minor**[2]

Specification Minor version

uint8_t **manufacturer_id**[4]

Manufacture id for device specific

uint8_t **model_id**[4]

Model id for device specific

uint8_t **mw_version**[4]

MW version

uint8_t **ranging_role**

Device Role

uint8_t **device_mac_addr**[SHORT_MAC_ADDR_LEN]

Source mac address

uint8_t **clock_drift**[2]

CLock frequency drift value

struct **UwbPhoneConfigData_t**

#include <UwbApi_Types.h>
Structure lists our the configurations to be loaded on the device sent from the phone counterpart.

struct **UwbDeviceConfigData_t**

#include <UwbApi_Types.h>
Structure lists out the configurations to be get from Accessory Config Data.

Public Members

uint8_t **spec_ver_major**[2]

Specification Major version

uint8_t **spec_ver_minor**[2]

Specification Minor version

uint8_t **chip_id**[2]

UWB Chip identifier

uint8_t **chip_fw_version**[2]

UWB Chip firmware version

uint8_t **mw_version**[3]

MW version

uint32_t **supported_config_ids**

Range of supported profiles by the device

uint8_t **ranging_role**

Device Role

uint8_t **device_mac_addr**[SHORT_MAC_ADDR_LEN]

All information provided in this document is subject to legal disclaimers.
© 2026 NXP B.V. All rights reserved.

Source mac address

struct **UserAccessoryConfigData_iOS_t**

#include <UwbApi_Types.h>
Structure lists out the mandatory configurations to be set for User Accessory Config Data.

struct **phUwbProfileInfo_t**

#include <UwbApi_Types.h>
Structure lists out Profile information.

Public Members

uint32_t **sessionHandle**

Session Handle is out param

uint8_t **mac_addr**[SHORT_MAC_ADDR_LEN]

Source mac address

eUWB_ProfileID_t **profileId**

Profile ID

uint8_t **deviceRole**

Device Role
- 0x00: Responder
- 0x01: Initiator
- 0x02: Master Anchor

uint8_t **deviceType**

Device Type, 0x00: Controlee, 0x01: Controller

struct **phUwbDataTransmit_t**

#include <UwbApi_Types.h>
Structure lists out the data control transmit notification.

Public Members

uint32_t **transmitNtf_connectionId**

Connection ID If the msb of MSB is 0b1, then the connectionId indicates a sessionHandle If the msb of MSB is 0b0, then the connectionId indicates a LL_Connect_Id.

uint16_t **transmitNtf_sequence_number**

Sequence number

uint8_t **transmitNtf_status**

Status

uint8_t **transmitNtf_txcount**

Tx count

struct **phUwbDataCredit_t**

#include <UwbApi_Types.h>
Structure lists out the data credit notification.

Public Members

uint32_t **connectionId**

Connection ID If the msb of MSB is 0b1, then the connectionId indicates a sessionHandle If the msb of MSB is 0b0, then the connectionId indicates a LL_Connect_Id.

All information provided in this document is subject to legal disclaimers.
© 2026 NXP B.V. All rights reserved.

uint8_t **credit_availability**

Credit availability

struct **phUwbRadarcirNtf_t**

#include <UwbApi_Types.h>
Structure lists out the Radar Cir notification.

Public Members

uint16_t **num_cirs**

Num of Cirs

uint8_t **cir_taps**

Cir Taps

uint8_t **rfu**

Rfu

uint16_t **cir_len**

Length of the Cir

uint8_t ***cirdata**

Cir data

struct **phUwbRadarTestIsoNtf_t**

#include <UwbApi_Types.h>
Structure lists out the Test Radar Antenna Isolation.

Public Members

uint8_t **antenna_tx**

Tx Antenna select

uint8_t **antenna_rx**

Rx Antenna select

uint16_t **anteena_isolation**

Radar type

struct **phUwbRadarPresenceDetectNtf_t**

#include <UwbApi_Types.h>
Structure lists out the Test Radar Presence detection.

Public Members

bool **presence_detected**

A boolean value indicating whether a presence has been detected

uint8_t **presence_detection_mode**

Representing the presence detection mode.

uint8_t **number_of_detections**

Representing the number of detections N (max. 5).

uint16_t **detection_distance**[5]

Distance of detection in cm, else set to 0xFFFF

int8_t **detection_aoa**[5]

All information provided in this document is subject to legal disclaimers.
© 2026 NXP B.V. All rights reserved.

AoA of detection as signed degree (int8_t format) if a second receiver is configured, else set to 0xFF

uint8_t **detection_track_id**[5]

Track ID for each target (previously RFU)

uint32_t **detection_value**[5]

value (threshold), set to 0xFFFFFFFF if not set

union **RADAR_MEAS**

#include <UwbApi_Types.h>
Union for Radar and Anteena isolation measurement information.

Public Members

phUwbRadarcirNtf_t **radr_cir**

Radar CIR notification Structure

phUwbRadarTestIsoNtf_t **radar_tst_ntf**

Radar CIR Antenna test isolation Structure

phUwbRadarPresenceDetectNtf_t **radar_presence_detect_ntf**

Radar Presence detection notificaiton Structure

struct **phUwbRadarNtf_t**

#include <UwbApi_Types.h>
Structure lists out the Radar notificaiton.

Public Members

uint32_t **sessionHandle**

Session Handle

uint8_t **radar_status**

Status of the radar

uint8_t **radar_type**

Radar type

RADAR_MEAS **radar_ntf**

Radar measures

struct **phCtlrPhaseList_t**

#include <UwbApi_Types.h>
Structure for storing List of Phases of Controller.

Public Members

uint32_t **phase_sessionHandle**

Secondary Session Handle

uint16_t **start_slot_index**

Start Slot Index

uint16_t **end_slot_index**

End Slot Index

uint8_t **control**

All information provided in this document is subject to legal disclaimers.
© 2026 NXP B.V. All rights reserved.

b0 : MAC addressing mode of the Controller of the Phase b0=0 for short MAC address b0=1 for extended MAC address b1 : Phase type b1=0 for CFP b1=1 for CAP

uint8_t **mac_addr**[EXT_MAC_ADDR_LEN]

MAC address of the participating device in the current phase

struct **phControllerHusSessionConfig_t**

#include <UwbApi_Types.h>
Structure for storing Controller HUS Session Configurations.

Public Members

uint32_t **sessionHandle**

Primary Session Handle

uint8_t **phase_count**

Number of Phases

phCtlrPhaseList_t ***phase_list**

Phase List It shall have memory allocation for phase_count number of counts.

struct **phCleePhaseList_t**

#include <UwbApi_Types.h>
Structure for storing List of Phases of Controlee.

Public Members

uint32_t **phase_sessionHandle**

Secondary Session Handle

struct **phControleeHusSessionConfig_t**

#include <UwbApi_Types.h>
Structure for storing Controlee HUS Session Configurations.

Public Members

uint32_t **sessionHandle**

Primary Session Handle

uint8_t **phase_count**

Number of Phases

phCleePhaseList_t ***phase_list**

Phase List It shall have memory allocation for phase_count number of counts.

struct **phDataTxPhaseMngList_t**

#include <UwbApi_Types.h>
Structure to store Data Transfer Phase Management List.

Public Members

uint8_t **mac_addr**[EXT_MAC_ADDR_LEN]

MAC address for which Data Tx slots are configured

uint8_t **slot_bitmap**[32]

Slot Bitmap

uint8_t **stop_data_transfer**

Stop Data Transfer 0x00: Data Transfer will be continued 0x01: Data Transfer will be stopped.

All information provided in this document is subject to legal disclaimers. © 2026 NXP B.V. All rights reserved.

If this field is set to 0x01 for a Controlee MAC Address, then the UWBS shall transmit this entry of the DTPML of the DTPCM at least once (recommended to send it in at least two subsequent blocks). Afterwards the Controller UWBS may completely remove the Controlee entry from the DTPML of the DTPCM. If this field is set to 0x01 for the Controller MAC Address, then the UWBS shall reject the command by sending STATUS_REJECTED

struct **phDataTxPhaseConfig_t**

#include <UwbApi_Types.h>
structure to store Data Transfer Phase Configurations.

Public Members

uint32_t **dtpcm_SessionHandle**

Session Handle to which Data Transfer phase to be configured

uint8_t **dtpcm_Repetition**

Data Transfer Phase Control Message Repetition

uint8_t **dataTransferCtrl**

Data Transfer Control

uint8_t **dtpml_size**

Data Transfer Phase Management List Size

phDataTxPhaseMngList_t ***dtpml**

Data Transfer Phase Management List

struct **phDataTxPhaseCfgNtf_t**

#include <UwbApi_Types.h>
Structure to store Data Transfer Phase Config notification values.

Public Members

uint32_t **sessionHandle**

Session Handle to which the DataTx phase is configured

uint8_t **status**

Data Tx phase Status

struct **phUwbappContext_t**

#include <UwbApi_Types.h>
Structure lists out the UWB-IoT init context.

Public Members

tUwbApi_AppCallback ***pCallback**

Standalone mode callback function

tUwbApi_TMLDataCallback ***pTmlCallback**

Application specific callback function

phUwbFWImageContext_t **fwImageCtx**

FW Image context

struct **phGenericError_t**

#include <UwbApi_Types.h>
Structure lists out the Generic Error notification.

Public Members

All information provided in this document is subject to legal disclaimers.
© 2026 NXP B.V. All rights reserved.

uint8_t **status**

Status

struct **phUwbSessionInfo_t**

#include <UwbApi_Types.h>
Structure lists out session information.

Public Members

uint32_t **sessionHandle**

Session Handle

uint8_t **state**

Session state

uint8_t **reason_code**

Reason code

struct **phControleeList_t**

#include <UwbApi_Types.h>
Structure List out Controlee info.

Public Members

uint16_t **short_address**

Short address

uint32_t **subsession_id**

Sub Session Handle

uint8_t **subsession_key**[32]

Controlee specific Sub-session Key 16/32 Bytes

struct **phMulticastControleeListContext_t**

#include <UwbApi_Types.h>
Structure for storing Multicast Controlee List Context.

Public Members

uint8_t **action**

Action

uint8_t **no_of_controlees**

Controlee Count

phControleeList_t **controlee_list**[8]

Controlee List

struct **phMulticastControleeStatusList_t**

#include <UwbApi_Types.h>
Structure for storing Multicast Controlee Status List Context.

struct **phMulticastControleeListRspContext_t**

#include <UwbApi_Types.h>
Structure for storing Multicast Controlee List Ntf Context.

struct **phMulticastControleeListNtfContext_t**

All information provided in this document is subject to legal disclaimers.
© 2026 NXP B.V. All rights reserved.

#include <UwbApi_Types.h>
Structure for storing Multicast Controlee List Ntf Context.

struct **phBlinkDataTxNtfContext_t**

#include <UwbApi_Types.h>
Structure for storing Blink Data Tx Ntf Context.

Public Members

uint8_t **repetition_count_status**

Repetition Count Status

struct **phUwbDataPkt_t**

#include <UwbApi_Types.h>
Structure for SendData.

Public Members

uint32_t **sessionHandle**

Session Handle

uint8_t **mac_address**[EXT_MAC_ADDR_LEN]

MAC Address
Note:
In case short address is used, only the two least significant Octets are considered, and the upper 6 Octets shall be set with the value zero.

uint16_t **sequence_number**

Sequence Number

uint16_t **data_size**

Data Size

uint8_t ***data**

Application Data

struct **phUwbRcvDataPkt_t**

#include <UwbApi_Types.h>
Structure for RcvData.

Public Members

uint32_t **connection_identifier**

If LINK_LAYER_MODE is 0x00, then it shall contain the Session handle. If LINK_LAYER_MODE is 0x01 and in case of a Controller broadcast link with LLM selector set to either 0x00 or 0x01, then it shall contain the LL_CONNECT_ID

uint8_t **status**

Status

uint8_t **src_address**[EXT_MAC_ADDR_LEN]

MAC Address
Note:
In case short address is used, only the two least significant Octets are considered, and the upper 6 Octets are ignored.

uint16_t **sequence_number**

Sequence Number

All information provided in this document is subject to legal disclaimers.
© 2026 NXP B.V. All rights reserved.

uint16_t **data_size**

Data Size

uint8_t **data**[MAX_RESPONSE_DATA_RCV]

Application Data

struct **phDataControlTransmitNtfContext_t**

#include <UwbApi_Types.h>
Structure for storing Data Control Transmit Ntf Context.

Public Members

uint32_t **sessionHandle**

Session Handle

uint8_t **status**

Status

struct **phLogicalLinkCreateCmd_t**

#include <UwbApi_Types.h>
Structure to create Logical Link Command (LOGICAL_LINK_CREATE_CMD).

Public Members

uint32_t **session_handle**

Session Handle to the Data Transfer session or phase for which this Logical Link is created

uint8_t **llm_selector**

Link Layer Mode Selector - one of the value from eSelectLogicalLinkModes_t

uint8_t **dst_address**[EXT_MAC_ADDR_LEN]

Logical destination address of the Controlee of the logical link.
Note:
In case of SHORT_ADDR mode is used, then each Octet from octets 2 - 7 shall be set to 0x00.

uint8_t **ll_class_length**

Logical Link Class length 0x00: no Logical Link Class field present

struct **phLogicalLinkCreateNtf_t**

#include <UwbApi_Types.h>
Structure to create Logical Link Notification (LOGICAL_LINK_CREATE_NTF).

Public Members

uint32_t **ll_connect_id**

Logical Link Connect ID

uint8_t **status**

Status Code

struct **phLogicalLinkUwbsCreateNtf_t**

#include <UwbApi_Types.h>
Structure for Logical Link Create notification from UWBS (LOGICAL_LINK_UWBS_CREATE_NTF).

Public Members

uint32_t **sessionHandle**

Session Handle of the Data Transfer session or phase for which this Logical Link is created

All information provided in this document is subject to legal disclaimers.
© 2026 NXP B.V. All rights reserved.

uint32_t **ll_connect_id**

Logical Link Connect ID

uint8_t **llm_selector**

Link Layer Mode Selector - one of the value from eSelectLogicalLinkModes_t

uint8_t **src_address**[EXT_MAC_ADDR_LEN]

MAC Address Logical destination address of the Controller of the Logical Link

Note:

In case of SHORT_ADDR mode is used, then each Octet from octets 2 - 7 shall be set to 0x00.

struct **phLogicalLinkUwbsCloseNtf_t**

#include <UwbApi_Types.h>

Structure for Logical Link close notification from UWBS (LOGICAL_LINK_UWBS_CLOSE_NTF).

Public Members

uint32_t **ll_connect_id**

Logical Link Connect ID

uint8_t **status**

Status code

struct **phLogicalLinkDataPkt_t**

#include <UwbApi_Types.h>

Structure for Logical Link Send Data (LL_DATA_MESSAGE_SND) and Receive Data (LL_DATA_MESSAGE_RCV).

Public Members

uint32_t **llConnectId**

Logical Link Connect ID

uint16_t **sequence_number**

Sequence Number

uint16_t **data_size**

Application Data Size

uint8_t ***data**

Application Data

struct **phLogicalLinkGetParamsRsp_t**

#include <UwbApi_Types.h>

Structure for Logical Link Mode get parameters (LOGICAL_LINK_GET_PARAM_CMD/LOGICAL_LINK_GET_PARAM_RSP).

Public Members

uint8_t **status**

Status code

uint16_t **control_field**

Control Field

uint16_t **ll_sdu_size**

Maximum LL SDU size

All information provided in this document is subject to legal disclaimers.

© 2026 NXP B.V. All rights reserved.

uint16_t **ll_pdu_size**

Maximum LL PDU size

uint8_t **Tx_window_size**

Transmit Window Size, TxW

uint8_t **Rx_window_size**

Receive Window Size, RxW

uint8_t **repetition_count**

Repetition count Max

uint8_t **link_to**

Link TO

uint8_t **port**

PORT

struct **phUwbCapInfo_t**

#include <UwbApi_Types.h>
Structure lists out the UWB Device Info Parameters.

Public Members

uint8_t **firaPhyLowerRangeMajorVersion**

FIRA phy lower range major version

uint8_t **firaPhyLowerRangeMinorMaintenanceVersion**

FIRA phy lower range minor maintenance version

uint8_t **firaPhyHigherRangeMajorVersion**

FIRA phy higher range major version

uint8_t **firaPhyHigherRangeMinorMaintenanceVersion**

FIRA phy higher range minor maintenance version

uint8_t **firaMacLowerRangeMajorVersion**

FIRA mac lower range major version

uint8_t **firaMacLowerRangeMinorMaintenanceVersion**

FIRA mac lower range minor maintenance version

uint8_t **firaMacHigherRangeMajorVersion**

FIRA mac higher range major version

uint8_t **firaMacHigherRangeMinorMaintenanceVersion**

FIRA mac higher range minor maintenance version

uint8_t **deviceTypes**

Device types
- b0 = FiRa Controller
- b1 = FiRa Controlee
- b2 = CCC Controller
- b3 = CCC Controlee
- b4 = Aliro User Device
- b5 = Aliro Reader
- b6-b7 = RFU eUwb_CapsDeviceType_t

All information provided in this document is subject to legal disclaimers.
© 2026 NXP B.V. All rights reserved.

uint16_t **deviceRoles**

Device roles

uint16_t **rangingMethod**

Ranging Method

- b0: TDoA
- b1: SS-TWR with Deferred Mode
- b2: DS-TWR with Deferred Mode
- b3: SS-TWR with Non-Deferred Mode
- b4: DS-TWR with Non-Deferred Mode
- b5: OWR DL-TDoA
- b6: OWR for AoA Measurement
- b7: eSS-TWR with non-deferred mode for contention-based ranging.
- b8: aDS-TWR for contention-based ranging.
- b9: Data transfer mode.
- b10: Reserved for CCC
- b11: Aliro deferred DS-TWR.
- b15-b24: RFU

uint8_t **stsConfig**

STS config

uint8_t **multiNodeMode**

Multinode mode

uint8_t **rangingTimeStruct**

Ranging time struct

uint8_t **scheduledMode**

Scheduled mode

uint8_t **hoppingMode**

Hopping mode

uint8_t **blockStriding**

Block striding

uint8_t **uwbInitiationTime**

UWB initiation time

uint8_t **channels**

FIRA phy ver range

uint8_t **rframeConfig**

RFRAME config

uint8_t **ccConstraintLength**

CC constraint length

uint8_t **bprfParameterSets**

BPRF parameter sets

uint8_t **hprfParameterSets**[5]

HPRF parameter sets

uint8_t **aoaSupport**

All information provided in this document is subject to legal disclaimers.
© 2026 NXP B.V. All rights reserved.

AOA support

uint8_t **extendedMacAddress**

Extended mac address

uint16_t **maxMessageSize**

Max message size

uint16_t **maxDataPacketPayloadSize**

Max data packet payload size

uint8_t **slotBitmask**

Slot bitmask

uint8_t **syncCodeIndexBitmask**[4]

Sync code index bitmask

uint8_t **hoppingConfigBitmask**

Hopping config bitmask

uint8_t **channelBitmask**

Channel bitmask

uint8_t **numSupportedProtocolVersions**

Num of Protocol version supported

uint16_t **supportedProtocolVersions**[2]

Supported protocol version

uint8_t **numSupportedUWBConfigIDs**

Num of supported UWB config Ids

uint16_t **supportedUWBConfigIDs**[2]

Supported UWB config ID

uint8_t **supportedPulseShapeCombo**[9]

Supported pulse shape combo

uint8_t **minRanMultiplier**

Minimum RAN multiplier supported

uint16_t **maxUciPayloadLength**

Maximum UCI payload size that can handle by UWBS

uint8_t **inbandDataBlockSize**

Data buffer block size in bytes which the UWBS manages for the overall in-band data transfer memory pool.

uint8_t **inbandDataMaxBlock**

Parameter to indicate the number of blocks available in the overall in-band data transfer memory pool.

uint16_t **llCapabilityParam**

Logical Link Capability Param. b0 = LL feature support 0: Not supported 1: Supported Below bits shall be ignored if LL feature support is set to 0. b1 = LL Aggregated Frame support b2 = Secure Endpoint support b3 = Non-Secure Endpoint support b7-b4 = RFU 0: Not supported 1: Supported b11-b8: Maximum number of Logical Links supported in UWBS. b15-b12: Maximum number of Logical Links support per session

uint8_t **bypassModeSupport**

Bypass Mode support 0x00 = Bypass mode not supported 0x01 = Bypass mode supported

All information provided in this document is subject to legal disclaimers.

© 2026 NXP B.V. All rights reserved.

uint16_t **minSlotDurationSupport**

Min slot duration supported by the UWBS Unsigned integer in the unit of RSTU

uint8_t **aliroSupportedMacMode**

Support of MAC modes

- b0 = 1 active Ranging Round in Ranging Block
- b1 = 2 active Ranging Rounds in Ranging Block
- b7-b2 = RFU

uint8_t **numAliroSupportedProtocolVersions**

Num of Aliro Protocol version supported

uint16_t **aliroSupportedProtocolVersion**[2]

lists a supported protocol version where the fields of the protocol versions are 2 bytes long. Protocol version as defined in the Aliro specification is 0x0100. If another future protocol version 0x0101 would be supported, response has to be: 0x0100, 0x0101. Aliro Specification Version 1.0 is coded as 0x0100

struct **phUwbQueryDataSize_t**

#include <UwbApi_Types.h>

Structure lists out the Query Data Size Parameters.

Public Members

uint32_t **connectionId**

In-param to fetch the Connection ID If the msb of MSB is 0b1, then the connectionId indicates a session Handle If the msb of MSB is 0b0, then the connectionId indicates a LL_Connect_Id.

1.1.4 UWB Functional APIs

*group***uwb_apis**

APIs exposed to application to access UWB Functionality.

Various UWB APIs

Defines

UWBAPI_SETAPPCONFIGMULTIPLEPARAMS(*SESSION_HANDLE, PARMS_LIST*)

Helper macro to limit the parameters and avoid error case

Functions

tUWBAPI_STATUS **UwbApi_Initialize**(*phUwbappContext_t *pAppCtx*)

Initialize the UWB Device stack in the required mode. Operating mode will be set as per the Callback functions. Operating Modes supported include Standalone mode [default mode] and MCTT mode. Atleast one of the call backs shall not be NULL. When all the callbacks are set then “Standalone” mode will take precedence.

Parameters

`pAppCtx` – **[in]** Pointer to phUwbappContext_t structure

Return values

- `UWBAPI_STATUS_OK` – on success
- `UWBAPI_STATUS_TIMEOUT` – if command is timeout
- `UWBAPI_STATUS_FAILED` – otherwise

tUWBAPI_STATUS **UwbApi_GetRegisteredAppCallback**(*tUwbApi_AppCallback **const pAppCallback*)

Get the registered application callback function.

All information provided in this document is subject to legal disclaimers.
© 2026 NXP B.V. All rights reserved.

Parameters

`pAppCtx` – **[out]** Pointer to registered application callback function

Return values

- `UWBAPI_STATUS_OK` – on success
- `UWBAPI_STATUS_NOT_INITIALIZED` – UWB library is not initialized
- `UWBAPI_STATUS_FAILED` – otherwise

tUWBAPI_STATUS **UwbApi_RegisterAppCallback**(*tUwbApi_AppCallback* **pAppCallback*)

Update the registered application callback function.

Parameters

`pAppCtx` – **[in]** Pointer to application callback function to be registered

Return values

- `UWBAPI_STATUS_OK` – on success
- `UWBAPI_STATUS_NOT_INITIALIZED` – UWB library is not initialized
- `UWBAPI_STATUS_FAILED` – otherwise

tUWBAPI_STATUS **UwbApi_SwitchToMCTTMode**(*tUwbApi_TMLDataCallback* **pAppCtx*)

To switch the Operating mode to MCTT.

Parameters

`pAppCtx` – **[in]** Pointer to phUwbappContext_t strucutre

Return values

- `UWBAPI_STATUS_OK` – on success
- `UWBAPI_STATUS_FAILED` – otherwise

tUWBAPI_STATUS **UwbApi_ShutDown**()

De-initializes the UWB Middleware stack Sequence of task deinitialization must be maintained -> Deinit client thread -> Deinit reader thread -> Deinit uwb_task thread.

Return values

- `UWBAPI_STATUS_OK` – on success
- `UWBAPI_STATUS_FAILED` – otherwise
- `UWBAPI_STATUS_TIMEOUT` – if command is timeout

tUWBAPI_STATUS **UwbApi_RecoverUWBS**()

API to recover from crash, cmd timeout.

Return values

- `UWBAPI_STATUS_OK` – on success
- `UWBAPI_STATUS_FAILED` – otherwise
- `UWBAPI_STATUS_TIMEOUT` – if command is timeout

tUWBAPI_STATUS **UwbApi_UwbdReset**(*uint8_t resetConfig*)

(UWBIOT_UWBD_SR04X)
Resets UWBD device to Ready State

Parameters

`resetConfig` – **[in]** Supported Value: UWBD_RESET

All information provided in this document is subject to legal disclaimers.
© 2026 NXP B.V. All rights reserved.

Return values

- `UWBAPI_STATUS_OK` – on success
- `UWBAPI_STATUS_NOT_INITIALIZED` – if UWB stack is not initialized
- `UWBAPI_STATUS_TIMEOUT` – if command is timeout
- `UWBAPI_STATUS_FAILED` – otherwise

tUWBAPI_STATUS **UwbApi_GetUwbDevState**(*uint8_t *pDeviceState*)

Gets UWB Device State.

Parameters

`pDeviceState` – **[out]** pointer to uint8_t to get Device State. Valid only if API status is success.

Return values

- `UWBAPI_STATUS_OK` – on success
- `UWBAPI_STATUS_INVALID_PARAM` – if parameter is invalid
- `UWBAPI_STATUS_NOT_INITIALIZED` – if UWB stack is not initialized
- `UWBAPI_STATUS_TIMEOUT` – if command is timeout
- `UWBAPI_STATUS_FAILED` – otherwise

tUWBAPI_STATUS **UwbApi_SessionInit**(*uint32_t sessionId, eSessionType sessionType, uint32_t *sessionHandle*)

Initializes session for a Type(Ranging/Data/Per).

Parameters

- `sessionId` – **[in]** Session ID.
- `sessionType` – **[in]** Type of Session(Ranging/Data/Per).
- `sessionHandle` – **[out]** Session Handle.

Return values

- `UWBAPI_STATUS_OK` – on success
- `UWBAPI_STATUS_NOT_INITIALIZED` – if UWB stack is not initialized
- `UWBAPI_STATUS_MAX_SESSIONS_EXCEEDED` – if more than 5 sessions are exceeded
- `UWBAPI_STATUS_TIMEOUT` – if command is timeout
- `UWBAPI_STATUS_FAILED` – otherwise

tUWBAPI_STATUS **UwbApi_SessionDeinit**(*uint32_t sessionHandle*)

De-initialize based on Session Handle.

Parameters

`sessionHandle` – **[in]** Initialized Session Handle

Return values

- `UWBAPI_STATUS_OK` – on success
- `UWBAPI_STATUS_NOT_INITIALIZED` – if UWB stack is not initialized
- `UWBAPI_STATUS_SESSION_NOT_EXIST` – if session is not initialized with sessionHandle
- `UWBAPI_STATUS_TIMEOUT` – if command is timeout
- `UWBAPI_STATUS_FAILED` – otherwise

tUWBAPI_STATUS **UwbApi_ConfigureNtfTimeout**(*uint32_t timeoutMs*)

API to configure notification timeout value for session related operations.

Note:

All information provided in this document is subject to legal disclaimers.
© 2026 NXP B.V. All rights reserved.

This timeout value affects all subsequent session operations and should be set before starting any ranging sessions. In case of multisession scenario, this timeout value will be applied to all sessions.

Parameters

`timeoutMs` – Timeout value in milliseconds for notification events

Return values

- `UWBAPI_STATUS_OK` – on success
- `UWBAPI_STATUS_NOT_INITIALIZED` – if UWB stack is not initialized
- `UWBAPI_STATUS_INVALID_PARAM` – if timeoutMs is having invalid value

tUWBAPI_STATUS **UwbApi_GetDeviceInfo**(*phUwbDevInfo_t *pdevInfo*)

Returns UCI, FW and MW version.

Parameters

`pdevInfo` – **[out]** Pointer to phUwbDevInfo_t

Return values

- `UWBAPI_STATUS_OK` – if successful
- `UWBAPI_STATUS_NOT_INITIALIZED` – if UCI stack is not initialized
- `UWBAPI_STATUS_INVALID_PARAM` – if invalid parameters are passed
- `UWBAPI_STATUS_FAILED` – otherwise
- `UWBAPI_STATUS_TIMEOUT` – if command is timeout

tUWBAPI_STATUS **UwbApi_SetRangingParams**(*uint32_t sessionHandle, const phRangingParams_t *pRanging Param*)

Set session specific ranging parameters.

For contention based ranging DST_MAC_ADDRESS and NO_OF_CONTROLEES parameter is not required both should be set to zero.

Example: For time based and contention based configuration given below.

```
// Time based Ranging :

phRangingParams_t inRangingParams = {0};
inRangingParams.noOfControlees = 1;
inRangingParams.dstMacAddr[] = {0x11,0x22};

// Contention based Ranging :

phRangingParams_t inRangingParams = {0};
inRangingParams.noOfControlees = 0;
inRangingParams.dstMacAddr[] = {0x00,0x00};
```

Parameters

- `sessionHandle` – **[in]** Initialized Session Handle
- `pRangingParam` – **[in]** Pointer to phRangingParams_t

Return values

- `UWBAPI_STATUS_OK` – on success
- `UWBAPI_STATUS_NOT_INITIALIZED` – if UWB stack is not initialized
- `UWBAPI_STATUS_INVALID_PARAM` – if invalid parameters are passed
- `UWBAPI_STATUS_SESSION_NOT_EXIST` – if session is not initialized with sessionHandle
- `UWBAPI_STATUS_TIMEOUT` – if command is timeout
- `UWBAPI_STATUS_FAILED` – otherwise

tUWBAPI_STATUS **UwbApi_GetRangingParams**(*uint32_t sessionHandle, phRangingParams_t *pRangingParams*)

All information provided in this document is subject to legal disclaimers. © 2026 NXP B.V. All rights reserved.

Get session specific ranging parameters.

Parameters

- **sessionHandle** – **[in]** Initialized Session Handle
- **pRangingParams** – **[out]** Pointer to phRangingParams_t .Valid only if API status is success

Return values

- **UWBAPI_STATUS_OK** – on success
- **UWBAPI_STATUS_NOT_INITIALIZED** – if UWB stack is not initialized
- **UWBAPI_STATUS_INVALID_PARAM** – if invalid parameters are passed
- **UWBAPI_STATUS_SESSION_NOT_EXIST** – if session is not initialized with sessionHandle
- **UWBAPI_STATUS_TIMEOUT** – if command is timeout
- **UWBAPI_STATUS_FAILED** – otherwise

tUWBAPI_STATUS **UwbApi_SetCccRangingParams**(*uint32_t sessionHandle, const phCccRangingParams_t *pCccRangingParam*)

Set CCC Mandatory Application configuration parameters.

Parameters

- **sessionHandle** – **[in]** Initialized Session Handle
- **pCccRangingParam** – **[in]** Pointer to `pCccRangingParam`

Return values

- **UWBAPI_STATUS_OK** – on success
- **UWBAPI_STATUS_NOT_INITIALIZED** – if UWB stack is not initialized
- **UWBAPI_STATUS_INVALID_PARAM** – if invalid parameters are passed
- **UWBAPI_STATUS_SESSION_NOT_EXIST** – if session is not initialized with sessionHandle
- **UWBAPI_STATUS_TIMEOUT** – if command is timeout
- **UWBAPI_STATUS_FAILED** – otherwise

tUWBAPI_STATUS **UwbApi_SetAppConfig**(*uint32_t sessionHandle, eAppConfig param_id, uint32_t param_value*)

Set session specific app config parameters.

Note:

FOR SR1XXT and SR2XXT this API can only be used to set FIRA-specific AppCfgs.

Note:

RANGING_ROUND_USAGE and SCHEDULE_MODE parameters are mandated to set only in SESSION_STATE_INIT state.

Warning:

For setting STATIC_STS_IV and UWB_INITIATION_TIME, use UwbApi_SetAppConfigMultipleParams API.

Parameters

- **sessionHandle** – **[in]** Initialized Session Handle
- **param_id** – **[in]** App Config Parameter Id
- **param_value** – **[in]** Param value for App config param id

All information provided in this document is subject to legal disclaimers.
© 2026 NXP B.V. All rights reserved.

Return values

- **UWBAPI_STATUS_OK** – on success
- **UWBAPI_STATUS_NOT_INITIALIZED** – if UWB stack is not initialized
- **UWBAPI_STATUS_INVALID_PARAM** – if invalid parameters are passed
- **UWBAPI_STATUS_SESSION_NOT_EXIST** – if session is not initialized with sessionHandle
- **UWBAPI_STATUS_TIMEOUT** – if command is timeout
- **UWBAPI_STATUS_FAILED** – otherwise

tUWBAPI_STATUS **UwbApi_SetAppConfigMultipleParams**(*uint32_t sessionHandle, uint8_t noOfparams, const UWB_AppParams_List_t *AppParams_List*)

Host shall use this API to set multiple application configuration parameters. Number of Parameters also needs to be indicated.

To easily set the AppParams list, following macros have been defined.

UWB_SET_APP_PARAM_VALUE(Parameter, Value): This macro sets the value of the corresponding parameter with the given Value.This shall be used to set all types of values of 8 or 16 or 32 bit wide.For more than 32-bit values, following macro shall be used.

UWB_SET_APP_PARAM_ARRAY(Parameter, ArrayValue, Length): This macro sets the value of the corresponding parameter as an array of 8bit. Length parameter contains the total length of the array.

Example: To set SFD Id to zero and static sts iv, macro shall be invoked as given below.

```
UWB_AppParams_List_t SetAppParamsList[] =
 {UWB_SET_APP_PARAM_VALUE(SFD_ID, 0)};
uint8_t static_sts_iv[] = {1,2,3,4,5,6};
UWB_AppParams_List_t SetAppParamsList[] =
 {UWB_SET_APP_PARAM_ARRAY(STATIC_STS_IV, static_sts_iv,
sizeof(static_sts_iv))};
```

Note:

FOR SR1XXT and SR2XXT this API can only be used to set FIRA-specific AppCfgs.

Note:

RANGING_ROUND_USAGE and SCHEDULE_MODE parameters are mandated to set only in SESSION_STATE_INIT state.

Parameters

- **sessionHandle** – **[in]** Initialized Session Handle
- **noOfparams** – **[in]** Number of App Config Parameters
- **AppParams_List** – **[in]** Application parameters values in tlv format

Return values

- **UWBAPI_STATUS_OK** – on success
- **UWBAPI_STATUS_NOT_INITIALIZED** – if UWB stack is not initialized
- **UWBAPI_STATUS_INVALID_PARAM** – if invalid parameters are passed
- **UWBAPI_STATUS_SESSION_NOT_EXIST** – if session is not initialized with sessionHandle
- **UWBAPI_STATUS_TIMEOUT** – if command is timeout
- **UWBAPI_STATUS_FAILED** – otherwise

tUWBAPI_STATUS **UwbApi_SetVendorAppConfigs**(*uint32_t sessionHandle, uint8_t noOfparams, const UWB_VendorAppParams_List_t *vendorAppParams_List*)

Host shall use this API to set multiple Vendor specific application configuration parameters. Number of Parameters also needs to be indicated.

To easily set the AppParams list, following macros have been defined.

All information provided in this document is subject to legal disclaimers.

© 2026 NXP B.V. All rights reserved.

UWB_SET_VENDOR_APP_PARAM_VALUE(Parameter, Value): This macro sets the value of the corresponding parameter with the given Value.This shall be used to set all types of values of 8 or 16 or 32 bit wide.For more than 32-bit values, following macro shall be used.

UWB_SET_VENDOR_APP_PARAM_ARRAY(Parameter, ArrayValue, Length): This macro sets the value of the corresponding parameter as an array of 8bit. Length parameter contains the total length of the array.

Example: To set MAC Payload encryption Id to zero and antenna config tx, macro shall be invoked as given below.

```
UWB_VendorAppParams_List_t SetAppParamsList[] =
 {UWB_SET_VENDOR_APP_PARAM_VALUE(MAC_PAYLOAD_ENCRYPTION, 0)};
uint8_t antennas_configuration_tx[] = {1,2,3,4,5,6};
UWB_VendorAppParams_List_t SetAppParamsList[] =
 {UWB_SET_VENDOR_APP_PARAM_ARRAY(ANTENNAS_CONFIGURATION_TX,
antennas_configuration_tx, sizeof(antennas_configuration_tx))};
```

Parameters

- **`sessionHandle`** – **[in]** Initialized Session Handle
- **`noOfparams`** – **[in]** Number of App Config Parameters
- **`vendorAppParams_List`** – **[in]** vendor specific Application parameters values in tlv format

Return values

- **`UWBAPI_STATUS_OK`** – on success
- **`UWBAPI_STATUS_NOT_INITIALIZED`** – if UWB stack is not initialized
- **`UWBAPI_STATUS_INVALID_PARAM`** – if invalid parameters are passed
- **`UWBAPI_STATUS_SESSION_NOT_EXIST`** – if session is not initialized with sessionHandle
- **`UWBAPI_STATUS_TIMEOUT`** – if command is timeout
- **`UWBAPI_STATUS_FAILED`** – otherwise

tUWBAPI_STATUS **UwbApi_GetAppConfig**(*uint32_t sessionHandle, eAppConfig param_id, uint32_t *param_value*)

Get session specific app config parameters.

Note:

FOR SR1XXT and SR2XXT this API can only be used to get FIRA-specific AppCfgs.

Parameters

- **`sessionHandle`** – **[in]** Initialized Session Handle
- **`param_id`** – **[in]** App Config Parameter Id
- **`param_value`** – **[out]** Param value for App config param id

Return values

- **`UWBAPI_STATUS_OK`** – on success
- **`UWBAPI_STATUS_NOT_INITIALIZED`** – if UWB stack is not initialized
- **`UWBAPI_STATUS_INVALID_PARAM`** – if invalid parameters are passed
- **`UWBAPI_STATUS_SESSION_NOT_EXIST`** – if session is not initialized with sessionHandle
- **`UWBAPI_STATUS_TIMEOUT`** – if command is timeout
- **`UWBAPI_STATUS_FAILED`** – otherwise

tUWBAPI_STATUS **UwbApi_GetAppConfigMultipleParams**(*uint32_t sessionHandle, uint8_t noOfparams, UWB_AppParams_List_t *AppParams_List*)

Host shall use this API to get multiple application configuration parameters. Number of Parameters also needs to be indicated.

All information provided in this document is subject to legal disclaimers.
© 2026 NXP B.V. All rights reserved.

Following macros can be used, to easily set the AppParams list

UWB_SET_GETAPP_PARAM(Parameter): This macro sets parameter, in UWB_AppParams_List_t structure. This shall be used to get all types of values of 8 or 16 or 32 bit wide. For more than 32-bit values, following macro shall be used.

UWB_SET_APP_PARAM_ARRAY(Parameter, ArrayValue, Length): This macro sets parameter and array of 8bit to store the configuration, in UWB_AppParams_List_t structure Length parameter contains the total length of the array.

Example: To get SFD Id and static sts iv, macro shall be invoked as given below.

```
uint8_t static_sts_iv[6];
UWB_AppParams_List_t SetAppParamsList[] =
 {UWB_SET_GETAPP_PARAM_VALUE(SFD_ID),

 UWB_SET_APP_PARAM_ARRAY(STATIC_STS_IV, static_sts_iv,
 sizeof(static_sts_iv)),};
```

Note:

FOR SR1XXT and SR2XXT this API can only be used to get FIRA-specific AppCfgs.

Parameters

- **`sessionHandle`** – **[in]** Initialized Session Handle
- **`noOfparams`** – **[in]** Number of App Config Parameters
- **`AppParams_List`** – **[in]** Application parameters

Return values

- **`UWBAPI_STATUS_OK`** – on success
- **`UWBAPI_STATUS_NOT_INITIALIZED`** – if UWB stack is not initialized
- **`UWBAPI_STATUS_INVALID_PARAM`** – if invalid parameters are passed
- **`UWBAPI_STATUS_SESSION_NOT_EXIST`** – if session is not initialized with sessionHandle
- **`UWBAPI_STATUS_TIMEOUT`** – if command is timeout
- **`UWBAPI_STATUS_FAILED`** – otherwise

tUWBAPI_STATUS **UwbApi_GetVendorAppConfigs**(*uint32_t sessionHandle, uint8_t noOfparams, UWB_Vendor AppParams_List_t *vendorAppParams_List*)

Host shall use this API to get multiple vendor application configuration parameters. Number of Parameters also needs to be indicated.

Following macros can be used, to easily set the AppParams list

UWB_SET_GETVENDOR_APP_PARAM_VALUE(Parameter): This macro sets parameter, in UWB_VendorApp Params_List_t structure. This shall be used to get all types of values of 8 or 16 or 32 bit wide. For more than 32-bit values, following macro shall be used.

UWB_VENDOR_SET_APP_PARAM_ARRAY(Parameter, ArrayValue, Length): This macro sets parameter and array of 8bit to store the configuration, in UWB_VendorAppParams_List_t structure Length parameter contains the total length of the array.

Example: To get MAC Payload encryption Id and antenna config tx, macro shall be invoked as given below.

```
uint8_t antennas_configuration_tx[6];
UWB_VendorAppParams_List_t SetAppParamsList[] =
 {UWB_SET_GETVENDOR_APP_PARAM_VALUE(MAC_PAYLOAD_ENCRYPTION),

 UWB_VENDOR_SET_APP_PARAM_ARRAY(ANTENNAS_CONFIGURATION_TX,
antennas_configuration_tx, sizeof(antennas_configuration_tx)),};
```


All information provided in this document is subject to legal disclaimers.

© 2026 NXP B.V. All rights reserved.

Parameters

- **sessionHandle** – **[in]** Initialized Session Handle
- **noOfparams** – **[in]** Number of App Config Parameters
- **vendorAppParams_List** – **[in]** Vendor Application parameters

Return values

- **UWBAPI_STATUS_OK** – on success
- **UWBAPI_STATUS_NOT_INITIALIZED** – if UWB stack is not initialized
- **UWBAPI_STATUS_INVALID_PARAM** – if invalid parameters are passed
- **UWBAPI_STATUS_SESSION_NOT_EXIST** – if session is not initialized with sessionHandle
- **UWBAPI_STATUS_TIMEOUT** – if command is timeout
- **UWBAPI_STATUS_FAILED** – otherwise

tUWBAPI_STATUS **UwbApi_SetStaticSts**(*uint32_t sessionHandle, uint16_t vendorId, uint8_t const *const staticSts Iv*)

Sets session specific app config parameters Vendor ID and Static STS IV.

Parameters

- **sessionHandle** – **[in]** Initialized Session Handle
- **vendorId** – **[in]** App Config Parameter Vendor Id
- **staticStsIv** – **[in]** Param value for App config param static Sts Iv It is the responsibility of the caller that STS IV is exactly UCI_PARAM_LEN_STATIC_STS_IV bytes long.

Return values

- **UWBAPI_STATUS_OK** – on success
- **UWBAPI_STATUS_NOT_INITIALIZED** – if UWB stack is not initialized
- **UWBAPI_STATUS_INVALID_PARAM** – if invalid parameters are passed
- **UWBAPI_STATUS_SESSION_NOT_EXIST** – if session is not initialized with sessionHandle
- **UWBAPI_STATUS_TIMEOUT** – if command is timeout
- **UWBAPI_STATUS_FAILED** – otherwise

tUWBAPI_STATUS **UwbApi_StartRangingSession**(*uint32_t sessionHandle*)

Start Ranging for a session. Before Invoking Start ranging its mandatory to set all the ranging configurations.

Parameters

sessionHandle – **[in]** Initialized Session Handle

Return values

- **UWBAPI_STATUS_OK** – on success
- **UWBAPI_STATUS_NOT_INITIALIZED** – if UWB stack is not initialized
- **UWBAPI_STATUS_SESSION_NOT_EXIST** – if session is not initialized with sessionHandle
- **UWBAPI_STATUS_TIMEOUT** – if command is timeout
- **UWBAPI_STATUS_FAILED** – otherwise

tUWBAPI_STATUS **UwbApi_SetDeviceConfig**(*eDeviceConfig param_id, uint8_t param_len, phDeviceConfigData_t *param_value*)

Sets device configuration.

All information provided in this document is subject to legal disclaimers.
© 2026 NXP B.V. All rights reserved.

Parameters

- `param_id` – **[in]** device configuration param id
- `param_len` – **[in]** Parameter length
- `param_value` – **[in]** Param value

Return values

- `UWBAPI_STATUS_OK` – on success
- `UWBAPI_STATUS_NOT_INITIALIZED` – if UWB stack is not initialized
- `UWBAPI_STATUS_INVALID_PARAM` – if invalid parameters are passed
- `UWBAPI_STATUS_TIMEOUT` – if command is timeout
- `UWBAPI_STATUS_FAILED` – otherwise

tUWBAPI_STATUS **UwbApi_GetDeviceConfig**(*eDeviceConfig param_id, phDeviceConfigData_t *param_value*)

Get device config parameters.

Parameters

- `param_id` – **[in]** Device Config Parameter Id
- `param_value` – **[inout]** Param value structure for device config param id

Return values

- `UWBAPI_STATUS_OK` – on success
- `UWBAPI_STATUS_NOT_INITIALIZED` – if UWB stack is not initialized
- `UWBAPI_STATUS_INVALID_PARAM` – if invalid parameters are passed
- `UWBAPI_STATUS_TIMEOUT` – if command is timeout
- `UWBAPI_STATUS_FAILED` – otherwise

tUWBAPI_STATUS **UwbApi_StopRangingSession**(*uint32_t sessionHandle*)

Stop Ranging for a session.

Parameters

`sessionHandle` – **[in]** Initialized Session Handle

Return values

- `UWBAPI_STATUS_OK` – on success
- `UWBAPI_STATUS_NOT_INITIALIZED` – if UWB stack is not initialized
- `UWBAPI_STATUS_TIMEOUT` – if command is timeout
- `UWBAPI_STATUS_FAILED` – otherwise

tUWBAPI_STATUS **UwbApi_EnableRangingDataNtf**(*uint32_t sessionHandle, uint8_t enableRangingDataNtf, uint16_t proximityNear, uint16_t proximityFar*)

Enable Ranging Data Notifications different options.

Parameters

- `sessionHandle` – **[in]** Initialized Session Handle
- `enableRangingDataNtf` – **[in]** Enable Ranging data notification 0/1/2. option 2 is not allowed when ranging is ongoing.
- `proximityNear` – **[in]** Proximity Near value valid if enableRangingDataNtf sets to 2
- `proximityFar` – **[in]** Proximity far value valid if enableRangingDataNtf sets to 2

All information provided in this document is subject to legal disclaimers.
© 2026 NXP B.V. All rights reserved.

Return values

- `UWBAPI_STATUS_OK` – on success
- `UWBAPI_STATUS_NOT_INITIALIZED` – if UWB stack is not initialized
- `UWBAPI_STATUS_INVALID_PARAM` – if invalid parameters are passed
- `UWBAPI_STATUS_TIMEOUT` – if command is timeout
- `UWBAPI_STATUS_FAILED` – otherwise

tUWBAPI_STATUS **UwbApi_GetSessionState**(*uint32_t sessionHandle, uint8_t *sessionState*)

Get Session State.

- If API returns UWBAPI_STATUS_OK, Session State would be one of the below values
  - UWBAPI_SESSION_INIT_SUCCESS - Session is Initialized
  - UWBAPI_SESSION_DEINIT_SUCCESS - Session is De-initialized
  - UWBAPI_SESSION_ACTIVATED - Session is Busy
  - UWBAPI_SESSION_IDLE - Session is Idle
  - UWBAPI_SESSION_ERROR - Session Not Found
- If API doesn't return UWBAPI_STATUS_OK, Session State is set to
  - UWBAPI_SESSION_ERROR - Error in getting the session state.

Parameters

- `sessionHandle` – **[in]** Initialized Session Handle
- `sessionState` – **[out]** Session Status

Return values

- `UWBAPI_STATUS_OK` – on success
- `UWBAPI_STATUS_NOT_INITIALIZED` – if UCI stack is not initialized
- `UWBAPI_STATUS_INVALID_PARAM` – if invalid parameters are passed
- `UWBAPI_STATUS_TIMEOUT` – if command is timeout
- `UWBAPI_STATUS_FAILED` – otherwise

tUWBAPI_STATUS **UwbApi_SendRawCommand**(*uint8_t data[], uint16_t data_len, uint8_t *pResp, uint16_t *pRespLen*)

Send UCI RAW command.

Parameters

- `data` – **[in]** UCI Command to be sent
- `data_len` – **[in]** Length of the UCI Command
- `pResp` – **[out]** Response Received
- `pRespLen` – **[out]** Response length

Return values

- `UWBAPI_STATUS_OK` – if successful
- `UWBAPI_STATUS_NOT_INITIALIZED` – if UCI stack is not initialized
- `UWBAPI_STATUS_INVALID_PARAM` – if wrong parameter is passed
- `UWBAPI_STATUS_BUFFER_OVERFLOW` – if response buffer is not sufficient to hold the response
- `UWBAPI_STATUS_TIMEOUT` – if command is timeout
- `UWBAPI_STATUS_FAILED` – otherwise

tUWBAPI_STATUS **UwbApi_UpdateControllerMulticastList**(*phMulticastControleeListContext_t *pControleeContext, phMulticastControleeListRspContext_t *pControleeListRspContext*)

Update Controller Multicast List.

Note:

All information provided in this document is subject to legal disclaimers.
© 2026 NXP B.V. All rights reserved.

The out param, pControleeListRspContext is updated only if response status is UWBAPI_STATUS_FAILED.

Parameters

- **pControleeContext** – **[in]** Controlee in Context of type phMulticastControleeListContext_t
- **pControleeListRspContext** – **[out]** Controlee Out Context of type phMulticastControleeListRspContext_t

Return values

- **UWBAPI_STATUS_OK** – On success
- **UWBAPI_STATUS_REJECTED** – If command is called in SESSION_STATE_INIT
- **UWBAPI_STATUS_ERROR_MULTICAST_LIST_FULL** – If the multicast list is full, while adding controlee to list
- **UWBAPI_STATUS_NOT_INITIALIZED** – If UWB stack is not initialized
- **UWBAPI_STATUS_INVALID_PARAM** – If invalid parameters are passed
- **UWBAPI_STATUS_FAILED** – Otherwise, please refer specification for the appropriate Status.

tUWBAPI_STATUS **UwbApi_GetTrng**(*uint8_t trng_size, uint8_t *ptrng*)

Get TRNG api.

Warning:

On SR040, maximum 4 bytes can be drawn during an active ranging session

Parameters

- **trng_size** – **[in]** Size of ptrng buffer and number of bytes expected
- **ptrng** – **[out]** : the size of this buffer shall be equal to trng size.

Return values

- **UWBAPI_STATUS_OK** – on success
- **UWBAPI_STATUS_NOT_INITIALIZED** – if UWB stack is not initialized
- **UWBAPI_STATUS_INVALID_PARAM** – if invalid parameters are passed
- **UWBAPI_STATUS_TIMEOUT** – if command is timeout
- **UWBAPI_STATUS_FAILED** – otherwise

tUWBAPI_STATUS **UwbApi_SetProfileParams**(*uint8_t *pProfileBlob, uint16_t blobSize, phUwbProfileInfo_t *pProfileInfo*)

Setting up Profile blob.

Parameters

- **pProfileBlob** – **[in]** : Profile Blob which contain all information.
- **blobSize** – **[in]** : Size of Blob
- **pProfileInfo** – **[inout]** : contains profile information.

Return values

- **UWBAPI_STATUS_OK** – on success
- **UWBAPI_STATUS_NOT_INITIALIZED** – if UWB stack is not initialized
- **UWBAPI_STATUS_INVALID_PARAM** – if invalid parameters are passed
- **UWBAPI_STATUS_TIMEOUT** – if command is timeout
- **UWBAPI_STATUS_FAILED** – otherwise

tUWBAPI_STATUS **UwbApi_GetUwbConfigData_iOS**(*UWB_DeviceRole_t device_role, AccessoryUwbConfigDataContent_t *uwb_data_content*)

Fill Accessory UWB related configuration data.

All information provided in this document is subject to legal disclaimers. © 2026 NXP B.V. All rights reserved.

Parameters

- **device_role** – -[in] device role
- **uwb_data_content** – -[Out] Pointer to structure of AccessoryUwbConfigDataContent_t

Return values

- **UWBAPI_STATUS_OK** – - on success
- **UWBAPI_STATUS_NOT_INITIALIZED** – - if UWB stack is not initialized
- **UWBAPI_STATUS_INVALID_PARAM** – - if invalid parameters are passed
- **UWBAPI_STATUS_FAILED** – - otherwise

tUWBAPI_STATUS **UwbApi_GetUwbConfigData_Android**(*UwbDeviceConfigData_t* *uwb_data_content*)

Populates the accessory UWB configuration data structure for Android.

This API fills the provided `UwbDeviceConfigData_t` structure with UWB-related configuration parameters required for accessory operation.

Note:

To enable Findhub-specific configuration, the caller must set `uwb_data_content->isFindhubEnabled` = true before invoking this API. When this flag is enabled, the following fields in `UwbDeviceConfigData_t` will be populated with Findhub-specific values:

- *supported_config_ids*
- *ranging_role*
- *device_mac_addr*
- *ranging_technology_role*
- *payload_size*
- *supported_channels*
- *preamble_index*
- *ranging_interval*
- *slot_duration*

Parameters

uwb_data_content – **[inout]** Pointer to a `UwbDeviceConfigData_t` structure that will be populated.

Return values

- **UWBAPI_STATUS_OK** – On success.
- **UWBAPI_STATUS_NOT_INITIALIZED** – If the UWB stack is not initialized.
- **UWBAPI_STATUS_INVALID_PARAM** – If invalid parameters are passed.
- **UWBAPI_STATUS_FAILED** – On general failure.

tUWBAPI_STATUS **UwbApi_GetDeviceCapability**(*phUwbCapInfo_t* *pDevCap*)

Frames the device capabilities in TLV format.

Parameters

pDevCap – - [Out] Pointer to structure of device capability data

Return values

- **UWBAPI_STATUS_OK** – - on success
- **UWBAPI_STATUS_NOT_INITIALIZED** – - if UWB stack is not initialized
- **UWBAPI_STATUS_INVALID_PARAM** – - if invalid parameters are passed
- **UWBAPI_STATUS_FAILED** – - otherwise

tUWBAPI_STATUS **UwbApi_SendData**(*phUwbDataPkt_t* *pSendData*)

Host shall use this API to send data over UWB interface.

All information provided in this document is subject to legal disclaimers. © 2026 NXP B.V. All rights reserved.

- If SESSION_DATA_TRANSFER_STATUS_NTF is disabled, then the UWBS shall not send SESSION_DATA_TRANSFER_STATUS_NTF for every Application Data transmission except for last.

Example: SESSION_DATA_TRANSFER_STATUS_NTF=0(Disable) and DATA_REPETITION_COUNT=5 and RANGING_ROUND_USAGE=200 ms

UWA_DM_DATA_TRANSMIT_STATUS_EVT will receive after 1,000 ms (DATA_REPETITION_COUNT * RANGING_ROUND_USAGE)

- Limitation:
  - Notification Read Timeout : Reading UWA_DM_DATA_TRANSMIT_STATUS_EVT Notification from the UWB will result in a read timeout if UWB Notification time exceeds the define limit.
  - Example: if DATA_REPETITION_COUNT=60 and SESSION_DATA_TRANSFER_STATUS_NTF=0 and RANGING_ROUND_USAGE=200 ms then UWA_DM_DATA_TRANSMIT_STATUS_EVT will receive after ~12 secs (DATA_REPETITION_COUNT * RANGING_ROUND_USAGE) by that time UwbApi_SendData API time out will happen .

Warning:

There is a possibility of UwbApi_SendData API returning Timeout (UWBAPI_STATUS_FAILED status) in the following sceanrio. Although API status is failed but the outcome of testing scenario to be treated as SUCCESS only.

Parameters

`pSendData` – **[in]** Send Data Content

Return values

- **`UWBAPI_STATUS_OK`** – on success
- **`UWBAPI_STATUS_NOT_INITIALIZED`** – if UWB stack is not initialized
- **`UWBAPI_STATUS_INVALID_PARAM`** – if invalid parameters are passed
- **`UWBAPI_STATUS_TIMEOUT`** – if command is timeout
- **`UWBAPI_STATUS_REJECTED`** – if session is not established when data packet sent
- **`UWBAPI_STATUS_NO_CREDIT_AVAILABLE`** – if buffer is not available to accept data
- **`UWBAPI_STATUS_DATA_TRANSFER_ERROR`** – if data is not sent due to an unrecoverable error
- **`UWBAPI_STATUS_FAILED`** – otherwise

tUWBAPI_STATUS **UwbApi_SessionQueryDataSize**(*phUwbQueryDataSize_t *pQueryDataSize*)

API to get max data size that can be transferred during a single ranging round.

Note:

This API must be called in session active state.

Parameters

`pQueryDataSize` – **[inout]** Pointer to structure of Query Data Size

Return values

- **`UWBAPI_STATUS_OK`** – on success
- **`UWBAPI_STATUS_NOT_INITIALIZED`** – if UWB stack is not initialized
- **`UWBAPI_STATUS_INVALID_PARAM`** – if invalid parameters are passed
- **`UWBAPI_STATUS_TIMEOUT`** – if command is timeout
- **`UWBAPI_STATUS_UNKNOWN`** – Unknown error
- **`UWBAPI_STATUS_FAILED`** – otherwise

tUWBAPI_STATUS **UwbApi_SetControllerHusSession**(*phControllerHusSessionConfig_t *pHusSessionCfg*)

Frames the HUS session config in TLV format for Controller.

All information provided in this document is subject to legal disclaimers.
© 2026 NXP B.V. All rights reserved.

Parameters

`pHusSessionCfg` – **[in]** : Pointer to structure of device HUS Controller session config data.

Return values

- `UWBAPI_STATUS_OK` – on success
- `UWBAPI_STATUS_NOT_INITIALIZED` – if UWB stack is not initialized
- `UWBAPI_STATUS_SESSION_NOT_EXIST` – if Session is not existing or not created.
- `UWBAPI_STATUS_SESSION_NOT_CONFIGURED` – if Session is not configured with required app configurations.
- `UWBAPI_STATUS_ERROR_SESSION_INVALID_SLOT_DURATION` – if Slot duration of the secondary session is not integer multiple of the slot duration of primary session.
- `UWBAPI_STATUS_ERROR_SESSION_INVALID_SLOT_ALLOCAION` – if Slot allocation for phases is not correct.
- `UWBAPI_STATUS_TIMEOUT` – if command is timeout
- `UWBAPI_STATUS_FAILED` – otherwise

tUWBAPI_STATUS **UwbApi_SetControleeHusSession**(*phControleeHusSessionConfig_t* **pHusSessionCfg*)

Frames the HUS session config in TLV format for Controlee.

Parameters

`pHusSessionCfg` – **[in]** : Pointer to structure of device HUS Controlee session config data.

Return values

- `UWBAPI_STATUS_OK` – on success
- `UWBAPI_STATUS_REJECTED` – if UWB stack is not initialized
- `UWBAPI_STATUS_SESSION_NOT_EXIST` – if Session is not existing or not created.
- `UWBAPI_STATUS_ERROR_SESSION_INVALID_SLOT_DURATION` – if Slot duration of the secondary session is not integer multiple of the slot duration of primary session.
- `UWBAPI_STATUS_SESSION_NOT_CONFIGURED` – if command is timed out
- `UWBAPI_STATUS_ERROR_SESSION_INVALID_SLOT_ALLOCAION` – if Slot allocation for phases is not correct.
- `UWBAPI_STATUS_FAILED` – otherwise

tUWBAPI_STATUS **UwbApi_SessionDtpcmConfig**(*phDataTxPhaseConfig_t* **phDataTxPhaseCfg*)

Frames the Data Transfer Phase Control Message in TLV format.

Note:

If application gets the notification UWBD_DATA_TRANSFER_PHASE_CONFIG_NTF, refer eDataTxPhaseCfg_NftCode for status codes.

Parameters

`phDataTxPhaseCfg` – - Pointer to structure of Data Transfer Phase Configuration.

All information provided in this document is subject to legal disclaimers.
© 2026 NXP B.V. All rights reserved.

Return values

- **UWBAPI_STATUS_OK** – - If DTPCM is configured for given MAC Address.
- **DTPCM_STATUS_ERROR_DUPLICATE_SLOT_ASSIGMENT** – - Reported when configured slot assignments is inconsistent, i.e., one slot is assigned to more than one FiRA device
- **DTPCM_STATUS_ERROR_DTPML_OVERSIZE** – - Reported when configured slot assignments cannot fit within one DTPCM.
- **UWBAPI_STATUS_TIMEOUT** – - If command is timed out
- **UWBAPI_STATUS_INVALID_PARAM** – - If given MAC address is not found.
- **UWBAPI_STATUS_FAILED** – - otherwise

tUWBAPI_STATUS **UwbApi_CreateLogicalLink**(*phLogicalLinkCreateCmd_t *phLogicalLinkCreateCmd, uint32_t *p LogicalLinkConnectId*)

Creates Logical Link for Data Transfer.

Note:

: The logical Link can be created in either in UWBAPI_SESSION_IDLE or UWBAPI_STATUS_SESSION_ ACTIVE, In case of Connection Oriented mode if it is called in UWBAPI_SESSION_IDLE the application has to take care of the synchronization of UwbApi_LogicalLinkSendData till the LOGICAL_LINK_CREATE_NTF with LOGICAL_CO_LINK_CONNECTED is received.

Note:

In case of SHORT_ADDR mode is used, then each Octet from octets 2 - 7 shall be set to 0x00.

Parameters

- **phLogicalLinkCreateCmd** – **[in]** - Pointer to structure that contains params to create Logical Link.
- **pLogicalLinkConnectId** – **[out]** - Pointer to uint32 that return the Logical Link connect ID.

Return values

- **UWBAPI_STATUS_OK** – on success
- **UWBAPI_STATUS_REJECTED** – if UWB stack is not initialized
- **UWBAPI_STATUS_SESSION_NOT_EXIST** – if Session is not existing or not created.
- **UWBAPI_STATUS_TIMEOUT** – if command is timeout
- **UWBAPI_STATUS_INVALID_PARAM** – if invalid parameters are passed
- **UWBAPI_STATUS_FAILED** – otherwise

tUWBAPI_STATUS **UwbApi_CloseLogicalLink**(*uint32_t LogicalLinkConnectId*)

Closes Logical Link for Data Transfer, that was established before.

Parameters

LogicalLinkConnectId – **[in]** - Session Handle value for which Logical Link would be created.

Return values

- **UWBAPI_STATUS_OK** – on success
- **UWBAPI_STATUS_REJECTED** – if UWB stack is not initialized
- **UWBAPI_STATUS_SESSION_NOT_EXIST** – if Session is not existing or not created.
- **UWBAPI_STATUS_TIMEOUT** – if command is timeout
- **UWBAPI_STATUS_INVALID_PARAM** – if invalid parameters are passed
- **UWBAPI_STATUS_FAILED** – otherwise

All information provided in this document is subject to legal disclaimers.
© 2026 NXP B.V. All rights reserved.

tUWBAPI_STATUS **UwbApi_LogicalLinkSendData**(*phLogicalLinkDataPkt_t* **pllSendData*)

Host shall use this API to send data over Logical Link.

Parameters

`pllSendData` – **[in]** Send Data Content

Return values

- `UWBAPI_STATUS_OK` – on success
- `UWBAPI_STATUS_NOT_INITIALIZED` – if UWB stack is not initialized
- `UWBAPI_STATUS_INVALID_PARAM` – if invalid parameters are passed
- `UWBAPI_STATUS_TIMEOUT` – if command is timeout
- `UWBAPI_STATUS_REJECTED` – if session is not established when data packet sent
- `UWBAPI_STATUS_NO_CREDIT_AVAILABLE` – if buffer is not available to accept data
- `UWBAPI_STATUS_DATA_TRANSFER_ERROR` – if data is not sent due to an unrecoverable error
- `UWBAPI_STATUS_FAILED` – otherwise

tUWBAPI_STATUS **UwbApi_LogicalLinkGetParams**(*uint32_t ConnectionIdentifier*, *phLogicalLinkGetParamsRsp_t* **phLogicalLinkGetParamsRsp*)

Gets the Logical Link parameters for the given connect ID. The API can be called to get the Logical Link parameters used for the logical link associated to the LL_CONNECT_ID or the Session Handle associated to the session.

Note:

To get the Link parameters of the Link created, this API needs to be called in UWBAPI_STATUS_SESSION_ACTIVE with the LL_CONNECT_ID. To get the default Link Params for the session, this API is to be called in either in UWBAPI_STATUS_SESSION_ACTIVE or UWBAPI_STATUS_SESSION_IDLE with session handle of the session.

Parameters

- `ConnectionIdentifier` – **[in]** - This can be either the CONNECT_ID or the Session Handle. Identifies the link or the data transfer session/phase params.
- `phLogicalLinkGetParamsRsp` – **[out]** - Pointer to structure that contains params for Logical Link.

Return values

- `UWBAPI_STATUS_OK` – On success
- `UWBAPI_STATUS_REJECTED` – If UWB stack is not initialized
- `UWBAPI_STATUS_SESSION_NOT_EXIST` – If Session is not existing or not created.
- `UWBAPI_STATUS_TIMEOUT` – If command is timeout
- `UWBAPI_STATUS_INVALID_PARAM` – If invalid parameters are passed
- `UWBAPI_STATUS_FAILED` – Otherwise

1.1.5 UWB Functional APIs (SR1XX/SR2XX Specific)

*group***uwb_apis_srxxx**

Various SR1XXT/SR2XXT Specific APIs

APIs for SR100 and SR150

APIs for SR100 and SR150

Defines

All information provided in this document is subject to legal disclaimers.

© 2026 NXP B.V. All rights reserved.

UWB_MAX_DEV_MGMT_RSP_TIMEOUT

UWBS response timeout

UWBD_DPD_TIMEOUT_MIN

Minimum timeout Supported by UWBS

UWBD_DPD_TIMEOUT_MAX

Maximum timeout Supported by UWBS

UWBD_VERSION_LENGTH_MAX

Maximum length of version Supported by UWBS

UWB_TAG_CMAC_LENGTH

UWBS_MAX_UCI_PACKET_SIZE

UWBS MAX UCI packet size

HOST_MAX_UCI_PACKET_SIZE

HOST MAX UCI packet size

MIN_DEVICE_PACKET_SIZE

MIN DEVICE packet size

EXTENDED_PARAM_ID_MASK

EXTENDED PARAM ID MASK

CCC_INFO_ID

CCC PARAM ID

MAX_UCI_CCC_VERSION_LEN

MAX UCI CCC Version Length

MAX_CCC_VERSION_LEN

MAX CCC Version Length

MAX_ALIRO_SPEC_VERSION_LEN

MAX Aliro Spec Version

MODULE_MAKER_ID_MAX_SIZE

MODULE MAKER ID MAX SIZE

MODULE_MAKER_ID_MAX_SIZE_FW

MAX_SESSION_ID_LIST

MAX SESSION ID LIST

MAX_UWB_CHIP_ID_LEN

MAX UWB CHID ID Length

MAX_PPM_VALUE_LEN

MAX UWBS PPM VALUE Length

MAX_TX_POWER_LEN

MAX TX Power Lenght

FW_BOOT_MODE_LEN

MAX FW BOOT Length

MAX_CAL_MODE_LEN

All information provided in this document is subject to legal disclaimers.
© 2026 NXP B.V. All rights reserved.

MAX CAL Mode Length

FW_GIT_HASH_LEN

FW GIT Hash Length

RANDOM_KEY_LEN

SESSION_KEY_LEN

WRAPPED_RDS_LEN

MAX_RDS_LIST_SIZE

Max RDS List Size

CHIP_CALIBRATION_POS

TX_POWER_POS

XTAL_CAP_VALUES_POS

RSSI_CONSTANT1_POS

RSSI_CONSTANT2_POS

TX_POWER_PARAMS_POS

PAPPPA_CALIB_CTRL_POS

TX_TEMP_COMP_POS

DELAY_CALIB_POS

PDOA_MFG_OFFSET_POS

PDOA_AOA_ANT_MULTIPOINT_CALIB

UWB_ANTENNA_SELECTION_GPIO_BIT_EF1_MASK

antenna selection gpio bit mask
Antenna selection gpio bit mask for EF1

UWB_ANTENNA_SELECTION_GPIO_BIT_EF2_MASK

Antenna selection gpio bit mask for EF2

UWB_ANTENNA_SELECTION_GPIO_BIT_GPIO6_MASK

Antenna selection gpio bit mask for GPIO6

UWB_ANTENNA_SELECTION_GPIO_BIT_GPIO7_MASK

Antenna selection gpio bit mask for GPIO7

UWB_ANTENNA_SELECTION_GPIO_BIT_GPIO9_MASK

Antenna selection gpio bit mask for GPIO9

UWB_ANTENNA_SELECTION_GPIO_BIT_GPIO10_MASK

Antenna selection gpio bit mask for GPIO10

UWB_ANTENNA_SELECTION_GPIO_BIT_GPIO11_MASK

Antenna selection gpio bit mask for GPIO11

UWB_ANTENNA_SELECTION_GPIO_BIT_GPIO14_MASK

Antenna selection gpio bit mask for GPIO14

MAX_RFRAME_MEAS

All information provided in this document is subject to legal disclaimers.
© 2026 NXP B.V. All rights reserved.

MAX RFRAME Measurements

MODELID_BOARD_TYPE

MODELID_CHIP_TYPE

RANGE_DATA_NTF_BOUND_AOA

RNG_DATA_NTF_PROXIMITY_FAR

RNG_DATA_NTF_PROXIMITY_NEAR

RNG_DATA_NTF

kAPPPARAMS_Type_Unknown

kAPPPARAMS_Type_u32

kAPPPARAMS_Type_au8

CSA_MAC_MODE

UWB_SET_APP_PARAM_VALUE(*PARAM, VALUE*)

Macro to set SetApp Configuration parameters value supported in UWB API layer.

UWB_SET_GETAPP_PARAM(*PARAM*)

Macro to set GetApp Configuration parameters value supported in UWB API layer.

UWB_SET_APP_PARAM_ARRAY(*PARAM, ARRAY, LENGTH*)

Macro to set SetApp/GetApp Configuration parameters array value supported in UWB API layer.

UWB_SET_VENDOR_APP_PARAM_VALUE

UWB_SET_GETVENDOR_APP_PARAM

UWB_SET_VENDOR_APP_PARAM_ARRAY

UWB_SET_DEBUG_PARAM_VALUE_u8(*PARAM, VALUE*)

Macro to set Set Debug Configuration parameters value supported in UWB API layer.

UWB_SET_DEBUG_PARAM_VALUE_u16(*PARAM, VALUE*)

UWB_SET_DEBUG_PARAM_VALUE_u32(*PARAM, VALUE*)

UWB_SET_GETDEBUG_PARAM_u8(*PARAM*)

Macro to set Get Debug Configuration parameters value supported in UWB API layer.

UWB_SET_GETDEBUG_PARAM_u16(*PARAM*)

UWB_SET_GETDEBUG_PARAM_u32(*PARAM*)

UWB_SET_DEBUG_PARAM_ARRAY(*PARAM, ARRAY, LENGTH*)

Typedefs

typedef UWB_AppParams_type_t **SetAppParams_type_t**

typedef UWB_AppParams_value_au8_t **SetAppParams_value_au8_t**

typedef UWB_AppParams_value_t **SetAppParams_value_t**

typedef UWB_AppParams_List_t **SetAppParams_List_t**

Enums

enum **ephBindingStatus**

All information provided in this document is subject to legal disclaimers.

© 2026 NXP B.V. All rights reserved.

Enumerated values for Secured Element Binding Status
Values:

enumerator **BindingStatus_NotBound**

Binding status 0x00: Not Bound,

enumerator **BindingStatus_BoundUnlocked**

0x01: Bound Unlocked,

enumerator **BindingStatus_BoundLocked**

0x02: Bound Locked,

enumerator **BindingStatus_Unknown**

0x03: Unknown (if any error occurred during getting binding state from SE)

enum **otpRWOption**

OTP Read Write Options.
Values:

enumerator **CALIB_PARAM**

enum **eAppConfig**

SetApp Configuration parameters supported in UWB API layer.
Values:

enumerator **UWB_DEVICE_TYPE**

Devce Type
- 0x00 = Controlee
- 0x01 = Controller

enumerator **RANGING_ROUND_USAGE**

Ranging Round Usage
- 0x00 = One Way Ranging UL-TDoA
- 0x01 = SS-TWR with Deferred Mode
- 0x02 = DS-TWR with Deferred Mode
- 0x03 = SS-TWR with Non-deferred Mode
- 0x04 = DS-TWR with Non-deferred Mode
- 0x05 = One Way Ranging DL-TDOA
- 0x06 = OWR for AoA Measurement
- 0x07 = eSS-TWR with Non-deferred Mode for Contention-based ranging
- 0x08 = aDS-TWR for Contention-based ranging
- 0x09 = Data Transfer Mode

enumerator **STS_CONFIG**

STS Config
- 0x00:Static STS
- 0x01:Dynamic STS
- 0x02:Dynamic STS for controlee individual key
- 0x03:Provisioned STS
- 0x04:Provisioned STS for Responder specific Sub-session Key
- 0xA0:To be set at Anchor and User device to distinguish the transition from Static STS to Dynamic STS
- 0x05 to 0xFF except 0xA0 : RFU

enumerator **MULTI_NODE_MODE**

Multi Node Mode

All information provided in this document is subject to legal disclaimers.
© 2026 NXP B.V. All rights reserved.

- 0x00 = O2O (One to one)
- 0x01 = O2M (One to many) Values 0x02 to 0xFF = RFU

enumerator **CHANNEL_NUMBER**

Possible values are {5, 6, 8, 9, 10, 12, 13, 14} (default = 9)

enumerator **NO_OF_CONTROLEES**

Number of Controlees To be configured by Host when MULTI_NODE_MODE is set other than 0x00. Number of Controlees(N) 1<=N<=8 (Default is 1)

enumerator **DEV_MAC_ADDRESS**

Device MAC Address MAC Address of the UWBS itself participating in UWB session. Size of this config is based on the MAC_ADDRESS_MODE.

Note:

In case of Extended MAC Addr mode, this config is to be set through UwbApi_SetAppConfigMultipleParams.

enumerator **DST_MAC_ADDRESS**

Destination MAC Addr MAC Address of the UWBS itself participating in UWB session. Size of this config is based on the MAC_ADDRESS_MODE.

Note:

In case of Extended MAC Addr mode, this config is to be set through UwbApi_SetAppConfigMultipleParams.

enumerator **SLOT_DURATION**

Unsigned integer that specifies duration of a ranging slot in the unit of RSTU (Ranging/Radar Standard Time Unit) (default = 2400)

enumerator **RANGING_DURATION**

Ranging Duration in ms Ranging duration in the unit of 1200 RSTU which is 1 ms. (default = 200)

enumerator **STS_INDEX**

STS index init value

enumerator **MAC_FCS_TYPE**

MAC FCS TYPE

- 0x00 = CRC 16 (default)
- 0x01 = CRC 32

enumerator **RANGING_ROUND_CONTROL**

1:Enable, 0:Disable

- Below bits are applicable when SCHEDULED_MODE is set to 0x01(Time scheduled ranging)
  - b0 - Measurement Report Phase
  - b1 - Control Phase
  - b2 - Configuration of RCP in Non-deferred Mode TWR
  - b3 : b5 - RFU
  - b6 - Measurement Report Phase (MRP) [UWBS shall ignore this bit]
  - b7 - Measurement Report Message (MRM) (default = 0x03)
- Below bits are applicable when SCHEDULED_MODE is set to 0x00(Contention-based ranging)
  - b0 - Ranging Result Report Message (RRRM) UWBS shall ignore this bit
  - b1 - 1 (Controller shall send a CM in-band and a Controlee shall expect a CM in-band)
  - b2 - 1 (RCP is excluded in Ranging Round)
  - b5 : b3 = RFU
  - b6 - Measurement Report Phase (MRP) ; If set to 0, MRP is not present (default) ; If set to 1, MRP is present
  - b7 - Measurement Report Message (MRM) UWBS shall ignore this bit. (default = 0x06)

enumerator **AOA_RESULT_REQ**

All information provided in this document is subject to legal disclaimers. © 2026 NXP B.V. All rights reserved.

AOA RESULT REQ

- 0x00 = AoA results are disabled.
- 0x01 = AoA results are enabled(default), return all the AOA type supported by the device
- 0x02 = Only AoA Azimuth is enabled
- 0x03 = Only AOA Elevation is enabled

enumerator **SESSION_INFO_NTF**

SESSION_INFO_NTF

- 0x00 = Disable range data SESSION_INFO_NTF
- 0x01 = Enable range data SESSION_INFO_NTF (default)
- 0x02 = Enable range data SESSION_INFO_NTF while inside proximity range
- 0x03 = Enable range data SESSION_INFO_NTF while inside AoA (upper and lower) bounds
- 0x04 = Enable range data SESSION_INFO_NTF while inside AoA bounds as well as inside proximity range
- 0x05 = Enable range data SESSION_INFO_NTF only when entering and leaving proximity range.
- 0x06 = Enable range data SESSION_INFO_NTF only when entering and leaving AoA (upper and lower) bound
- 0x07 = Enable range data SESSION_INFO_NTF only when entering and leaving AoA bounds as well as entering and leaving proximity range.

enumerator **NEAR_PROXIMITY_CONFIG**

Proximity near in cm (default = 0)

enumerator **FAR_PROXIMITY_CONFIG**

Proximity far in cm (default = 20000)

enumerator **DEVICE_ROLE**

Device Role

- 0x00 = Responder
- 0x01 = Initiator
- 0x02 = Assigned
- 0x03 = Assigned
- 0x04 = Assigned
- 0x05 = Advertiser
- 0x06 = Observer
- 0x07 = DT-Anchor
- 0x08 = DT-Tag

enumerator **RFRAME_CONFIG**

Activate or deactivate RSSI reporting

- 0x00 = SP0 (Reserved value for test purpose)
- 0x01 = SP1
- 0x02 = RFU
- 0x03 = SP3 Values 0x04 to 0xFF = RFU (default = 0x03)

enumerator **RSSI_REPORTING**

Activate or deactivate RSSI reporting

- 0x00 : Disable(default)
- 0x01 : Enable
- 0x02-0xFF :RFU

enumerator **PREAMBLE_CODE_INDEX**

Preamble Code Index

- [9-12] - BPRF,
- [25-32] - HPRF, used for RADAR_MODE 1, 2 and 3

All information provided in this document is subject to legal disclaimers.
© 2026 NXP B.V. All rights reserved.

- [95-102] - LPRF used for RADAR_MODE 5 and 6 (default = 10)

enumerator **SFD_ID**

[0,2]:BPRF, [1,2,3,4]:HPRF (default = 2)

enumerator **PSDU_DATA_RATE**

PSDU DATA RATE
- 0x00 = 6.81 Mbps (default)
- 0x01 = 7.80 Mbps
- 0x02 = 27.2 Mbps
- 0x03 = 31.2 Mbps
- 0x04 = 850 Kbps Values 0x00, 0x02, 0x04 map to K=3 and 0x01, 0x03 map to K=7.

enumerator **PREAMBLE_DURATION**

Preamble Duration
- 0:32 symbols
- 1:64 symbols
- 0xA0:1024 symbols

enumerator **LINK_LAYER_MODE**

Link Layer Mode
- 0x00 – Bypass Mode (Default),
- 0x01 – Logical link data transfer

enumerator **DATA_REPETITION_COUNT**

Data Repetition Count
- 0x00 – No repetition (Default),
- 0xFF – Repeat infinite number of times

enumerator **RANGING_TIME_STRUCT**

1:Block based (default) & 0:[2-255]:RFU

enumerator **SLOTS_PER_RR**

Number of slots for per ranging round (default = 25)

enumerator **AOA_BOUND_CONFIG**

AOA_BOUND_CONFIG
- [1:0] AOA_BOUND_CONFIG_LOWER_BOUND_AOA_AZIMUTH range -180 (default) to 180
- [3:2] AOA_BOUND_CONFIG_UPPER_BOUND_AOA_AZIMUTH range -180 to 180 (default)
- [5:4] AOA_BOUND_CONFIG_LOWER_BOUND_AOA_ELEVATION -90 (default) to 90
- [7:6] AOA_BOUND_CONFIG_UPPER_BOUND_AOA_ELEVATION -90 to 90 (default)

enumerator **PRF_MODE**

PRF MODE
- 0x00 = 62.4 MHz PRF. BPRF mode (default)
- 0x01 = 124.8 MHz PRF. HPRF mode.
- 0x02 = 249.6 MHz PRF. HPRF mode with data rate 27.2 and 31.2 Mbps

enumerator **CAP_SIZE_RANGE**

Configuration parameter sets the minimum and maximum CAP size to be used by the Controller/Initiator in the Contention-based ranging session, in the units of Ranging Slots.
- Octet [0] - represents the maximum CAP size. Default = SLOTS_PER_RR-1.
- Octet [1] - represents the minimum CAP size. Default = 5.

enumerator **SCHEDULED_MODE**

Parameter is used to set the multi-node Ranging Type.

All information provided in this document is subject to legal disclaimers.
© 2026 NXP B.V. All rights reserved.

- 0x00 = Contention-based ranging
- 0x01 = Time scheduled ranging (default)
- 0x02 = Hybrid-based ranging Values 0x03 to 0xFF = RFU

enumerator **KEY_ROTATION**

Key Rotation
- 1: Enable,
- 0:Disable [Default]

enumerator **KEY_ROTATION_RATE**

Key Rotation rate 2^n where 0<=n<=15

enumerator **SESSION_PRIORITY**

Session Priority 1-100, default : 50

enumerator **MAC_ADDRESS_MODE**

MAC address mode Default 0 [SHORT]

enumerator **VENDOR_ID**

Unique vendor Id

enumerator **STATIC_STS_IV**

Vendor defined static sts

enumerator **NUMBER_OF_STS_SEGMENTS**

Number of STS segments in the frame. 0x00:No STS Segments(if PPDU_COFIG is 0). If PPDU Config is set to 1 or 3 then 0x01:1 STS Segment(default), 0x02:2 STS Segments

enumerator **MAX_RR_RETRY**

Number of Failed Ranging Round attempts before terminating the session. Default : 0

enumerator **UWB_INITIATION_TIME**

Indicates when ranging operation shall start after ranging start request is issued from AP. Default : 0

Note:

UWB_INITIATION_TIME in past is accepted for Aliro Session Type or if ALIRO_CONTROLEE_EXTENSIONS = 0x01 (Enable)

enumerator **HOPPING_MODE**

Modes for the hopping.
- 0x00: No hopping
- 0x02: Adaptive hopping using MODULO
- 0x03: continuous hopping using MODULO
- 0x04: adaptive hopping using AES
- 0x05: continuous hopping using AES Default : 0

eCCC_HopModes_t

enumerator **BLOCK_STRIDE_LENGTH**

Block Stride Length. 0x00:Default, [0x01-0xFF]:Application use case specific value

enumerator **RESULT_REPORT_CONFIG**

Config to enable result report, 0: Disable, This is applicable only RANGING_ROUND_CONTROL enabled

enumerator **IN_BAND_TERMINATION_ATTEMPT_COUNT**

Indicates how many times in-band termination signal needs to be sent by controller/initiator to a controlee device. Default : 1

enumerator **SUB_SESSION_ID**

All information provided in this document is subject to legal disclaimers. © 2026 NXP B.V. All rights reserved.

Sub-Session Handle for the controlee device. This config is mandatory and it is applicable if STS_CONFIG is set to 2 for controlee device

enumerator **BPRF_PHR_DATA_RATE**

PHR coding rate:
- 0x00 = 850 kbps (default)
- 0x01 = 6.81 Mbps

enumerator **MAX_NUMBER_OF_MEASUREMENTS**

Maximum Number of ranging blocks to executed in a session Default : 0

In case of RANGING_ROUND_USAGE is set to 0x05 (OWR DL-TDoA), then this parameter indicates max number of ranging blocks to be executed in a session.

enumerator **UL_TDOA_TX_INTERVAL**

This parameter specifies the average transmission interval of Blink UTMs from UT-Tags and/or Synchronization UTMs from UT-Synchronization Anchors, as defined by the UL-TDoA TX Interval MAC configuration parameter.

By default, UL_TDOA_TX_INTERVAL = 2000ms.

enumerator **UL_TDOA_RANDOM_WINDOW**

Length of the randomization window within which Blink and Synchronization UTMs may be transmitted.

enumerator **STS_LENGTH**

STS length
- 0: 32 symbols ,
- 1: 64 symbols(Default),
- 2: 128 symbols

enumerator **SUSPEND_RANGING_ROUNDS**

configuration allows the Ranging Rounds to be suspended

enumerator **UL_TDOA_NTF_REPORT_CONFIG**

UT-Anchor configuration to specify if UL-TDoA related SESSION_INFO_NTF

enumerator **UL_TDOA_DEVICE_ID**

This value shall be used to specify the length and presence of the UL-TDoA Device ID in UTMs.

enumerator **UL_TDOA_TX_TIMESTAMP**

Presence and length of TX timestamps in UTMs.

enumerator **MIN_FRAMES_PER_RR**

Minimum number of frames to be transmitted in a ranging round.(default = 4)

enumerator **MTU_SIZE**

Maximum Transfer Unit (MTU) Size represents the maximum size of allowed payload size to be transmitted in a frame

enumerator **INTER_FRAME_INTERVAL**

The configuration in units of 1200 RSTU to configure the interval between the RFRAMES transmitted in the “OWR for AoA Measurement” ranging round.(default = 1)

enumerator **DLTDOA_RANGING_METHOD**

DL-TdoA ranging round
- 0: SS-TWR
- 1: DS-TWR (default)

enumerator **DLTDOA_TX_TIMESTAMP_CONF**

All information provided in this document is subject to legal disclaimers.
© 2026 NXP B.V. All rights reserved.

DL-TdoA Tx timestamp conf

- b0: TX timestamp type
- b1-2: TX timestamp length Default: 00000011

enumerator **DL_TDOA_HOP_COUNT**

controls cluster sync field 1: inter cluster sync filed in every poll DTM

enumerator **DLTDOA_ANCHOR_CFO**

DL-TdoA anchor CFO

- 0: not included,
- 1:Anchor CFO included (default)

enumerator **DLTDOA_ANCHOR_LOCATION**

DL-TdoA anchor location

enumerator **DLTDOA_TX_ACTIVE_RANGING_ROUNDS**

DL-TdoA tx active ranging rounds

- 0: not present (default)
- 1: present

enumerator **DL_TDOA_BLOCK_SKIPPING**

To configure number of blocks that shall be skipped by a DT-Tag between two active ranging blocks, 0x00 : No blocks striding(default) 0x01-0xFF : no. of blocks to be skipped by DT-Tag

enumerator **DLTDOA_TIME_REF_ANCHOR**

global time rference of dl-tdoa network 0: Disable 1: Set global metric time

enumerator **SESSION_KEY**

Session Key provided for Provisioned STS mode (STS_CONFIG equal to 0x03 or 0x04)

- If the Session Key is not provided by the Host in Provisioned STS mode, the UWBS shall fetch the Session Key from the Secure Component. This parameter is valid only in Provisioned STS mode and shall be ignored otherwise.

enumerator **SUB_SESSION_KEY**

Sub-session Key provided for Provisioned STS for Responder specific Key mode (STS_CONFIG equal to 0x04).

- If the Sub-session Key is provided by the Host, the Host shall also provide the SESSION_KEY.
- If the Sub-session Key is not provided by the Host for Provisioned STS for Responder specific Key mode, the UWBS shall fetch the Sub-session Key from the Secure Component. This parameter is valid only in Provisioned STS for Responder specific Key mode and shall be ignored otherwise.

enumerator **SESSION_DATA_TRANSFER_STATUS_NTF_CONFIG**

This parameter is used to configure the SESSION_DATA_TRANSFER_STATUS_NTF.

- 0x00 = Disable SESSION_DATA_TRANSFER_STATUS_NTF (Default).
- 0x01 = Enable SESSION_DATA_TRANSFER_STATUS_NTF. If SESSION_DATA_TRANSFER_STATUS_NTF is disabled, then the UWBS shall not send SESSION_DATA_TRANSFER_STATUS_NTF for every Application Data transmission except for last transmission.

enumerator **SESSION_TIME_BASE**

Configures a reference time base for the given session.

- Octet 0:
  - b0: Reference time base feature
    - 0b0 = Disable (default)

All information provided in this document is subject to legal disclaimers.
© 2026 NXP B.V. All rights reserved.

  - 0b1 = Enable
- b1: continue/stop the session(s) when reference session is not in SESSION_STATE_ACTIVE Session State
  - 0b0 = stop (default)
  - 0b1 = continue
- b2: Resync time grid in case the reference session will become active again after it has been inactive.
  - 0b0 = No resync (default)
  - 0b1 = Resync
- Octet 1-5: Session Handle of the reference session
- Octet 5-8: Session offset time in microseconds

enumerator **DL_TDOA_RESPONDER_TOF**

This parameter specifies whether a DT-Anchor with the Responder role in a given ranging round shall include the estimated Responder ToF Result in a Response DTM.

- 0x00: Responder ToF Result shall not be added to Response DTMs (default).
- 0x01: Responder ToF Result shall be added to Response DTMs.

enumerator **SECURE_RANGING_NEFA_LEVEL**

This parameter is required for PHY-layer security level in the form of an upper limit. Normalized effective false acceptance rate (NEFA) as defined in spec, during secure ranging operation.

- 0x00 = default, NEFA <= 1.0
- 0x01 = Low, NEFA <= 2-10
- 0x02 = Medium, NEFA <= 2-20
- 0x03 = High, NEFA <= 2-48
- 0x04-0xFF = RFU

Note:

This parameter is only used for secure ranging with Provisioned STS or Test mode. A given implementation of UWBS might achieve a better level of PHY-layer security performance than the permissible upper limit set by the NEFA Level.

enumerator **SECURE_RANGING_CSW_LENGTH**

The length of the PHY-layer critical search window (CSW) as defined in spec. In the course of secure ranging operation. The unit of this parameter is 0.25 meter. For example, a value of “8” corresponds to a distance of 2 meters. If not present its value shall be set to 0x04, a distance of 1 meter

Note:

Only used for secure ranging with Provisioned STS or Test mode.

enumerator **APPLICATION_DATA_ENDPOINT**

Local endpoint configuration of the session It defines which endpoint is used by the UWBS for Application data exchange using the non-secure or secure message connection. When using the Bypass mode, all data shall be exchanged using the Non-secure endpoint Values

- b3-b0: Non-secure end point configuration
  - 0x0 : Host (default)
  - 0x1: Secure Component 0xF- 0x2 : RFU
- b7-b4: Secure end point configuration
  - 0x0 : Host (default)
  - 0x1: Secure Component 0xF- 0x2 : RFU

enumerator **HOP_MODE_KEY**

Key to generate hopping sequence. This value is used for both AES and MODULO hopping formula. For MODULO hopping, only first 4 bytes are used as converted to 4byte integer. (default key for AES hopping formula = 0x4c,0x57,0x72,0xbc) (default key for MODULO hopping formula = 0xbc72574c)

enumerator **RESPONDER_SLOT_INDEX**

1: responder1,…,N: responder N

All information provided in this document is subject to legal disclaimers. © 2026 NXP B.V. All rights reserved.

enumerator **RANGING_PROTOCOL_VER**

Version of the ranging protocol (defined by CCC) [0x0000 – 0xFFFF] (default = 0x0100)

enumerator **UWB_CONFIG_ID**

UWB Configuration ID [0x0000 – 0xFFFF] (default = 0x0001)

enumerator **PULSESHAPE_COMBO**

Pulse Shape Combinations. Possible combinations are written in format: Pulse shape combo value - Initiator transmit pulse shape - Responder transmit pulse shape

- 0x00 - 0x0 - 0x0
- 0x01 - 0x0 - 0x1
- 0x02 - 0x0 - 0x2
- 0x10 - 0x1 - 0x0
- 0x11 - 0x1 - 0x1
- 0x12 - 0x1 - 0x2
- 0x20 - 0x2 - 0x0
- 0x21 - 0x2 - 0x1
- 0x22 - 0x2 - 0x2 Support for value 0x00 is mandatory. (default = 0x00)

enumerator **URSK_TTL**

URSK expiration time, in minutes (max 12 hours). After this time from setting URSK, the session will go to idle. [0x001 - 0x2D0] (default = 0x2D0)

enumerator **RESPONDER_LISTEN_ONLY**

Responder is in listen only mode. If it is enabled then the responder will not send range frame. In this mode, the responder will report 0xD (responder is in listen only mode) as higher nibble of ranging status in RANGE_CCC_DATA_NTF 0x00: Responder is in normal mode. 0x01: Responder is in listen only mode (default = 0x00)

enumerator **LAST_STS_INDEX_USED**

Parameter used to get the STS index of the UWB session. When SESSION_GET_APP_CONFIG_CMD issued for this config during SESSION_STATE_ACTIVE the UWBS shall return the last STS Index of the latest completed ranging block.

enumerator **ALIRO_MAC_MODE**

This configuration is used to configure:

1. The number of active ranging rounds in a RANGING_DURATION.
2. Offset between two active ranging rounds in a RANGING_DURATION.

- [b7-b6]: Number of active ranging round(s)
  - 0 = One active ranging round (default).
  - 1 = Two active ranging rounds (CSA use case).
  - 2 and 3 = RFU.
- [b5-b0]: Offset between two active ranging rounds.
  - 1 to (Nround - 1)

Note:

- *Bits [b5, b0] SHALL be set if [b7, b6] set to decimal value 1. Otherwise, bits [b5, b0] will be ignored.*
- *Nround is calculated based on RANGING_DURATION, SLOTS_PER_RR and SLOT_DURATION.*

enumerator **END_OF_SUPPORTED_APP_CONFIGS**

End of App Configs

enum **eVendorAppConfig**

Set Get Vendor App Configuration parameters supported in UWB API layer.

Values:

All information provided in this document is subject to legal disclaimers.

© 2026 NXP B.V. All rights reserved.

enumerator **MAC_PAYLOAD_ENCRYPTION**

This parameter shall enable disable encryption of Payload data

- 0x00 - Plain Text
- 0x01 - Encrypted(default)

enumerator **ANTENNAE_CONFIGURATION_TX**

The antenna used for TX. If Octet[0] of ANTENNAE_CONFIGURATION_TX is 0 and Octet[1] of ANTENNAE_CONFIGURATION_RX is 0, FW automatically enters Scan Phase.

- Octet[0] - Define number of TX Antennas to follow (default value is 1).
- Octet[1] - Tx Antennas ID as defined by ANTENNA_TX_DEFINE (default value is 1). So we transmit by default with Antennas ID 1 (As a pre requisite: an antenna with ID 1 using ANTENNA_TX_IDX_DEFINE must be explicitly pre-defined).
- Octet[2] - Tx Antennas ID as defined by ANTENNA_TX_DEFINE Must be 0 for SR100T, SR150

enumerator **ANTENNAE_CONFIGURATION_RX**

The session specific antenna configuration for Rx. If Octet[0] of ANTENNAE_CONFIGURATION_TX is 0, and Octet[0] of ANTENNAE_CONFIGURATION_RX is 0, FW automatically enters Scan Phase.

- Octet [0] : Mode of RX operation
  - 0 : Configuration ToA Mode
    - ToF Only Mode
    - Recommended for TEST_PER, TEST_RX, TEST_SSTWR test modes, Optionally TEST_LOOPBACK test modes (AoA may not be reported) (for HELIOS-2)
  - 1 : Configuration AoA Mode
    - Dual / Single AoA usecase
    - ToA Mode with implicit Rx mode as per ANTENNAS_RX_PAIR_DEFINE (for HELIOS-1)
    - Recommended for TEST_LOOPBACK test mode, Optionally TEST_PER, TEST_RX, TEST_SSTWR test modes
  - 2 : Configuration Mode 2: Radar Mode
    - Default 0x01 (SR100S only)
  - 3 : Configuration Mode 3: ToA usecase using different Rx antenna pair for RFM
  - 4 : Configuration Mode 4: AoA usecase using different Rx antenna pair for RFM
- Octet [1] : Number of Antennas or Antenna pairs to follow

Default Value:

- For all Sessions
  - Octet[0] : 0x00 = ToA Mode
  - Octet[1] : 0x01 = Number fo RX Antenna

Note:

Please refer the spec for more details on each configuration mode.

enumerator **RAN_MULTIPLIER**

Return the possible RAN multiplier value for a new session

enumerator **STS_LAST_INDEX_USED**

Parameter used to get the STS index of the UWB session. When GET_VENDOR_APP_CONFIG_CMD issued for this config during SESSION_STATE_ACTIVE the UWBS shall return the last

enumerator **CIR_LOG_NTF**

- 0x00: Disable (default)
- 0x01: Enable

enumerator **PSDU_LOG_NTF**

- 0x00: Disable (default)
- 0x01: Enable

All information provided in this document is subject to legal disclaimers.
© 2026 NXP B.V. All rights reserved.

enumerator **RSSI_AVG_FILT_CNT**

This parameter is used to filter out the outliers in RSSI measurements in PER RX Test. If the RSSI filtering count is set to N and total packet count is set to M then UWBS shall report the average of (M-2N) RSSI values in TEST_PER_RX_NTF excluding(N) maximum and (N) minimum RSSI values.

Note:

M is the total packet count to be received in PER Rx test (default = 0)

enumerator **CIR_CAPTURE_MODE**

CIR sampling position for incoming UWB packet

- bits[7:4] - CIR1 capture mode
- bits[3:0] - CIR0 capture mode

CIR capture modes:

- 0x0 - Pre SYNC RX1
- 0x1 - Pre SYNC RX2
- 0x2 - Pre STS RX1
- 0x3 - Pre STS RX2
- 0x4 - Post SYNC RX1
- 0x5 - Post SYNC RX2
- 0x6 - Post STS RX1
- 0x7 - Post STS RX2
- 0x8 - 0xF - RFU (default = 0x76)

enumerator **SESSION_SYNC_ATTEMPTS**

Number of times scheduler shall attempt to sync in controlee session before reporting error notification. This config is applicable for controlee session only. Range: [3 : 255] (Default: 3)

enumerator **SESSION_SCHED_ATTEMPTS**

Number of times scheduler shall attempt to schedule ranging round before reporting error notification Range: [1 : 255] (Default: 3)

enumerator **SCHED_STATUS_NTF**

Enable/disable SCHEDULER_STATUS_NTF

- 0x00 - Disable (default)
- 0x01 - Enable for including all the sessions information in notification
- 0x02 - Enable for include only failure sessions information in notification
- 0x03-0xFF: RFU

enumerator **TX_POWER_DELTA_FCC**

Session specific Tx power ID offset applied on top of Tx POWER calibration parameter. configured via SET_DEVICE_CALIBRATION_CMD.

- 0:No offset (default)
- 1 to 127: Attenuation (0.25 dB per steps)
- 128 to 255: RFU

enumerator **TEST_KDF_FEATURE**

This parameter is used to enable/disable KDF notification generation.

- 0x00: Disable (default)
- 0x01: Enable

enumerator **TX_POWER_TEMP_COMPENSATION**

This parameter is used to enable/disable Tx power temperature compensation

- 0x00: Disable (Default)
- 0x01: Enable

enumerator **WIFI_COEX_MAX_TOLERANCE_COUNT**

All information provided in this document is subject to legal disclaimers.
© 2026 NXP B.V. All rights reserved.

WiFi-CoEx maximum tolerance count, after the expiry of the number of count the UWBS shall make the “Medium Grant Request” with priority field set to “Critical”. This parameter can be modified when session is in SESSION_STATE_ACTIVE Session State. Range: [1 : 25] (Default: 3)

enumerator **ADAPTIVE_HOPPING_THRESHOLD**

This parameter can be used to configure the required number of successful responses(T) from Responders to conclude a successful ranging round. If numbers of responses is less than this given threshold(T) when Initiator device acting as Controller then initiator device triggers a hop to a different round index within the next block. Range: [0<T<= NUMBER_OF_CONTROLEES] (Default: NUMBER_OF_CONTROLEES)

Note:

This parameter is applicable when HOPPING_MODE = 0xA0 (NXP Adaptive Hopping mode is Enabled)

enumerator **AUTHENTICITY_TAG**

Config to enable/disable authenticity tag in RANGE_DATA_NTF. This config is applicable when STS_CONFIG is set to Dynamic STS.

- 0x00 = Disable (Default)
- 0x01 = Enable

enumerator **RX_NBIC_CONFIG**

- Octet [0] : Used to configure NBIC settings
  - b[0]: Enable / Disable NBIC.
    - 0 = Disable (default)
    - 1 = Enable
  - b[1:2]: Content of register MA_FILTER_BW_SET. Filter bandwidth setting (default: 0x3)
  - b[3:4]: Content of register MA_FILTER_BW_START_SET. Starting filter bandwidth setting for estimation (default: 0x3)
  - b[5:7]: RFU
- Octet [1]: Content of register PSD_WEIGHT_SET (Default: 0x14) (Default: 0x40 only applicable when NBIC is enabled)

enumerator **MAC_CFG**

Config is used to configure the MAC Header and MAC Footer

- b[0]: MAC Header present
- b[1]: MAC Footer present
- b[7:2]: RFU (Default value: 0x03 for FIRA Session) (Default value: 0x00 for Test Mode Session)

enumerator **SESSION_INBAND_DATA_TX_BLOCKS**

Amount of blocks which should be reserved for the given session for inband data transfer for transmitting data. If set to 0, transmitting inband data is not allowed for this session.

Note:

The sum of this value for all active session must not exceed UWBS_INBAND_DATA_MAX_BLOCKS and will prevent the first session which is causing to exceed the limit to get started. Default Value: 0 for Ranging Session UWBS_INBAND_DATA_BUFFER_BLOCK_SIZE for Data Session with DEVICE_TYPE set to Controlee 0 for Data Session with DEVICE_TYPE set to Controller

enumerator **SESSION_INBAND_DATA_RX_BLOCKS**

Amount of blocks which should be reserved for the given session for inband data transfer for receiving data. If set to 0, receiving inband data is not allowed for this session.

Note:

The sum of this value for all active session must not exceed UWBS_INBAND_DATA_MAX_BLOCKS and will prevent the first session which is causing to exceed the limit to get started. Default Value: 0 for Ranging Session UWBS_INBAND_DATA_BUFFER_BLOCK_SIZE for Data Session with DEVICE_TYPE set to Controller 0 for Data Session with DEVICE_TYPE set to Controlee

enumerator **DATA_TRANSFER_TX_STATUS_CONFIG**

This configuration shall be used to configure DATA_TRANSMISSION_STATUS_NTF indication

All information provided in this document is subject to legal disclaimers. © 2026 NXP B.V. All rights reserved.

- 0x00 : Always ON
- 0x01 : Always OFF
- 0x02 : Notify when error (Default: 0x00)

Note:

The UWBS shall always send DATA_TRANSMISSION_STATUS_NTF whenever it receives DATA_MESSAGE_SND, the subsequent DATA_TRANSFER_TX_STATUS_NTF on RF transmit shall be sent based on DATA_TRANSFER_TX_STATUS_CONFIG configuration

enumerator **RFRAME_LOG_NTF**

This configuration is used to enable/disable RFRAME LOG NTF.

- 0x00 = Disable (default)
- 0x01 = Enable Values 0x02 to 0xFF = RFU

enumerator **TX_ADAPTIVE_PAYLOAD_POWER**

This configuration is used to enable/disable adaptive payload power for TX.

- 0x00 = Disable
- 0x01 = Enable (default) Values 0x02 to 0xFF = RFU

enumerator **RML_PROXIMITY_CONFIG**

- Octet 0: RML_NEAR_PROXIMITY (default = 0) This parameter sets the lower bound in meters where the discovered devices are added into the RML list. Should be less than or equal to RML_FAR_ _CONFIG value.
- Octet 1: RML_FAR_PROXIMITY (default = 5) This parameter sets the upper bound in meters above which the RML list is not added with the discovered devices. Should be greater than or equal to RML_NEAR_CONFIG value.

enumerator **RESYNC_ENABLE**

This configuration is used to Enable/Disable sync with controller

- 0x00 : Re-sync is disabled
- 0x01 : Re-sync is enabled (Default)

RCM Re-sync enable:

- If the device misses 5 RCM messages and it is assumed that sync is lost(probably the controller isn't sending them), it will respond by increasing its listening time. Specifically, it will open its Rx window to twice the Ranging Duration(RANGING_DURATION).
- This extended listening continues until the device successfully re-establishes sync.

RCM Re-sync disable:

- Once it is assumed that sync is lost, Controlee will not open its RCM Rx window to twice the Ranging Duration(RANGING_DURATION).

Note:

- *The vendor-specific application parameter is applicable for Aliro sessions or if ALIRO_CONTROLEE_EXTENSIONS = 0x01 (Enable).*
- *If this parameter is set for any other session type, FW will return an INVALID_PARAM.*
- *If the resynchronization logic is disabled, the controlee may or may not re-sync again. In this case host has to restart the session in order to resync*

enumerator **FOV_ENABLE**

2D AoA FoV Processing Enable/Disable This parameter decides whether to enable or disable 2D AoA FoV Processing.

- 0x00 : 2D AoA FoV Processing Disabled (Default)
- 0x01 : 2D AoA FoV Processing Enabled

enumerator **AZIMUTH_FIELD_OF_VIEW**

Field of View for Horizontal Antenna Pair. This parameter indicates if the peer device is in the configured FoV of the device or not.

All information provided in this document is subject to legal disclaimers.
© 2026 NXP B.V. All rights reserved.

- Octet[0] : Horizontal RX Antenna Pair ID as defined in ‘ANTENNAS_RX_PAIR_DEFINE UCI parameter’. Value 0 shall be rejected.
- Octet[1] : FoV Coverage in degrees.

enumerator **CSA_FINAL_DATA2_CONFIG**

Configuration to enable/disable transmission of Final Data 2 from Responder to Initiator.

- 0x00 = Responder shall not transmit Final Data 2 message (default).
- 0x01 = Responder shall transmit the Final Data 2 message

enumerator **ENABLE_FOM**

Enable / Disable FoM processing.

- Bit[0] : 3D AoA FoM: Enable/Disable 3D AoA FoM Default is 0.
- Bit[7:1] : RFU

Note:

For ENABLE_FOM = 1, SLOT_DURATION should be greater than or equal to 2ms (2400 RSTU).

enumerator **SET_USECASE_ID**

- Octet [0]: Use Case
  - 0x00 = None (default)
  - 0x01 = Secure payment
  - 0x02 = NXP Static STS ranging
  - 0x03 = Mifare Static STS ranging
  - 0xFF-0x04 = RFU
- Octet [1-2]: Feature bitmask specific to Use case (Octet [0]) The below bitmask applies for Use Case set to 0x01
  - b0 = APDU Aggregation
    - 1: Enable (default)
    - 0: Disable
  - b1 = in-band static STS to dynamic STS switch
    - 1: Enable (default)
    - 0: Disable
  - b15-b2 = RFU

Note:

Octet [1-2] shall be ignore by UWBS when Octet [0] is set to 0x02

enumerator **SESSION_INFO_NTF_FILTER_NUM**

Represents a single byte number:

- 1~255 (default: 1)
- 0 = Invalid Range
- 1 = No filtering applied. Host is notified immediately with SESSION_INFO_NTF upon PROXIMITY criteria being met
- 2~255 = The number of consecutive Ranging Rounds which must meet the PROXIMITY criteria before a Notification is sent to the Host This can be applied to either an Initiator device or Responder device

Note:

Applicable for SESSION_INFO_NTF = 0x02, 0x05.

enumerator **RADAR_MODE**

RADAR mode .

- 0x01: Medium distance, e.g. used for vital sign detection .
- 0x02: Close distance, e.g. used for hand gesture recognition .
- 0x03: Far distance, e.g. used for presence detection .
- 0x04: RFU .
- 0x05: Medium distance, e.g. used for static object support .

All information provided in this document is subject to legal disclaimers.
© 2026 NXP B.V. All rights reserved.

- 0x06: Far distance, e.g. used for static object support . (Default value: 0x01) .

enumerator **RADAR_RX_GAIN**

- bits[31:24]
  - 0: Enable AGC
  - 1: Force gain index values
  - 2: Force AGC to be re-executed
- bits[23:16]: Gain index for RXA
- bits[15:8] : Gain index for RXB
- bits[7:0] : Gain index for RXC Default: 0x00000000

enumerator **RADAR_SINGLE_FRAME_NTF**

Defines the mode used to transmit CIR samples.

- 0x00: use a buffer to transmit the CIR samples if the buffer is . full or session switch will be performed.
- 0x01: transmit every single CIR sample . (Default: 0) .

enumerator **RADAR_CIR_NUM_SAMPLES**

Number of TAPs to be captured . (Default = 128, MIN = 16, MAX = 128) .

enumerator **RADAR_CIR_START_OFFSET**

Defines the start index of the CIR in case of Radar modes using HPRF:

- Octet[0]: Length in Words (Default:3)
  - Word[1] start index for RXC for antenna pair 0
  - Word[2] start index for RXB for antenna pair 0
  - Word[3] start index for RXA for antenna pair 0
  - Word[4] start index for RXC for antenna pair 1
  - Word[5] start index for RXB for antenna pair 1
  - Word[6] start index for RXA for antenna pair 1 (Default: 0) If length is set to 3, octet 4 / 5 / 6 shall not be present. All other values except 3 and 6 for length are invalid. Start index of 0 triggers the automatic calculation of the start index; Any other value sets the start index (HPRF range is[1:727])

enumerator **RADAR_RFRI**

RFRI (Radar Frame Repetition Interval):

- Octet[3:0]: RANGING_INTERVAL
  - Duration between the start of two, consecutive Radar rounds in ms
- Octet[5:4]: SLOT_DURATION
  - Duration between the start times of two consecutive Radar frames in RSTU.
- Octet[6]: SLOTS_PER_RR
  - Number of Radar frames within the Radar round

enumerator **RADAR_PRESENCE_DET_CFG**

This UCI parameter shall check the following if Presence Detection Mode is Enabled:

- RADAR_MODE with value set as 1
- RADAR_CIR_NUM_SAMPLES with value must be greater than (MaxDistance in cm/15cm)
- Octet[0]: Presence Detection Mode
  - Default: Disabled
  - Bit 0: Presence detection enabled
  - Bit 1: Distance and AoA reporting enabled
  - Bit 2: Report all the targets
  - Bit 3: CIR Injection used
  - Bit 4: Detection trigger value reporting
  - If Bit 4 is set, also Bit 1 has to be set.
- Octet[1]: Periodic data reporting
  - Default:0

All information provided in this document is subject to legal disclaimers.
© 2026 NXP B.V. All rights reserved.

  - Bit 0: sending raw CIRs to the host
  - Bit 2-1:
    - 00: no periodic presence reporting
    - 01: presence reporting every 50ms
    - 10: presence reporting every 400ms
    - 11: presence reporting every 1600ms
- Octet[2] - Presence Detection Sensitivity in Q4.4 Format (4 bits for Fractional, 4 bits for integer)
  - Default: 3.75
  - Min: 1, Max 7
- Octet[3] - Set to enable GPIO to notify presence
  - Bit 3-0: GPIO line to be used, 0 disabled
  - 1 : GPIO1 - 9 : GPIO9
  - 2 : GPIO2 -10 : GPIO10
  - 3 : GPIO3 -11 : GPIO11
  - 4 : GPIO4 -12 : GPIO12
  - 5 : GPIO5 -13 : GPIO13
  - 6 : GPIO6 -14 : GPIO14
  - 7 : GPIO7 -15 : GPIO15
  - 8 : GPIO8
  - Bit 6-4: RFU
  - Bit 7: 1 to disable RADAR_RX_NTF(Presence Detection)
  - (Default:0)
  -
- Octet[4-5]: Presence Detection Distance Min in cm
  - Default: 30
  - Min: 30, Max - 800
- Octet[6-7]: Presence Detection Distance Max in cm
  - Default: 800
  - Min: 50, Max - 800
- Octet[8-9]: Presence Detection Hold Delay : delay until presence is reported as lost in ms
  - Default: 1600
  - Min: 400, Max:5000
- Octet[10]: Set Minimum Angle
  - Default: -90, range -90, +90, int8_t format, default means that no minimum angle is used
- Octet[11]: Set Maximum Angle
  - Default: 90, range -90, +90, int8_t format, needs to be greater than minimum angle, default means that no maximum angle is used

Note:

: Both GPIO and PD notification should not be disabled at one time. FW will return invalid range error.

enumerator **RADAR_FCC_TEST_MODE**

Parameter to test FCC 10 second rule

- 0: Test mode OFF (default)
- 1: Test mode ON

enumerator **RADAR_DC_FREEZE**

Enable DC Freeze

- 0: DC Freeze Disabled
- 1: DC Freeze Enabled (default)

enumerator **RADAR_PULSE_SHAPE**

enumerator **RADAR_DRIFT_COMPENSATION**

All information provided in this document is subject to legal disclaimers. © 2026 NXP B.V. All rights reserved.

bit[15:0] : Alpha in Q1.15 format (Default: 0x00)

Note:

Value 0 denotes that Drift compensation is disabled

enumerator **RADAR_CFG**

1 byte - Number of Radar Session Configs(N) (0-2)

- RADAR_CFG[0]:
  - Octet[0]: RADAR_MODE
  - Octet[1]: PREAMBLE_CODE_INDEX
  - Octet[2]: TX_ANT_ID
  - Octet[3-4]: Start Index for RX1
  - Octet[5-6]: Start Index for RX2
  - Octet[7-8]: Start Index for RX3
  - Octet[9]: ANT CFG for RX1
  - Octet[10]: ANT CFG for RX2
  - Octet[11]: ANT CFG for RX3
  - Octet[12]: Gain index for RX1
  - Octet[13]: Gain index for RX2
  - Octet[14]: Gain Index for RX3
  - Octet[15]:
    - 0: Enable AGC
    - 1: Force gain index values
    - 2: Force AGC to be re-executed
    - Default: 0x0
  - Octet[16]: TX Pulse Shape
    - Default: 0x00

enumerator **INITIAL_SYNC_RX_WINDOW_CONFIG**

This parameter allows user to set the RX window for the initial UWB sync, with a dynamic increase in the window size after certain retry counts.

- Octet [3:0] - RX_WINDOW_DURATION
  - This parameter is used to configure the UWB listening window size duration in the unit of RSTU
  - (Default: 0 (This implies the feature is not applicable))

Non-Zero value range:

1200 <= RX_WINDOW_DURATION <= (25% of RANGING_DURATION)

If the value is set ‘0’, then below fields are not applicable.

- Octet [5:4] – RX_WINDOW_STEP_SIZE
  - Rx window step size duration added at each RETRY_RX_COUNT in the unit of RSTU.
  - (Default : 0)
- Octet [6] – RETRY_RX_COUNT
  - Number of ranging timeout before increasing the RX_WINDOW_DURATION
  - (Default : 1)
  - Min RETRY_RX_COUNT : 1

For first ranging block,

RX window = INITIATION_TIME +/- RX_WINDOW_DURATION.

For subsequent blocks,

RX window = RANGING_DURATION +/- (RX_WINDOW_DURATION + RX_WINDOW_STEP)

Note:

- *This parameter is only applicable if UWB_INITIATION_TIME is set to a Non-Zero value.*
- *For optimal power saving, all the sessions which are using UWB_INITIATION_TIME should set the non-zero value for RX_WINDOW_DURATION*

Note:

All information provided in this document is subject to legal disclaimers.

© 2026 NXP B.V. All rights reserved.

- *This parameter is only applicable for controlee device type.*
- *This param is applicable for only Aliro session type(0xA2).*
- *The RX_WINDOW opening will be done once per ranging block even if 2 ranging rounds are configured per block in ALIRO_MAC_MODE.*
- *This param is applicable in FiRa session on Controlee device if ALIRO_CONTROLEE_EXTENSIONS = 0x01 (Enable)*
- *In case of Aliro Session Type, the RX Window opening will be done once per Ranging Block ,*
- *even if 2 Ranging rounds are configured per Block in ALIRO_MAC_MODE*

Note:

`SESSION_STATUS_NTF` *will throw an error with the following reason codes:*

- `ERROR_INVALID_INITIAL_SYNC_RX_WINDOW_CONFIG_CONFIGURED` *This occurs if: RX_WINDOW_DURATION is configured > 25% of RANGING_DURATION*
- `ERROR_INITIAL_SYNC_RX_WINDOW_INITIATION_TIME_NOT_CONFIGURED` *This occurs if: RX_WINDOW_DURATION is configured without configuring UWB_INITIATION_TIME*

enum **UWB_DBG_CFG_t**

Debug Configuration parameters supported in UWB API layer.

Values:

enumerator **kUWB_DBG_CFG_DATA_LOGGER_NTF**

enumerator **kUWB_DBG_CFG_TEST_CONTENTION_RANGING_FEATURE**

enumerator **kUWB_DBG_CFG_CIR_CAPTURE_WINDOW**

enumerator **kUWB_DBG_CFG_RANGING_TIMESTAMP_NTF**

enumerator **END_OF_SUPPORTED_EXT_DEBUG_CONFIGS**

End of Ext Debug Configs

enum **UWB_SR2XX_DBG_DATALOGTRG_CFG_t**

Values:

enumerator **kUWB_SR2XX_DBG_DATALOGTRG_CFG_RX_STATE_INIT**

enumerator **kUWB_SR2XX_DBG_DATALOGTRG_CFG_RX_STATE_ACQ_SEARCH**

enumerator **kUWB_SR2XX_DBG_DATALOGTRG_CFG_RX_STATE_ACQ_FOUND**

enumerator **kUWB_SR2XX_DBG_DATALOGTRG_CFG_RX_STATE_SFD_SEARCH**

enumerator **kUWB_SR2XX_DBG_DATALOGTRG_CFG_RX_STATE_SFD_FOUND**

enumerator **kUWB_SR2XX_DBG_DATALOGTRG_CFG_RX_STATE_SIG_REJECTED**

enumerator **kUWB_SR2XX_DBG_DATALOGTRG_CFG_RX_STATE_STS1_GUARD1**

enumerator **kUWB_SR2XX_DBG_DATALOGTRG_CFG_RX_STATE_STS1**

enumerator **kUWB_SR2XX_DBG_DATALOGTRG_CFG_RX_STATE_STS1_INTERNAL_GUARD**

enumerator **kUWB_SR2XX_DBG_DATALOGTRG_CFG_RX_STATE_STS1_END**

enumerator **kUWB_SR2XX_DBG_DATALOGTRG_CFG_RX_STATE_STS1_GUARD2**

enumerator **kUWB_SR2XX_DBG_DATALOGTRG_CFG_RX_STATE_PHR**

enumerator **kUWB_SR2XX_DBG_DATALOGTRG_CFG_RX_STATE_PHR_END**

enumerator **kUWB_SR2XX_DBG_DATALOGTRG_CFG_RX_STATE_PSDU**

enumerator **kUWB_SR2XX_DBG_DATALOGTRG_CFG_RX_STATE_PSDU_END**

All information provided in this document is subject to legal disclaimers. © 2026 NXP B.V. All rights reserved.

enumerator **kUWB_SR2XX_DBG_DATALOGTRG_CFG_RX_STATE_STS2_GUARD1**

enumerator **kUWB_SR2XX_DBG_DATALOGTRG_CFG_RX_STATE_STS2**

enumerator **kUWB_SR2XX_DBG_DATALOGTRG_CFG_RX_STATE_STS2_INTERNAL_GUARD**

enumerator **kUWB_SR2XX_DBG_DATALOGTRG_CFG_RX_STATE_STS2_END**

enumerator **kUWB_SR2XX_DBG_DATALOGTRG_CFG_RX_STATE_STS2_GUARD2**

enumerator **kUWB_SR2XX_DBG_DATALOGTRG_CFG_RX_STATE_EOF**

enumerator **kUWB_SR2XX_DBG_DATALOGTRG_CFG_RX_STATE_OFF**

enumerator **kUWB_SR2XX_DBG_DATALOGTRG_CFG_RX_STATE_HW_NOISE_EST**

enumerator **kUWB_SR2XX_DBG_DATALOGTRG_CFG_RX_STATE_BINNING_ONLY**

enumerator **kUWB_SR2XX_DBG_DATALOGTRG_CFG_RX_STATE_ADC_REC**

enumerator **kUWB_SR2XX_DBG_DATALOGTRG_CFG_RX_STATE_SIDELOBE_SUPPR**

enumerator **kUWB_SR2XX_DBG_DATALOGTRG_CFG_RX_STATE_FIRSTPATH_SEARCH**

enumerator **kUWB_SR2XX_DBG_DATALOGTRG_CFG_RX_STATE_FAIL**

enum **UWB_AppParams_type_t**

Set/Get App Configuration parameters type supported in UWB API layer.
Values:

enumerator **kUWB_APPPARAMS_Type_Unknown**

We don't know the type

enumerator **kUWB_APPPARAMS_Type_u32**

It's a 32 bit value

enumerator **kUWB_APPPARAMS_Type_au8**

It's an array of 8 bit values

enum **UWB_DebugParams_type_t**

Set/Get Debug Configuration parameters type supported in UWB API layer.
Values:

enumerator **kUWB_DEBUGPARAMS_Type_u8**

It's a 8 bit value

enumerator **kUWB_DEBUGPARAMS_Type_u16**

It's a 16 bit value

enumerator **kUWB_DEBUGPARAMS_Type_u32**

It's a 32 bit value

enumerator **kUWB_DEBUGPARAMS_Type_au8**

It's an array of 8 bit values

enum **eDeviceConfig**

Device Configuration parameters supported in UWB API layer.
Values:

enumerator **LOW_POWER_MODE**

0:DISABLE, 1:ENABLE

All information provided in this document is subject to legal disclaimers.
© 2026 NXP B.V. All rights reserved.

enumerator **WIFI_COEX_FEATURE**

This configuration is used to configure the wifi co-ex feature.

- Octet[0]: Enable/Disable wifi co-ex feature
  - 0x00 : To Disable(default)
  - b3-b0: Enable/Disable functionality CoEx:
    - 0x2: Enable CoEx Interface with Debug Verbose only
    - 0x3: Enable CoEx Interface with Warnings Verbose only
    - 0x4: Enable CoEx Interface with both Debug and Warning Verbose
  - b4: CoEx Interface (GPIO/UART/One Wire) selection:
    - 0x0: GPIO Interface
    - Rest of the values are Reserved
- Octet[1]: Number of channels N Shall be more than or equal 1 (0 will be rejected by UWBS) N*4 octets to follow
- Octet[2]: Channel ID
- Octet[3]: MIN_GUARD_DURATION: Minimum gap in milliseconds between the end of previous active grant duration and next medium access request
- Octet[4]: MAX_GRANT_DURATION: Maximum duration for which the UWB can request for medium access
- Octet[5]: ADVANCED_GRANT_DURATION: Minimum preparation time in milliseconds required by the WLAN subsystem to relinquish the CoEx Medium for UWB subsystem usage. It shall be set to at least 6ms for GPIO based CoEx Interface. Ranging interval value in ms should be greater or equal to following expression: (ADVANCED GRANT DURATION + ROUND DURATION + MIN_GUARD_DURATION)

enumerator **AOA_CALIBRATION_CTRL**

AOA calibration(4*8*4)

- 4 bytes value description:
  - bit31-AoA phase flip enable (default = 0),
  - bits [30:16]-Antenna spacing(default = 20mm),
  - bits [15:0]-PDoA offset 8 antenna pairs: 4 byte value repeated 8 times for each antenna pair 4 channels: 4*8 bytes repeated for 4 channels

enumerator **DPD_WAKEUP_SRC**

DPD wakeup source bit1: GPIO1, bit3: GPIO3

enumerator **DPD_ENTRY_TIMEOUT**

(UWBIOT_UWBD_SR2XXT)
DPD entry timeout in ms (default = 500ms)

enumerator **GPIO_SELECTION_FOR_DUAL_AOA**

This configuration is used to indicate whether the RX Antenna selection should take place w.r.t EF2 or GPIO14 (for Antenna switching)

- 0 : EF2 based Antenna selection (Default)
- 1 : GPIO14 based Antenna selection

enumerator **TX_BASE_BAND_CONFIG**

0:DISABLE, 1:ENABLE

enumerator **DDFS_TONE_CONFIG**

DDFS tone config (4*8 bytes repeated for 4 channels)

- 18 bytes value description:
  - Octet[0]: channel number
  - Octet[1]: Tx antenna selection. Possible values are 1 or 2.
  - Octet[5:2]: Content of register TX_DDFS_TONE_0
  - Octet[9:6]: Content of register TX_DDFS_TONE_1
  - Octet[13:10]: Duration of the spur, in 124.8 MHz resolution (~ 8 ns)

All information provided in this document is subject to legal disclaimers.
© 2026 NXP B.V. All rights reserved.

  - Octet[14]: Content of register GAINVAL_SET
  - Octet[15]: Content of register DDFSGAINBYPASS_ENBL
  - Octet[17:16]: Periodicity of spur in terms of gap interval in the PER command.
- 4 Blocks: 18 Octets repeated for each block
  - Octets[17:0] correspond to Block1
  - Octets[35:18] correspond to Block2
  - Octets[53:36] correspond to Block3
  - Octets[71:54] correspond to Block4

enumerator **TX_PULSE_SHAPE_CONFIG**

Preamble pulse shape setting
- Octet[0]: Preamble pulse shape id
- Octet[1]: Payload Tx pulse shape id
- Octet[2]: STS Tx pulse shape ID
- Octet[3]: DAC Stage Config

Values:
- Octet[0-2] = [2(default), 30, 34, 36 and 37]
- Octet[3] = Value is defined as below
  - bit0: To set the DAC gain
    - 0: Unchanged(UWBS Keeps previous assigned value) .
    - 1: UWBS set to 0x24
  - bit1: To set LPF(Tx DAC C)
    - 0: UWBS shall set to 0
    - 1: UWBS shall set to 0x5F
  - bit7-bit2: RFU

enumerator **HOST_MAX_UCI_PAYLOAD_LENGTH**

Parameter is used to set host capability of handling max UCI payload. FW shall use this parameter to send the UCI responses/notification to host. FW shall use PBF bit if UCI payload goes more than HOST_MAX_UCI_PAYLOAD_LENGTH size. 255<= HOST_MAX_UCI_PAYLOAD_LENGTH<= UWBS_MAX_UCI_PAYLOAD_LENGTH(capability parameter) (Default = 255)

enumerator **NXP_EXTENDED_NTF_CONFIG**

- 0x00 = FIRA generic Response/Notification (Default)
- 0x01 = Vendor extended Response/Notification

enumerator **INITIAL_RX_ON_OFFSET_ABS**

Negative offset when Rx should be enabled to receive the first message of a ranging round on Controlee compared to expected reception time. Default = 100 us

enumerator **INITIAL_RX_ON_OFFSET_REL**

Negative offset when Rx should be enabled to receive the first message of a ranging round on Controlee compared to expected reception time. Default = 100 ppm

enumerator **CLK_CONFIG_CTRL**

- Octet[0] : INPUT_CLK_CTRL
  - bit[0]: 32kHz clock source
    - 0: Use on board crystal (default)
    - 1: Use external 32k clock
  - bit[2:1]: External 32k clock waveform
    - 0b00: Sinus (default)
    - 0b01: 1.8V square

All information provided in this document is subject to legal disclaimers.
© 2026 NXP B.V. All rights reserved.

  - 0b10: 1.2V square
- bit[3]: RF clock source
  - 0: Use on board crystal (default)
  - 1: Use external clock
- bit[5:4]: RF clock frequency
  - 0b00: Use 38.4MHz
  - 0b01: Use 26MHz
  - 0b10: Use 19.2MHz
- bit[6]: External clock waveform
  - 0: Sinus (default)
  - 1: Square
- bit[7]: RFU

- Octet[2:1] : SLOW_CLK_WAITING_TIME Maximum waiting time in us until 32.768 kHz clock is present. Default: 1000us.
- Octet[4:3] : RF_CLK_WAITING_TIME Maximum waiting time in us until RF clock is present. Default: 1000us.

enumerator **DDFS_CONFIG_PER_PULSE_SHAPE**

Allow to override the TX_BASEBAND_CONFIG parameter for specific pulse shapes.

- Octet[0]: DDFS config for FiRa PS
- Octet[1]: DDFS config for CCC0 PS
- Octet[2]: DDFS config for CCC1 PS
- Octet[3]: DDFS config for CCC2 PS (Default: 0x00 00 00 00 for Octet[0:3])
- Content of each byte:
  - bit[0]: override DDFS config
    - 0: No override
    - 1: TX_BASEBAND_CONFIG is overridden for this pulse shape (DDFS config will depends on bit[4:1]
  - bit[1]: DDFS config for SP0 frame
    - 0: Disabled
    - 1: Enabled
  - bit[2]: DDFS config for SP1 frame
    - 0: Disabled
    - 1: Enabled
  - bit[3]: RFU
  - bit[4]: DDFS config for SP3 frame
    - 0: Disabled
    - 1: Enabled
  - bit[7:5] RFU

enumerator **AOA_CALIB_CTRL_AVG_PDOA**

PDoA average of vertical sweeps for all the horizontal angles and for all the IDs from 0xE4 0x40 to E4 0x43, in that order. All are signed PDoA values in Q9.7 (2 bytes, little-endian) format.

- Octet[1:0]: Avg. of PDoAs in octet[21:0] of 0xE4 0x40
- Octet[3:2]: Avg. of PDoAs in octet[43:22] of 0xE4 0x40
- Octet[5:4]: Avg. of PDoAs in octet[65:44] of 0xE4 0x40
- Octet[7:6]: Avg. of PDoAs in octet[87:66] of 0xE4 0x40
- Octet[9:8]: Avg. of PDoAs in octet[109:88] of 0xE4 0x40
- Octet[11:10]: Avg. of PDoAs in octet[131:110] of 0xE4 0x40
- Octet[13:12]: Avg. of PDoAs in octet[153:132] of 0xE4 0x40
- Octet[15:14]: Avg. of PDoAs in octet[175:154] of 0xE4 0x40
- Octet[17:16]: Avg. of PDoAs in octet[197:176] of 0xE4 0x40
- Octet[19:18]: Avg. of PDoAs in octet[219:198] of 0xE4 0x40
- Octet[21:20]: Avg. of PDoAs in octet[241:220] of 0xE4 0x40 Similarly,

All information provided in this document is subject to legal disclaimers. © 2026 NXP B.V. All rights reserved.

- Octet[43:22]: for 0xE4 0x41 Octet[65:44]: for 0xE4 0x42
- Octet[175:154]: for 0xE4 0x43

enumerator **AOA_CALIB_CTRL_THRESHOLD_PDOA**

enumerator **PDOA_CALIB_TABLE_DEFINE**

PdoA Calibration Table Definition :

- Octet[0] - Calibration Step Size : The calibration table step size in degrees which indicates the step size between two consecutive points.
  - Allowed Range : 10° to 15° (default = 12°)
  - Example : For maximum span of 60°, with step size 12° then the calibration table would look like[-60°, -48°, -36° , …, 0°, …, 36°, 48°, 60°].
- Octet [1] - Number of Steps: The number of calibration steps needs to be an odd number.
  - Allowed Range : 3<=M<=21(to include 0°)(default = 11)
  - Example : With number of step size as 10° and number of steps as 13, the achieved calibration span is -60, -50, -40, …, -10, 0, 10, …, 40, 50, 60

Note:

- *Total Calibration Span = ((Number of Steps - 1) * Step Size). Example :*
  - *If steps are 11 and step size is 12° then total span is 120°(-60° to 60° including 0°).*
  - *If steps are 13 and step size is 15° then total span is 180°(-90° to 90° including 0°).*

enumerator **ANTENNA_RX_IDX_DEFINE**

To define/create antenna identifier for RX

- Octet[0] : Number of Entries. (N). This must be equal to “MAX_N”.

Array of entries:

- Octet[X + 0] : RX Antennae ID.
  - Index of the Antennae.
  - Value 0 shall be Invalid.
  - Value shall from 1 to MAX_N
- Octet[X + 2, X + 1]: GPIO Filter Mask
  - This mask defines which GPIOs shall be changed during the state transition, if a GPIO bit is set to 0 the GPIO shall not change its state.
  - It’s a 2 byte value in Little Endian format.
- Octet[X + 4, X + 3]: GPIO State / Value
  - This mask defines the GPIO state, if the corresponding GPIO bit is 0 in the GPIO Filter mask, the state shall be ignored and not changed It’s a 2 byte value in Little Endian format.

enumerator **ANTENNA_TX_IDX_DEFINE**

To define/create antenna identifier for TX

- Octet[0] : N Entries.

Array of entries:

- In case, same Antenna is used for Tx/Rx Use the same ID.
- In case few GPIOs have to be switched for toggling,
- Octet[1] and Octet[2] would be different when TX/RX antennae are defined.
- Octet[0] : Antennae ID.
  - Index of the Antennae.
  - Value 0 shall be Invalid.
  - Value shall from 1 to N.
- Octet[1]: GPIO Filter Mask
  - This mask defines which GPIOs shall be changed during the state transition, if a GPIO bit is set to 0 the GPIO shall not change its state.
- Octet[2]: GPIO State / Value
  - This mask defines the GPIO state, if the corresponding GPIO bit is 0 in the GPIO Filter mask, the state shall be ignored and not changed

All information provided in this document is subject to legal disclaimers.
© 2026 NXP B.V. All rights reserved.

- Octet[6:3] Group Delay

Max number of entries per product variant for max value of N is same as ANTENNA_RX_IDX_DEFINE For Calibration of TX Power, use TX_POWER from Calibration Parameters

enumerator **ANTENNAE_RX_PAIR_DEFINE**

To define/create antenna identifier for RX Pair

- Octet[0] : N Entries.

Array of entries:

- This may be an H or V Combination. Repeat of [
- Octet[0] : Antennae PAIR ID
  - This IDx is used along with ANTENNAE_CONFIGURATION(Session Config) and for reporting.
  - ID 0 shall not be used.
  - For non-scanning mode/for backward compatibility, Default ID for Horizontal has to be 1.
  - And ID for Vertical has to be 2.
  - Use Odd Values for Horizonal Pair And Even Values for Vertical Paris.
  - So, FW can decide to move to lower or higher Pair ID.
  - FW can also use IDs X, X+1 to identify which ID is making a group for 3D Configuration.
- Octet[1] : RX1 Antennae ID as defined by ANTENNAE_RX_IDX_DEFINE
- Octet[2] : RX2 Antennae ID as defined by ANTENNAE_RX_IDX_DEFINE.
- Octet[4:3]: PDOA Zero Offset
  - 2 bytes PDOA1_OFFSET, PDOA2_OFFSET
- Octet[6:5] : Relative Angle of View / Field of View.
  - Assuming we have 4 Antennae pairs symmetrically placed on a UWB System.
  - For a given selected Antennae pair, the peer object may be at 0°, but reported AoA has to be either 0°, 90°, 180° or 270° depending on this relative Angle of View.

When this is set to 0, then Host has to use RX Antennae info and derive the relative angle. When this is set to 0, the FW has no IDEA which antennae group to switch to in case of antennae moves out of some configuration.

Max number of entries per product variant for max value of N is same as ANTENNA_RX_IDX_DEFINE

enumerator **END_OF_SUPPORTED_DEVICE_CONFIGS**

End of device Configs

enum **kUWBAntCfgRxMode_t**

Antenna Configuration and slection mode

Values:

enumerator **kUWBAntCfgRxMode_ToA_Mode**

enumerator **kUWBAntCfgRxMode_AoA_Mode**

enumerator **kUWBAntCfgRxMode_Radar_Mode**

For Rx RADAR

enumerator **kUWBAntCfgRxMode_ToA_Rfm_Mode**

For Rx TOA RFM

enumerator **kUWBAntCfgRxMode_AoA_Rfm_Mode**

For Rx AOA RFM

enumerator **kUWBAntCfgRxMode_CSA_ToA_Mode**

ToA mode for CSA

enumerator **kUWBAntCfgRxMode_CSA_AoA_Mode**

AoA mode for CSA

All information provided in this document is subject to legal disclaimers. © 2026 NXP B.V. All rights reserved.

enum **eCalibStates_t**

Enumeration for device calibration states.

This enumeration defines the possible calibration states of a UWB device. It is used to indicate whether the device has been properly calibrated or not.

See Also:

CHIP_CALIBRATION_STATE

Values:

enumerator **eCalibState_DEVICE_NOT_CALIBRATED**

If DO_CHIP_CALIBRATION_CMD was not processed by UWBS.

enumerator **eCalibState_DEVICE_CALIBRATED**

If DO_CHIP_CALIBRATION_CMD was processed by UWBS.

enum **eCalibParam**

Calibrations Configuration parameters supported in UWB API layer.

Values:

enumerator **CHIP_CALIBRATION**

Chip calibration Do_Chip_Calibration_Cmd triggers a write operation on SRnxx's Flash memory

Value determined during calibration process and this parameter is channel dependent. Channel number to be provide in calibration commands to set this parameter.

- bits[0-14] : Calibration code
- bit[15] : Calibration status as below
  - 0: Calibration failure
  - 1: Calibration success

Return the calibration outcome per channel. This parameter is channel dependent.

- bit[8:0] : Calibration code
- bit[13:9] : RFU
- bit[15:14] : Calibration status as below
  - 0b10 : Calibration is valid
  - Other: Calibration is invalid

enumerator **RF_CLK_ACCURACY_CALIB**

Channel independent and remains same for each channel and channel number in calibration command shall be ignored by UWBS.

Rest all variants (Except [0x2A, 0x03])

- Octet [0] : Number of registers (must be 0x03)
- Octet [2-1]: 38.4 MHz XTAL CAP1 (Default: 0x0F)
- Octet [4-3]: 38.4 MHz XTAL CAP2 (Default: 0x0F)
- Octet [6-5]: 38.4 MHz XTAL GM CURRENT CONTROL (Default: 0x21)

Values : [0x00-0xFF] for 38.4 MHz XTAL CAP Values : [0x00-0x3F] for 38.4 MHz XTAL GM

Defines the capacitors values applied to 38.4MHz XTAL. Channel independent and remains same for each channel.

- Octet [0] : Number of registers (must be 0x03)
- Octet [2-1]: 38.4 MHz XTAL CAP1
- Octet [4-3]: 38.4 MHz XTAL CAP2
- Octet [6-5]: 38.4 MHz XTAL GM CURRENT CTRL

Values : [0x00-0xFF] for 38.4 MHz XTAL CAP Values : [0x00-0x3F] for 38.4 MHz XTAL GM

Note:

HELIOS-1 For Board Variant: [0x2A, 0x03]

- *Octet [0] : Number of registers (must be 0x03)*
- *Octet [2-1]: 38.4 MHz XTAL CAP1 (Default: 0x13)*

All information provided in this document is subject to legal disclaimers. © 2026 NXP B.V. All rights reserved.

- *Octet [4-3]: 38.4 MHz XTAL CAP2 (Default: 0x13)*
- *Octet [6-5]: 38.4 MHz XTAL GM CURRENT CONTROL (Default: 0x30)*

Note:

Stored in SR2XX's Flash

enumerator **RX_ANT_DELAY_CALIB**

Delay Calibration for each RX Antenna. This parameter is channel dependent. Total of 1 + (N*3) Octets.

- Octet[0]: Number of Entries (Must be greater than 0)

iteration - Start

- Octet[1] : RX Antennas IDs as defined by ANTENNA_RX_IDX_DEFINE
- Octet[3-2] : Octets each: RX Delay (unsigned Q14.2)

iteration - End

(Default: 0x0000 for all RX antenna IDs)

Note:

**** Stored in SR2XX's Flash ****

Note:

: Delay calib is used and effective only for the Rx antenna, that is part of Rx Antenna Pair that is used for receiving the STS Segment. As of now, only vertical pair is used and it can change in future. Function revisions may choose to use multiple antennaes for multiple STS Segments.

enumerator **PDOA_OFFSET_CALIB**

PDOA Offset Calibration - This parameter is channel dependent. Total of 1 + N*(1+2) Octets.

- Octet[0]: Number of Entries (Must be greater than 0) (Depends on available RAM)

Next octets would be read as set of 3 octets for number of entries reported in octet[0] above.

iteration - Start

- Octet[1] : RX Antennas PAIR IDs as defind by ANTENNAS_RX_PAIR_DEFINE
- Octet[3-2] : PDoA Offsets. (signed value in Q9.7 format)

iteration - End

(Default: 0x0000 for all RX)

Note:

Channel_ID 5 setting is applied to channels 5 and 8. Channel_ID 9 setting is applied to channels 6 and 9.

enumerator **TX_POWER_PER_ANTENNA**

TX POWER - This parameter is channel dependent. Total of 1 + (N*5) Octets.

- Octet[0] : Number of Entries (Must be greater than 0).

Next octets would be read as set of 3 octets for number of entries reported in octet[0] above.

iteration - Start

- Octet[1] : TX Antenna IDs as defind by ANTENNAS_TX_IDX_DEFINE
- Octet[3-2] : TX_POWER_DELTA_PEAK
  - If TX_ADAPTIVE_PAYLOAD_POWER is enabled, this value represents the delta peak ie : PEAK_ID - RMS_ID. This value is a delta and comes from calibration process, It ranges from 0 (max power) to 127 (min power) with steps of 0.25.
  - If TX_ADAPTIVE_PAYLOAD_POWER is not enabled, this value has no specific use.
- Octet[5-4] : TX_POWER_ID_RMS
  - If TX_ADAPTIVE_PAYLOAD_POWER is enabled, this value come from RMS calibration as RMS_ID
  - If TX_ADAPTIVE_PAYLOAD_POWER is not enabled, this value serves as a gain index for TX power control. It ranges from 0 (max power) to 127 (min power) with steps of 0.25.

iteration - End (Default value for all: 0x00)

Note:

: TEST_LOOPBACK_CMD is rejected if TX_POWER_ID_RMS is below than 80

enumerator **AOA_PHASEFLIP_ANTSPACING**

Antenna phase flip and spacing. This parameter is channel dependent. Total of 1+(N*3) Octets.

All information provided in this document is subject to legal disclaimers. © 2026 NXP B.V. All rights reserved.

- Octet[0]: Number of Entries (Must be greater than 0)

Next octets would be read as set of 3 octets for number of entries reported in octet[0] above.

iteration - Start

- Octet[1] : RX Antenna Pair ID
- Octet[3-2] :
  - Bit 15 Phase flip (1 bit)
  - Bit 14:0 Antenna spacing (15 bits in Q6.9 format)

iteration - End (Default: 0x0028 for all RX antenna pair IDs)

enumerator **SYSTEM_CLK_ACCURACY_CALIB**

Channel independent (same value is applied for each channel):

- Octet[0] : 32 kHz XTAL CAP CRTL
  - Default: 0x03(HELIOS-2)
  - Default: 0x02(YUKON)
  - Value Range: [0x00 : 0x1F]
- Octet[1] : 32 kHz XTAL GM CURRENT CTRL
  - Default: 0x00
  - Value Range: [0x00 : 0x07]

enumerator **PLATFORM_ID**

16-bytes string defining the platform name - Channel independent

Note:

**** Stored in SR2XX's and Flash ****

enumerator **CONFIG_VERSION**

Version number of the secure parameters - Channel independent

Note:

**** Stored in SR2XX's and Flash ****

enumerator **DBG_SE_DUMP**

Channel independent. 1 byte config to enable/disable the SE_COMM_DATA_NTF.

- 0x00: Disable
- 0x01: Enable

Note:

: Allowed in all lifecycles.

enumerator **MANUAL_TX_POW_CTRL**

MANUAL_TX_POW_CTRL - Refer Tx Power Control parameter for details For Helios 1

- Octet [0]: PA_GAIN (default: 0x00)
- Octet [1]: PA_DRIVE_GAIN (default: 0x00)
- Octet [2]: DIG_GAIN (default: 0x00)
- Octet [3]: TX_DAC_GAIN (default: 0x38)
- Octet [4]: RFU

For Helios 2

- Octet[0] : TX_PA_GAIN
- Octet[1] : TX_PPA_GAIN
- Octet[2] : TX_DIG_GAIN
- Octet[3] : DAC_GAIN
- Octet[4] : TX_MIX_GAIN

enumerator **AOA_ANTENNAS_PDOA_CALIB**

PDOA Calibration tables - Added for 360 FoV. This is of 1 + N*(1 + S*S*2) Octets.

- Octet[0] : No of entries, it shall be set to 1 for Set Calibration Cmd. (Refer to a) of Note for this limitation.) Next octets would be read as set of (1 + S*S*2) octets for number of entries reported in octet[0] above.

All information provided in this document is subject to legal disclaimers.
© 2026 NXP B.V. All rights reserved.

iteration - Start Octet[1] : RX Pair Antennas ID as defined earlier by ANTENNAS_RX_PAIR_DEFINE Octet[2-M] : PDoA calibration table for specified RX Pair Antennas ID.

iteration - End

For details on the format refer AOA_CALIB_CTRL_RX_ANT_PAIR_1_CH5. The older entries would get overwritten, if more than supported data is uploaded to the UWBS. Refer specification for max supported calibration tables per channel per antennas pair. So, for SRnxx, 2 Channels X 2 Pairs X (S*S*2) is pre-allocated.

S is number of steps is Octet[1] of PDOA_CALIB_TABLE_DEFINE. Allowed Range: 3<= S<= 17 (to include 0o(default : 11) Allowed Range: 3<= S <= 11 (to include 0o(default : 11)

N = 1 Always

b) The Get Calibration Cmd response for this ID returns the total number of “RX Pair Antennas IDs” for which PDOA calibration table is set in Octet[0].

Note:

: a) In order to support Hosts that can not send more than 255 + 4 bytes, this UCI interface can only be used to set PDoA calibration table for one RX Pair Antennas ID at a time.

enumerator **TX_ANT_DELAY_CALIB**

Delay Calibration of TX Part of Antenna. Thsi is of 1 + (N*3) Octets.

- Octet[0]: Number of Entries (Must be greater than 0)

iteration - Start

- Octet[1] : TX Antennas IDs as defined by ANTENNA_TX_IDX_DEFINE
- Octets[3-2] : TX Delay offset (signed Q14.2)

iteration - End (Repeat)

enumerator **PDOA_MANUFACT_ZERO_OFFSET_CALIB**

Zero offset Manufacturing PDOA Calibration. This is of 1 + (N*1) + (N*2) Octets

- Octet[0]: Number of Entries (Must be greater than 0) (Depends on available RAM)

iteration - Start

- Octet[1] : RX Antennas PAIR IDs as defind by ANTENNAS_RX_PAIR_DEFINE
- Octet[3-2] : Manufacture PDOA1 Offset

iteration - End (Repeat)

Note:

Supersedes Antenna Spacing enable in “AOA_CALIBRATION_CTRL”. This value (PDOA_MANUFACT_ZERO_OFFSET_CALIB) will have higher precedence as compared to this PDOA_OFFSET_CALIB.

enumerator **AOA_THRESHOLD_PDOA**

AoA Threshold PDOA. This is of 1 + (N*1) + (N*2) Octets.

- Octet[0]: Number of Entries (Must be greater than 0)

iteration - Start

- Octet[1] : RX Antennas PAIR IDs as defind by ANTENNAS_RX_PAIR_DEFINE
- Octets[3-2] : Threshold Values

iteration - End (Repeat)

Note:

Supersedes Threshold PDoA in “AOA_CALIB_CTRL_THRESHOLD_PDOA”

enumerator **TX_TEMPERATURE_COMP_PER_ANTENNA**

TX temperature compensation per antenna. This parameter is channel dependent. This is of 1 + N*(1+8) Octets

- Octet[0]: Number of Entries (Must be greater than 0)

iteration - Start

- Octet[1] : TX Antenna IDs as defind by ANTENNAS_TX_IDX_DEFINE
- Octet[2] : RANGE1_TX_POWER_UPPER_BOUND
- Octet[3] : RANGE1_TX_POWER_GAIN_OFFSET
- Octet[4] : RANGE2_TX_POWER_UPPER_BOUND

All information provided in this document is subject to legal disclaimers. © 2026 NXP B.V. All rights reserved.

- Octet[5] : RANGE2_TX_POWER_GAIN_OFFSET
- Octet[6] : RANGE3_TX_POWER_UPPER_BOUND
- Octet[7] : RANGE3_TX_POWER_GAIN_OFFSET
- Octet[8] : RANGE4_TX_POWER_UPPER_BOUND
- Octet[9] : RANGE4_TX_POWER_GAIN_OFFSET

iteration - End (Repeat)

Associated offset from TX_POWER_GAIN_OFFSET will be applied if measured temperature is below the associated TX_POWER_UPPER_BOUND.

Note:

TX_POWER_UPPER_BOUND:Upper bound for the temperature value. It is a signed number to allow the configuration of negative temperature Default value: 0x80(Disable) TX_POWER_GAIN_OFFSET: Offset to be applied on the Tx power gain table for a specific temperature range. it's a signed number. Default value : 0x00

enumerator **RSSI_CALIB_CONSTANT_PER_ANTENNA**

RSSI Offset per RX Antenna for High Power - Channel dependent. This is of 1 + (N*1) Octets.

- Octet[0]: Number of Entries (Must be greater than 0)

iteration - Start

- Octet[1] : RX Antenna IDs as defind by ANTENNAS_RX_IDX_DEFINE
- Octet[2] : RSSI Calib (value in dB)

iteration - End (Repeat)

Default value for all: 0x00

It is an unsigned value which will be substracted to the RSSI value computed by the FW

enumerator **TRA2_LOFT_CALIB**

Calibration parameter for reducing the TX Loft

- Octet[0] : TX_DAC_OFFSET_I_SET (2's complement coded on 8 bits)
- Octet[1] : TX_DAC_OFFSET_Q_SET (2's complement coded on 8 bits)
- Octet[2] : TX_DC_OFFSET_RE (2's complement coded on 8 bits)
- Octet[3] : TX_DC_OFFSET_IM (2's complement coded on 8 bits)

enumerator **TRA1_LOFT_CALIB**

enumerator **CHIP_CALIBRATION_STATE**

On Get calib, Indicates if device is calibrated via DO_CHIP_CALIBRATION_CMD.

See Also:

eCalibStates_t

Possible Values:

- 0x00 = DEVICE_NOT_CALIBRATED
- 0x01 = DEVICE_CALIBRATED

Note:

This is a read-only parameter

enumerator **AOA_ANTENNAS_PDOA_CALIB_EXTENDED_SUPPORT**

This parameter is used to set the PDoA Calibration Table for 90 Fov

enum **eOtpCalibParam**

Calibrations Configuration parameters supported in UWB API layer.

Values:

enumerator **OTP_CALIB_CHIP_CALIBRATION**

CHIP CALIBRATIONL Calibration, channel dependent

All information provided in this document is subject to legal disclaimers.
© 2026 NXP B.V. All rights reserved.

enumerator **OTP_CALIB_TX_POWER**

TX POWER Calibration, channel and antenna dependent

enumerator **OTP_CALIB_RF_XTAL_CAP**

38.4MHz XTAL, channel and antenna Independent

enumerator **OTP_CALIB_RSSI_CALIB_CONST1**

RSSI Calibration, channel and antenna dependent

enumerator **OTP_CALIB_RSSI_CALIB_CONST2**

RSSI Calibration, channel and antenna dependent

enumerator **OTP_CALIB_RFU1**

rfu

enumerator **OTP_CALIB_MANUAL_TX_POW_CTRL**

power control parameters

enumerator **OTP_CALIB_RFU2**

rfu

enumerator **OTP_CALIB_PAPPPA_CALIB_CTRL**

PA_PPA Calibration control

enumerator **OTP_CALIB_TX_TEMPARATURE_COMP**

Tx Temperature Compensation

enumerator **OTP_CALIB_RFU3**

rfu

enumerator **OTP_CALIB_DELAY_CALIB**

Delay calibration value to adjust the distance measurement from ranging

enumerator **OTP_PDOA_MFG_ZERO_OFFSET_CALIB**

PDOA mfg zero offset calibration

enumerator **OTP_AOA_ANT_MULTIPOINT_CALIB**

Point calibration entries

enum **eCalibTagOption**

Calibration integrity protection tag options.
Values:

enumerator **DEVICE_SPECIFIC**

Device Specific tag option

enumerator **MODEL_SPECIFIC**

Model Specific tag option

enum **eCalibState**

Calibration Parameter States.
Values:

enumerator **DEFAULT**

Calibration parameter carries default value by UWBS

enumerator **CUSTOM_NOT_INTEGRITY_PROTECTED**

All information provided in this document is subject to legal disclaimers.
© 2026 NXP B.V. All rights reserved.

Calibration parameter is not integrity protected either by a Device specific or Model specific authentication tag. The parameter is applied in UWBS for usage.

enumerator **CUSTOM_AUTH_PENDING**

Calibration parameter integrity check is pending either by a Device specific or Model specific tag. Mainline FW: Until the tag verification finishes, the parameter will not be applied

enumerator **CUSTOM_DEVICE_SPECIFIC_TAG_AUTHENTICATED**

Calibration parameter integrity check is verified successfully by a Device specific tag.

enumerator **CUSTOM_MODEL_SPECIFIC_TAG_AUTHENTICATED**

Calibration parameter integrity check is verified successfully by a Model specific tag

enumerator **INVALID_STATE**

Calibration data is either not set or its integrity can't be verified. SR250 uses default value.

enum **ephChannel**

channel for aoa fine calibration, supported parameters
Values:

enumerator **CH_5**

channel 5

enumerator **CH_9**

channel 9

enum **eAntPair**

Antenna pair for aoa fine calibration, supported parameters
Values:

enumerator **ANT_1**

enumerator **ANT_2**

enum **UWB_CoEx_InterfaceSelection_t**

Enumeration lists out the CoEx Interface to be selected.
Values:

enumerator **kUWB_CoEx_Gpio_Interface**

UWB_WiFiCoEx_AdvGrantDuration

enumerator **kUWB_CoEx_Uart_Interface**

Set CoEx Interface as Uart Interface

enumerator **kUWB_CoEx_OneWire_Interface**

Set CoEx Interface as One Wire Interface.

enum **UWB_CoEx_NtfSelection_t**

Enumeration lists out the configurations for the WiFi CoEx feature Notifications.
Values:

enumerator **kUWB_CoEx_Disable**

Disable Wifi CoEx Feature (default)

enumerator **kUWB_CoEx_En_WoDebug_WoWarning**

Enable CoEx Interface without Debug and without Warning Verbose.

enumerator **kUWB_CoEx_En_Debug**

All information provided in this document is subject to legal disclaimers.
© 2026 NXP B.V. All rights reserved.

Enable CoEx Interface with Debug Verbose only

enumerator **kUWB_CoEx_En_Warning**

Enable CoEx Interface with Warnings Verbose only

enumerator **kUWB_CoEx_En_Debug_Warning**

Enable CoEx Interface with both Debug and Warning Verbose

enum **UWB_CoEx_ChannelCfg_t**

Enumeration lists out the configurations for the WiFi CoEx feature.
Values:

enumerator **kUWB_CoEx_CH5**

Enable Wifi CoEx on Channel 5

enumerator **kUWB_CoEx_CH6**

Enable Wifi CoEx on Channel 6

enumerator **kUWB_CoEx_CH8**

Enable Wifi CoEx on Channel 8

enumerator **kUWB_CoEx_CH9**

Enable Wifi CoEx on Channel 9

enumerator **kUWB_CoEx_AllCH**

Enable Wifi CoEx for all channels i.e ch5,ch6,ch8 and ch9

enum **eCalibParamSecure**

Secure Calibration Configuration parameters supported in UWB API layer.
Values:

enumerator **RF_CLK_ACCURACY_CALIB_SECURE**

RF_CLK_ACCURACY_CALIB - Channel independent

enumerator **TX_ANT_DELAY_CALIB_SECURE**

Delay Calibration of TX Part of Antenna

enumerator **RX_ANT_DELAY_CALIB_SECURE**

Delay Calibration of rX Part of Antenna

enumerator **PLATFORM_ID_SECURE**

16-bytes string defining the platform name - Channel independent

enumerator **CONFIG_VERSION_SECURE**

Version number of the secure parameters - Channel independent

enumerator **DBG_SE_DUMP_SECURE**

Config to enable/disable the SE_COMM_DATA_NTF. 0x00: Disable 0x01: Enable Note: Allowed in all lifecycles.

enum **eLocalizationZone_t**

Enumurator lists out the possible values of Localization Zones.
Values:

enumerator **eLocZone_Undefined**

enumerator **eLocZone_Frontside**

enumerator **eLocZone_Backside**

All information provided in this document is subject to legal disclaimers.
© 2026 NXP B.V. All rights reserved.

Functions

tUWBAPI_STATUS **UwbApi_GetFwCrashLog**(*phFwCrashLogInfo_t* *pLogInfo*)

APIs exposed to application to access UWB Board Specific Functionality.
Get Firmware Crash Log

Parameters

`pLogInfo` – **[out]** Pointer to phFwCrashLogInfo_t

Return values

- `UWBAPI_STATUS_OK` – on success
- `UWBAPI_STATUS_FAILED` – otherwise

tUWBAPI_STATUS **UwbApi_SetDefaultCoreConfigs**()

Set Default Core configs.

Return values

- `UWBAPI_STATUS_OK` – on success
- `UWBAPI_STATUS_FAILED` – otherwise

tUWBAPI_STATUS **UwbApi_DoChipCalibration**(*uint8_t channel, phDoCalibNtfStatus_t* *pDoCalibStatus*)

Do calibration parameters.

Parameters

- `channel` – **[in]** Channel
- `pDoCalibStatus` – **[out]** Pointer to phDoCalibNtfStatus_t

Return values

- `UWBAPI_STATUS_OK` – on success
- `UWBAPI_STATUS_NOT_INITIALIZED` – if UWB stack is not initialized
- `UWBAPI_STATUS_INVALID_PARAM` – if invalid parameters are passed
- `UWBAPI_STATUS_SESSION_NOT_EXIST` – if session is not initialized with sessionHandle
- `UWBAPI_STATUS_FAILED` – otherwise
- `UWBAPI_STATUS_TIMEOUT` – if command is timeout

tUWBAPI_STATUS **UwbApi_SetCalibration**(*uint8_t channel, eCalibParam paramId, uint8_t* *calibrationValue, *uint16_t length*)

Set calibration parameters.

Parameters

- `channel` – **[in]** channel
- `paramId` – **[in]** Calibration parameter ID
- `calibrationValue` – **[in]** Calibration value
- `length` – **[in]** Calibration value array length

Return values

- `UWBAPI_STATUS_OK` – on success
- `UWBAPI_STATUS_NOT_INITIALIZED` – if UWB stack is not initialized
- `UWBAPI_STATUS_INVALID_PARAM` – if invalid parameters are passed
- `UWBAPI_STATUS_SESSION_NOT_EXIST` – if session is not initialized with sessionHandle
- `UWBAPI_STATUS_FAILED` – otherwise
- `UWBAPI_STATUS_TIMEOUT` – if command is timeout

All information provided in this document is subject to legal disclaimers.
© 2026 NXP B.V. All rights reserved.

tUWBAPI_STATUS **UwbApi_GetCalibration**(*phGetCalibInputParams_t *pCalibInput, phCalibRespStatus_t *pCalibResp*)

Get calibration parameters.

Note: For AOA_ANTENNAS_PDOA_CALIB, calibResp acts as an in/out param To get the calibration values of the specified antennaId, the antennaId needs to be passed from application, by accessing the member of phCalibRespStatus_t i.e. inRxAntennaPair.

Example to get the calibration values for AOA_ANTENNAS_PDOA_CALIB with the specific antennaID:

```
phCalibRespStatus_t calibResp = {0x00};
phGetCalibInputParams_t calibInput = {0x00};

calibInput.channel = channel_id;
calibInput.paramId = AOA_ANTENNAS_PDOA_CALIB;
status = UwbApi_GetCalibration(&CalibInput, &calibResp);
if (status != UWBAPI_STATUS_OK) {
NXPLOG_APP_E("Set Calib param AOA_ANTENNAS_PDOA_CALIB Failed");
goto exit;
}
```

Parameters

- **`pCalibInput`** – **[in]** Pointer to phGetCalibInputParams_t
- **`pCalibResp`** – **[out]** Pointer to phCalibRespStatus_t

Return values

- **`UWBAPI_STATUS_OK`** – on success
- **`UWBAPI_STATUS_NOT_INITIALIZED`** – if UWB stack is not initialized
- **`UWBAPI_STATUS_INVALID_PARAM`** – if invalid parameters are passed
- **`UWBAPI_STATUS_SESSION_NOT_EXIST`** – if session is not initialized with sessionHandle
- **`UWBAPI_STATUS_FAILED`** – otherwise
- **`UWBAPI_STATUS_TIMEOUT`** – if command is timeout

tUWBAPI_STATUS **UwbApi_SetSecureCalibration**(*phSecureCalibParams_t *pSecureCalibParams*)

Sets the secured calibration parameters.

Note:

sensitive calibration parameters can't be modified when UWBS is in protected mode with the UwbApi_SetCalibration API. For Helios 2 (SR250), UwbApi_SetSecureCalibration API to be used.

Note:

Application needs to allocate memory for pSecureCalibParams->calibrationValue

Parameters

`pSecureCalibParams` – **[in]** - Pointer to phSecureCalibParams_t

Return values

- **`UWBAPI_STATUS_OK`** – - on success
- **`UWBAPI_STATUS_NOT_INITIALIZED`** – - if UWB stack is not initialized
- **`UWBAPI_STATUS_INVALID_PARAM`** – - if invalid parameters are passed
- **`UWBAPI_STATUS_FAILED`** – - otherwise

tUWBAPI_STATUS **UwbApi_SetDebugParams**(*uint32_t sessionHandle, uint8_t noOfparams, const UWB_DebugParams_List_t *DebugParams_List*)

Set Uwb Debug Configuration Parameters. This API Can be used to set any number of debug parameters at once.

To easily set the DebugParams list, following macros have been defined.

- UWB_SET_DEBUG_PARAM_VALUE_u8(Parameter, Value): This macro sets the value of the corresponding parameter with the given Value.This shall be used to set value of 8 bit wide.

All information provided in this document is subject to legal disclaimers. © 2026 NXP B.V. All rights reserved.

- UWB_SET_DEBUG_PARAM_VALUE_u16(Parameter, Value): This macro sets the value of the corresponding parameter with the given Value.This shall be used to set value of 16 bit wide.
- UWB_SET_DEBUG_PARAM_VALUE_u32(Parameter, Value): This macro sets the value of the corresponding parameter with the given Value.This shall be used to set value of 32 bit wide.
- UWB_SET_DEBUG_PARAM_ARRAY(PARAM, ARRAY, LENGTH): This macro sets the value of the corresponding parameter with given Value & length. This shall be used to set value of arrays.

Example: To set DATA_LOGGER_NTF to zero, macro shall be invoked as given below.

```
UWB_DebugParams_List_t SetDebugParamsList[] =
 {UWB_SET_DEBUG_PARAM_VALUE(DATA_LOGGER_NTF, 0)};
UWB_DebugParams_List_t SetDebugParamsList[] =
 {UWB_SET_DEBUG_PARAM_ARRAY(DATA_LOGGER_NTF, uint8_t*, 6)};
DATA_LOGGER_NTF is 6 byte for SR2XX.
```

Parameters

- **`sessionHandle`** – **[in]** Initialized Session Handle
- **`noOfparams`** – **[in]** Number of App Config Parameters
- **`DebugParams_List`** – **[in]** Debug parameters values in tlv format

Return values

- **`UWBAPI_STATUS_OK`** – on success
- **`UWBAPI_STATUS_NOT_INITIALIZED`** – if UWB stack is not initialized
- **`UWBAPI_STATUS_INVALID_PARAM`** – if invalid parameters are passed
- **`UWBAPI_STATUS_SESSION_NOT_EXIST`** – if session is not initialized with sessionHandle
- **`UWBAPI_STATUS_TIMEOUT`** – if command is timeout
- **`UWBAPI_STATUS_FAILED`** – otherwise

tUWBAPI_STATUS **UwbApi_GetDebugParams**(*uint32_t sessionHandle, uint8_t noOfparams, UWB_DebugParams_List_t *DebugParams_List*)

Get Uwb Debug Configuration Parameters. This API Can be used to get any number of debug parameters at once.

To easily get the DebugParams list, following macro has been defined.

- UWB_SET_GETDEBUG_PARAM_u8(Parameter): This macro gets the value of the corresponding parameter. This shall be used to get values of 8 bit wide.
- UWB_SET_GETAPP_PARAM_u16(Parameter): This macro gets the value of the corresponding parameter. This shall be used to get values of 16 bit wide.
- UWB_SET_GETAPP_PARAM_u32(Parameter): This macro gets the value of the corresponding parameter. This shall be used to get values of 32 bit wide.

Example: To get DATA_LOGGER_NTF macro shall be invoked as given below.

```
UWB_DebugParams_List_t GetDebugParamsList[] =
 {UWB_SET_GETAPP_PARAM(DATA_LOGGER_NTF),};
```

Parameters

- **`sessionHandle`** – **[in]** Initialized Session Handle
- **`noOfparams`** – **[in]** Number of App Config Parameters
- **`DebugParams_List`** – **[in]** Debug parameters values in tlv format

All information provided in this document is subject to legal disclaimers.
© 2026 NXP B.V. All rights reserved.

Return values

- `UWBAPI_STATUS_OK` – on success
- `UWBAPI_STATUS_NOT_INITIALIZED` – if UWB stack is not initialized
- `UWBAPI_STATUS_INVALID_PARAM` – if invalid parameters are passed
- `UWBAPI_STATUS_SESSION_NOT_EXIST` – if session is not initialized with sessionHandle
- `UWBAPI_STATUS_TIMEOUT` – if command is timeout
- `UWBAPI_STATUS_FAILED` – otherwise

tUWBAPI_STATUS **UwbApi_GetBindingCount**(*phSeGetBindingCount_t* **getBindingCount*)

Get the binding count using this API.

Parameters

`getBindingCount` – **[out]** getBindingCount data. valid only if API status is success

Return values

- `UWBAPI_STATUS_OK` – on success
- `UWBAPI_STATUS_NOT_INITIALIZED` – if UWB stack is not initialized
- `UWBAPI_STATUS_INVALID_PARAM` – if invalid parameters are passed
- `UWBAPI_STATUS_FAILED` – otherwise
- `UWBAPI_STATUS_TIMEOUT` – if command is timeout

tUWBAPI_STATUS **UwbApi_QueryTemperature**(*uint8_t* **pTemperatureValue*)

API to get the current temperature.

Return values

- `UWBAPI_STATUS_OK` – on success
- `UWBAPI_STATUS_NOT_INITIALIZED` – if UWB stack is not initialized
- `UWBAPI_STATUS_INVALID_PARAM` – if invalid parameters are passed
- `UWBAPI_STATUS_FAILED` – otherwise
- `UWBAPI_STATUS_TIMEOUT` – if command is timeout

tUWBAPI_STATUS **UwbApi_QueryUwbTimestamp**(*uint8_t len, uint8_t pTimestampValue[]*)

API to get the UWB Timestamp for UWB time synchronization.

On successful execution, this buffer will contain 8 bytes timestamp value.

Parameters

- `len` – **[in]** Length of i/p buffer. It should be 8 to hold 8 bytes timestamp value
- `pTimestampValue` – **[out]** Timestamp data. It should be 8 bytes in size to hold 8 bytes timestamp value.

Return values

- `UWBAPI_STATUS_OK` – on success
- `UWBAPI_STATUS_NOT_INITIALIZED` – if UWB stack is not initialized
- `UWBAPI_STATUS_INVALID_PARAM` – if invalid parameters are passed
- `UWBAPI_STATUS_TIMEOUT` – if command is timeout
- `UWBAPI_STATUS_BUFFER_OVERFLOW` – if response length is more than expected response size
- `UWBAPI_STATUS_FAILED` – otherwise

All information provided in this document is subject to legal disclaimers.
© 2026 NXP B.V. All rights reserved.

tUWBAPI_STATUS **UwbApi_UpdateActiveRoundsAnchor**(*uint32_t sessionHandle, uint8_t nActiveRounds, UWB_MacAddressMode_t macAddressingMode, const phActiveRoundsConfig_t roundConfigList[], phNotActivatedRounds_t *pNotActivatedRound*)

Update the active rounds during the DL-TDoA Session for a initiator or responder device.

Parameters

- **`sessionHandle`** – **[in]** : Unique Session Handle
- **`nActiveRounds`** – **[in]** : Number of active rounds
- **`macAddressingMode`** – **[in]** : MAC addressing mode- 2/8 bytes
- **`roundConfigList`** – **[in]** : List/array of size nActiveRounds of round index + role tuple
- **`pNotActivatedRound`** – **[out]** : Structure containing list of not activated index which couldn't be activated, in case return code is UWBAPI_STATUS_ERROR_ROUND_INDEX_NOT_ACTIVATED

Return values

- **`UWBAPI_STATUS_OK`** – on success
- **`UWBAPI_STATUS_NOT_INITIALIZED`** – if UWB stack is not initialized
- **`UWBAPI_STATUS_SESSION_NOT_EXIST`** – if session is not initialized with sessionHandle
- **`UWBAPI_STATUS_ERROR_ROUND_INDEX_NOT_ACTIVATED`** – if one or more rounds couldn't be activated
- **`UWBAPI_STATUS_ERROR_NUMBER_OF_ACTIVE_RANGING_ROUNDS_EXCEEDED`** – one or more given rounds exceed number of rounds available
- **`UWBAPI_STATUS_TIMEOUT`** – if command is timeout
- **`UWBAPI_STATUS_FAILED`** – otherwise

tUWBAPI_STATUS **UwbApi_SetAppConfigWrappedRDS**(*uint32_t sessionHandle, uint8_t *pWrappedRds, size_t WrappedRdsLen*)

Host shall use this API to set Wrapped RDS application configuration parameter.

Parameters

- **`sessionHandle`** – **[in]** Initialized Session Handle
- **`pWrappedRds`** – **[in]** Wrapped RDS
- **`WrappedRdsLen`** – **[in]** Wrapped RDS Length

Return values

- **`UWBAPI_STATUS_OK`** – on success
- **`UWBAPI_STATUS_NOT_INITIALIZED`** – if UWB stack is not initialized
- **`UWBAPI_STATUS_INVALID_PARAM`** – if invalid parameters are passed
- **`UWBAPI_STATUS_TIMEOUT`** – if command is timeout
- **`UWBAPI_STATUS_FAILED`** – otherwise

tUWBAPI_STATUS **UwbApi_ConfigureData_iOS**(*uint8_t *pShareableData, uint16_t ShareableDataLength, phUwbProfileInfo_t *pProfileInfo, uint8_t noOfVendorAppParams, const UWB_VendorAppParams_List_t *VendorAppParams_List, uint8_t noOfDebugParams, const UWB_DebugParams_List_t *DebugParams_List*)

Set iOS UWB configuration data.

All information provided in this document is subject to legal disclaimers.
© 2026 NXP B.V. All rights reserved.

Parameters

- **pShareableData** – **[in]** : sharable data which contain all information.
- **ShareableDataLength** – **[in]** : Size of sharable data
- **pProfileInfo** – **[inout]** : contains profile information.
- **noOfVendorAppParams** – **[in]** : number of VendorAppParams.
- **VendorAppParams_List** – **[in]** : List of VendorAppParams to be set.
- **noOfDebugParams** – **[in]** : number of DebugParams.
- **DebugParams_List** – **[in]** : List of Debug params to be set.

Return values

- **UWBAPI_STATUS_OK** – - on success
- **UWBAPI_STATUS_NOT_INITIALIZED** – - if UWB stack is not initialized
- **UWBAPI_STATUS_INVALID_PARAM** – - if invalid parameters are passed
- **UWBAPI_STATUS_FAILED** – - otherwise

tUWBAPI_STATUS **UwbApi_ConfigureData_Android**(*uint8_t *pUwbPhoneConfigData, uint16_t UwbPhoneConfigDataLen, phUwbProfileInfo_t *pProfileInfo, uint8_t noOfVendorAppParams, const UWB_VendorAppParams_List_t *VendorAppParams_List, uint8_t noOfDebugParams, const UWB_DebugParams_List_t *DebugParams_List*)

Set Android UWB configuration data.

Note:

There are 3 configurations supported namely CONFIG_ID_1, CONFIG_ID_2 and CONFIG_ID_3. The API sets/configures the respective configuration based on the value of config_id from Android jetpack.

Parameters

- **pUwbPhoneConfigData** – **[in]** : UwbPhoneConfigData_t data which contain all information.
- **UwbPhoneConfigDataLen** – **[in]** : Size of phone configuration data
- **pProfileInfo** – **[inout]** : contains profile information
- **noOfVendorAppParams** – **[in]** : number of VendorAppParams.
- **VendorAppParams_List** – **[in]** : List of VendorAppParams to be set.
- **noOfDebugParams** – **[in]** : number of DebugParams.
- **DebugParams_List** – **[in]** : List of Debug params to be set.

Return values

- **UWBAPI_STATUS_OK** – - on success
- **UWBAPI_STATUS_NOT_INITIALIZED** – - if UWB stack is not initialized
- **UWBAPI_STATUS_INVALID_PARAM** – - if invalid parameters are passed
- **UWBAPI_STATUS_FAILED** – - otherwise

tUWBAPI_STATUS **UwbApi_SessionSetLocZone**(*phSessionSetLocZone_t *pSetLocZone*)

Command to set the localization Zone information which shall be sent in Final Data 2 message in the following Ranging Blocks.

Note:

This command is only applicable to controlee.

Parameters

psetLocZone – **[in]** : Session set localization zone information set from Application.

All information provided in this document is subject to legal disclaimers.

© 2026 NXP B.V. All rights reserved.

Return values

- **UWBAPI_STATUS_OK** – - on success
- **UWBAPI_STATUS_NOT_INITIALIZED** – - if UWB stack is not initialized
- **UWBAPI_STATUS_INVALID_PARAM** – - if invalid parameters are passed
- **UWBAPI_STATUS_TIMEOUT** – - if command is timeout
- **UWBAPI_STATUS_FAILED** – - otherwise

tUWBAPI_STATUS **UwbApi_WriteOtpCalibDataCmd**(*uint8_t channel, uint16_t bitMask, phCalibPayload_t *pCalib Payload*)

Write the calibration parameters to OTP.

Parameters

- **channel** – **[in]** Channel ID
- **bitMask** – **[in]** bit mask for the configurations which are set by user and passed part of phCalibPayload_t.
  - bitmask description:
    - set bit 0 -> CHIP_CALIBRATION
    - set bit 1 -> TX_POWER_ID
    - set bit 2 -> XTAL_CAP_VALUES
    - set bit 3 -> RSSI_CONSTANT1
    - set bit 4 -> RSSI_CONSTANT2
    - bit 5 -> RFU
    - set bit 6 -> TX_POWER_PARAMS
    - bit 7 -> RFU
    - set bit 8 -> PA_PPA_CALIB_CTRL
    - set bit 9 -> TX_TEMP_COMP
    - bit 10 -> RFU
    - set bit 11 -> DELAY_CALIB
    - bit 12 to 15 -> RFU
- **pCalibPayload** – **[in]** param values to be set

Return values

- **UWBAPI_STATUS_OK** – on success
- **UWBAPI_STATUS_NOT_INITIALIZED** – if UWB stack is not initialized
- **UWBAPI_STATUS_INVALID_PARAM** – if invalid parameters are passed
- **UWBAPI_STATUS_TIMEOUT** – if command is timeout
- **UWBAPI_STATUS_FAILED** – otherwise

tUWBAPI_STATUS **UwbApi_ReadOtpCalibDataCmd**(*uint8_t channel, uint16_t bitMask, phCalibPayload_t *pCalib Payload*)

Read the calibration parameters from OTP.

All information provided in this document is subject to legal disclaimers.
© 2026 NXP B.V. All rights reserved.

Parameters

- `channel` – **[in]** Channel ID
- `bitMask` – **[in]** bit mask for the params to be retrieved from OTP
  - bitmask description:
    - set bit 0 -> CHIP_CALIBRATION
    - set bit 1 -> TX_POWER_ID
    - set bit 2 -> XTAL_CAP_VALUES
    - set bit 3 -> RSSI_CONSTANT1
    - set bit 4 -> RSSI_CONSTANT2
    - bit 5 -> RFU
    - set bit 6 -> TX_POWER_PARAMS
    - bit 7 -> RFU
    - set bit 8 -> PA_PPA_CALIB_CTRL
    - set bit 9 -> TX_TEMP_COMP
    - bit 10 -> RFU
    - set bit 11 -> DELAY_CALIB
    - bit 12 to 15 -> RFU
- `pCalibPayload` – **[out]** Calib payload structure

Return values

- `UWBAPI_STATUS_OK` – on success
- `UWBAPI_STATUS_NOT_INITIALIZED` – if UWB stack is not initialized
- `UWBAPI_STATUS_INVALID_PARAM` – if invalid parameters are passed
- `UWBAPI_STATUS_TIMEOUT` – if command is timeout
- `UWBAPI_STATUS_FAILED` – otherwise

struct **phCalibPayload_t**

#include <UwbApi_Types_Proprietary.h>
Calib params payload.

Public Members

uint16_t **CHIP_CALIBRATION**

CHIP CALIBRATION code

uint8_t **TX_POWER_ID**[4]

Tx power Id

uint8_t **XTAL_CAP_VALUES**[3]

XTAL cap values. It Can be set only once. Channel independent and remains same for each channel.

- Octet [0]: 38.4 MHz XTAL CAP1
- Octet [1]: 38.4 MHz XTAL CAP2
- Octet [2]: 38.4 MHz XTAL GM CURRNT CONTROL
- Values : [0x00-0xFF] for Octet[1:0]
- Values : [0x00-0x3F] for Octet[2]

uint8_t **RSSI_CONSTANT1**[8]

RSSI CONSTANT1(4*2). this parameter is channel and antenna pair dependent.

- Channel number to be provide in calibration commands to set this parameter.
- 4 antenna pairs with 2 RX each
- 2 RX: RX1 and RX2

All information provided in this document is subject to legal disclaimers.
© 2026 NXP B.V. All rights reserved.

uint8_t **RSSI_CONSTANT2**[8]

RSSI CONSTANT2(4*2). this parameter is channel and antenna pair dependent.

- Channel number to be provide in calibration commands to set this parameter.
- 4 antenna pairs with 2 RX each
- 2 RX: RX1 and RX2

uint8_t **TX_POWER_PARAMS**[16]

Tx power parameters (4*4) 4 Antenna pairs 4 parameters for each antenna pairs

- Octet[0]: PA_GAIN
- Octet[1]: PA_DRIVE_GAIN
- Octet[2]: DIG_GAIN
- Octet[3]: TX_DAC_GAIN

uint8_t **TX_TEMP_COMP**[16]

Tx tempreature comp. This parameter is dependent on the chosen channel and Tx antenna. (2*4*2):

- 2 bytes value description
  - Octet [0]:TX_POWER_TEMP_UPPER_BOUND
  - Octet [1]:TX_POWER_GAIN_INDEX
- 4 bytes: 2 Octets repeated 4 times to allow up to 4 different temperature ranges
  - Octet[1-0]: Temperature range 1 and gain index
  - Octet[3-2]: Temperature range 2 and gain index
  - Octet[5-4]: Temperature range 3 and gain index
  - Octet[7-6]: Temperature range 4 and gain index
- 2 bytes: Number of Tx antenna selection options

uint16_t **DELAY_CALIB_VALUE**

Delay calibration. It Can be set only once. Same value will be applied for all channels

struct **UWB_AppParams_value_au8_t**

#include <UwbApi_Types_Proprietary.h>
Set/Get App Configuration parameters value type supported in UWB API layer.

union **UWB_AppParams_value_t**

#include <UwbApi_Types_Proprietary.h>
Set/Get App Configuration parameters value structure supported in UWB API layer.

Public Members

uint32_t **vu32**

UWB_AppParams_value_au8_t **au8**

struct **UWB_Debug_Params_value_t**

#include <UwbApi_Types_Proprietary.h>
Set/Get Debug Configuration parameters value type supported in UWB API layer.

union **UWB_DebugParams_value_t**

#include <UwbApi_Types_Proprietary.h>
Set/Get Debug Configuration parameters value structure supported in UWB API layer.

Public Members

uint8_t **vu8**

uint16_t **vu16**

uint32_t **vu32**

All information provided in this document is subject to legal disclaimers.
© 2026 NXP B.V. All rights reserved.

UWB_Debug_Params_value_t **param**

struct **UWB_AppParams_List_t**

#include <UwbApi_Types_Proprietary.h>
Set/Get App Configuration parameters list supported in UWB API layer.

Public Members

eAppConfig **param_id**

Input: search this tag

UWB_AppParams_type_t **param_type**

Filled Implicitly: Expected type.

UWB_AppParams_value_t **param_value**

Input: Parameter Value

struct **UWB_VendorAppParams_List_t**

#include <UwbApi_Types_Proprietary.h>
Set/Get Vendor App Configuration parameters list supported in UWB API layer.

Public Members

eVendorAppConfig **param_id**

Input: search this tag

UWB_AppParams_type_t **param_type**

Filled Implicitly: Expected type.

UWB_AppParams_value_t **param_value**

Input: Parameter Value

struct **UWB_WiFiCoEx_Ftr_List_t**

#include <UwbApi_Types_Proprietary.h>
Structure lists doen the Multi channel WiFi Coex Features.

Public Members

uint8_t **UWB_WiFiCoEx_channel_Id**

Channel Id, for which the WiFiCoEx feature is configured

struct **UWB_WiFiCoEx_Ftr_t**

#include <UwbApi_Types_Proprietary.h>
This configuration is used to configure the WiFi CoEx feature.

Public Members

uint8_t **UWB_WiFiCoEx_noOfChannels**

Number of channels

UWB_WiFiCoEx_Ftr_List_t **wifiCoexFtrList**[MAX_CHANNELS]

List of WiFi CoEx Feature Configs

struct **UWB_DebugParams_List_t**

#include <UwbApi_Types_Proprietary.h>
Set/Get App Configuration parameters list supported in UWB API layer.

Public Members

All information provided in this document is subject to legal disclaimers.
© 2026 NXP B.V. All rights reserved.

UWB_DBG_CFG_t **param_id**

Input: search this tag

UWB_DebugParams_type_t **param_type**

Filled Implicitly: Expected type.

UWB_DebugParams_value_t **param_value**

Input: Parameter Value

struct **phUwbDevInfo_t**

#include <UwbApi_Types_Proprietary.h>
Structure lists out the UWB Device Info Parameters.

Public Members

uint16_t **uciGenericVersion**

UCI generic version

uint8_t **macMajorVersion**

Mac Major version

uint8_t **macMinorMaintenanceVersion**

Mac Minor version

uint8_t **phyMajorVersion**

Phy Major version

uint8_t **phyMinorMaintenanceVersion**

Phy Minor version

uint16_t **uciTestVersion**

UCI test version

uint8_t **devName**[UCI_EXT_PARAM_ID_DEVICE_NAME_LEN]

Device Name

uint8_t **fwMajor**

Fw Major Version

uint8_t **fwMinor**

Fw Minor Version

uint8_t **fwPatchVersion**

Fw Patch Version

uint8_t **mwMajor**

MW Major Version

uint8_t **mwMinor**

MW Minor Version

uint8_t **devMinor**

Device Minor Version

uint8_t **devMajor**

Device Major Version

uint8_t **serialNo**[UCI_EXT_PARAM_ID_SERIAL_NUMBER_LEN]

All information provided in this document is subject to legal disclaimers.
© 2026 NXP B.V. All rights reserved.

Serial No

uint8_t **dspMajor**

DSP Fw Major Version

uint8_t **dspMinor**

DSP Fw Minor Version

uint8_t **dspPatchVersion**

DSP Fw Patch Version

uint8_t **bbMajor**

Fw Major Version

uint8_t **bbMinor**

Fw Minor Version

uint8_t **bbPatchVersion**

Fw Patch Version

uint8_t **cccVersion**[UCI_EXT_PARAM_ID_CCC_VERSION_LEN]

Fw Patch Version
CCC VERSION

uint8_t **devNameLen**

Device Name length

uint8_t **fwRc**

Fw Rc Version

uint8_t **vendorUciMajor**

Vendor UCI Major Version

uint8_t **vendorUciMinor**

Vendor UCI Minor Version

uint8_t **vendorUciPatch**

Vendor UCI Patch Version

uint8_t **uwbChipId**[16]

UWB Chip Id

uint8_t **maxPpmValue**

Max PPM Value

int16_t **txPowerValue**

TX Power Value

uint32_t **lifecycle**

Parameter to indicate the lifecycle of device.

uint8_t **mwRc**

Mw Rc Version

uint8_t **uciGenericMajor**

Vendor FIRA UCI generic major version

uint8_t **uciGenericMinorMaintenanceVersion**

All information provided in this document is subject to legal disclaimers.
© 2026 NXP B.V. All rights reserved.

Vendor FIRA UCI generic minor version

uint8_t **uciGenericPatch**

Vendor FIRA UCI generic patch version

uint8_t **uciTestMajor**

Vendor FIRA UCI test major version

uint8_t **uciTestMinor**

Vendor FIRA UCI test minor version

uint8_t **uciTestPatch**

Vendor FIRA UCI test patch version

uint8_t **fwGitHash**[17]

Parameter to indicate git hash .

- Octet[15-0] = Git hash (coded in ASCII)
- Octet[16] = working tree state
  - 'z': dirty;
  - '-': clean;

uint8_t **fwBootMode**

Fw Boot Mode

uint8_t **uciCccVersion**[2]

UCI CCC VERSION

uint8_t **aliroSpecVersion**[8]

Aliro Spec VERSION

struct **phGetCalibInputParams_t**

#include <UwbApi_Types_Proprietary.h>
Calibration Parameter States.

Public Members

uint8_t **channel**

Channel ID

eCalibParam **paramId**

Calibration Parameter ID eCalibParam

uint8_t **rxAntennaPairID**

Calibration antenna pair only for aoa antennae pdoa calib

struct **phCalibRespStatus_t**

#include <UwbApi_Types_Proprietary.h>
Structure lists out the calibration command response/notification.

Public Members

uint8_t **status**

Status

uint8_t **numOfTagIds**

One byte Num of Tag Ids

uint8_t **calibValueOut**[MAX_UCI_PACKET_SIZE]

All information provided in this document is subject to legal disclaimers.
© 2026 NXP B.V. All rights reserved.

Calibration value out

uint16_t **length**

Calibration value length

uint8_t **calibState**

Calibration State in case of get_calib command and not used in case of do_calib command eCalibState

eCalibParam **paramId**

Calibration Parameter ID eCalibParam

uint16_t **calibrationValuelength**

Calibration value length. In parameter. It is assumed calibrationValuelength bytes allocated for pCalibration Value pointer. To be verified in the Calibration response that the received length matches with this

uint8_t ***pCalibrationValue**

Calibration value out Here is the base address and can be accessed till the length of calibrationValuelength.
Note:
memory to hold the caliberation value to be allocated and buffer address to be set by calling application.

struct **phDoCalibNtfStatus_t**

#include <UwbApi_Types_Proprietary.h>
Structure lists out the calibration (DO_CHIP_CALIBRATION_NTF) notification.

Public Members

uint8_t **calibNtfStatus**

Status values returned in notification from UWBS for DO_CHIP_CALIBRATION_CMD Below fields are valid if status is STATUS_OK

uint16_t **calibNtfValue**

Calibration value

For Helios 1 (SR150) bit[8:0] : Calibration code bit[14:9] : RFU bit[15] : Calibration status as below

- 0b1: Calibration is valid
- 0b0: Calibration is invalid

For Helios 2 (SR250) - Return the calibration outcome per channel. bit[8:0] : Calibration code bit[13:9] : RFU bit[15:14] : Calibration status as below

- 0b10: Calibration is valid
- Other: Calibration is invalid

struct **phGenerateTagRespStatus_t**

#include <UwbApi_Types_Proprietary.h>
Structure lists out the Generate Tag command response/notification.

Public Members

uint8_t **status**

Status

uint8_t **cmactag**[0x10U]

CMAC Tag

struct **phBlinkRespStatus_t**

#include <UwbApi_Types_Proprietary.h>
Structure lists out the blink command response/notification.

Public Members

All information provided in this document is subject to legal disclaimers.
© 2026 NXP B.V. All rights reserved.

uint8_t **repetition_count_status**

repetition count status

struct **phSessionIdList_t**

#include <UwbApi_Types_Proprietary.h>
UWBD Type for URSK_DELETION_REQ Notification.

Public Members

uint32_t **sessionId**

Session Id

uint8_t **status**

Status

struct **phUrskDeletionRequestStatus_t**

#include <UwbApi_Types_Proprietary.h>
UWBD Type for URSK_DELETION_REQ Notification.

Public Members

uint8_t **status**

Status

uint8_t **noOfSessionIds**

No of Session Ids

phSessionIdList_t **sessionIdList**[0x08]

Session Id list

struct **phSeGetBindingCount_t**

#include <UwbApi_Types_Proprietary.h>
UWBD Type for SE_GET_BINDING_COUNT_RSP.

Public Members

uint8_t **bindingStatus**

Binding Status

uint8_t **uwbdBindingCount**

Remaining binding count in uwb device

uint8_t **seBindingCount**

Remaining binding count in SE.
Warning:
for SR150, this field has to be 0.

struct **phTestLoopData_t**

#include <UwbApi_Types_Proprietary.h>
Structure lists out the SE test loop information.

Public Members

uint8_t **status**

Status

uint16_t **loop_cnt**

No of times loop was run

All information provided in this document is subject to legal disclaimers.
© 2026 NXP B.V. All rights reserved.

uint16_t **loop_pass_count**

No of times loop successfully completed

struct **phSeCommError_t**

#include <UwbApi_Types_Proprietary.h>
Structure lists out the SE Comm Error notification.

Public Members

uint8_t **status**

Status

uint16_t **cla_ins**

T=1 command for which SE communication is failed.

uint16_t **t_eq_1_status**

T=1 status codes(SW1SW2)

struct **phPdoaTableDef_t**

#include <UwbApi_Types_Proprietary.h>
Structure lists out the pdoa table define config.

Public Members

uint8_t **calibStepSize**

The calibration table step size in degrees which indicates the step size between two consecutive points

struct **phAoACalibCtrlAvgThreashPdoa_t**

#include <UwbApi_Types_Proprietary.h>
Structure lists out the aoa Calibration average threshold.

Public Members

uint16_t **avg_thresh_pdoa**[NO_OF_CALIB_PAIRS]

avg threshold pdoa for all angle sweeps

struct **phAoACalibCtrlAvgPdoa_t**

Public Members

uint16_t **avg_pdoa**[AOA_FINE_CALIB_HOR_SWEEPS]

avg pdoa for all angle sweeps

struct **phAoaCalib_t**

#include <UwbApi_Types_Proprietary.h>
Structure lists out the aoa calibration config.

Public Members

uint32_t **aoa_calib_antenna1**

aoa calibration antenna1

uint32_t **aoa_calib_antenna2**

aoa calibration antenna2

uint32_t **aoa_calib_antenna3**

aoa calibration antenna3

uint32_t **aoa_calib_antenna4**

All information provided in this document is subject to legal disclaimers.
© 2026 NXP B.V. All rights reserved.

aoa calibration antenna4

uint32_t **aoa_calib_antenna5**

aoa calibration antenna5

uint32_t **aoa_calib_antenna6**

aoa calibration antenna6

uint32_t **aoa_calib_antenna7**

aoa calibration antenna7

uint32_t **aoa_calib_antenna8**

aoa calibration antenna8

struct **phClkConfigSrc_t**

#include <UwbApi_Types_Proprietary.h>
Clock config parameters.

Public Members

uint8_t **clk_src_opt**

Clock source option

uint16_t **slow_clk_wait**

Slow clock waiting time Maximum waiting time in us until 32.768 kHz clock is present

uint16_t **rf_clk_wait**

RF clock waiting time Maximum waiting time in us until RF clock is present

struct **phDdfsToneConfig_t**

#include <UwbApi_Types_Proprietary.h>
Structure lists out the ddfs tone config.

Public Members

uint8_t **channel_no**

channel no

uint8_t **tx_antenna_selection**

Tx antenna selection

uint32_t **tx_ddfs_tone_0**

content of TX_DDFS_TONE_0

uint32_t **tx_ddfs_tone_1**

content of TX_DDFS_TONE_1

uint32_t **spur_duration**

spur duration

uint8_t **gainval_set**

value of GAINVAL_SET

uint8_t **ddfsgainbypass_enbl**

content of DDFSGAINBYPASS_ENBL

uint16_t **periodicity**

periodicity

All information provided in this document is subject to legal disclaimers.
© 2026 NXP B.V. All rights reserved.

struct **phAntennaDefines_t**

#include <UwbApi_Types_Proprietary.h>
Structure for antenna Identifier defines.

union **phDeviceConfigData_t**

#include <UwbApi_Types_Proprietary.h>
Device config data.

Public Members

uint8_t **lowPowerMode**

low power mode

uint16_t **dpdEntryTimeout**

DPD entry timeout

uint16_t **hpdEntryTimeout**

DPD entry timeout

uint8_t **mhrInCcm**

MHR_IN_CCM

uint8_t **ddfsToneConfig**

DDFS tone config enable

phTxPulseShapeConfig_t **txPulseShapeConfig**

Tx Telec config

uint8_t **nxpExtendedNtfConfig**

nxp extended ntf config

uint8_t **dpdWakeupSrc**

DPD wakeup src

uint8_t **wtxCountConfig**

WTX count config

uint8_t **txBaseBandConfig**

DDFS tone config enable

uint8_t **rxAntennaSelectionConfig**

Anttena selection Config

UWB_WiFiCoEx_Ftr_t **wifiCoExFtr**

WiFi CoEx Feature

phDdfsToneConfig_t **ddfsToneConfig**[NO_OF_CHANNELS]

DDFS Tone confg

uint16_t **hostMaxUCIPayloadLen**

Host Max UCI Payload Len

phPdoaTableDef_t **pdoaCalibTableDef**

phAoACalibCtrlAvgThreashPdoa_t **aoaCalibThresholdPdoa**

PDoA threshold values for all the calib pairs

uint16_t **initialRxOnOffsetAbs**

All information provided in this document is subject to legal disclaimers.
© 2026 NXP B.V. All rights reserved.

Negative offset when Rx should be enabled to receive the first message of a ranging round on Controlee compared to expected reception time. Default = 100 us

uint16_t **initialRxOnOffsetRel**

Negative offset when Rx should be enabled to receive the first message of a ranging round on Controlee compared to expected reception time. Default = 100 ppm

phAntennaDefines_t **antennaDefines**

To Define/Create all antenna Identifier for RX/TX/RX Pair

phClkConfigSrc_t **clockConfigCtrl**

Clock config parameters

uint8_t **wlanChannelInfo**

This configuration is used to configure the frequency of UWB Requests to WLAN for getting WLAN Channel Information.

uint16_t **clockPresentWaitingTime**

Maximum waiting time until clock is present

uint32_t **ddfsCfgPerPulseShape**

DDFS config PER pulse shape

uint8_t **aoaMode**

Used to define the type of computation to be considered by UWBS for obtaining AoA

phAoaCalib_t **aoaCalib**[NO_OF_CHANNELS]

AOA calibration control

phAoACalibCtrlAvgPdoa_t **aoaCalibAvgPdoa**[NO_OF_CALIB_PAIRS]

PDoA average of vertical sweeps for all the horizontal angles and for all the IDs

struct **phActiveRoundsConfig_t**

#include <UwbApi_Types_Proprietary.h>
Structure for storing Active round config Context.

Public Members

uint8_t **roundIndex**

Active Round Index

uint8_t **rangingRole**

Device role within the given round index. See UWB_DeviceRole_t

uint8_t **noofResponders**

Number M of Responder MAC Addresses, Possible values are between 1 to 8.

uint8_t ***responderMacAddressList**

Responder MAC Address List for the specified ranging round as Initiator DT-Anchor.

uint8_t **responderSlotScheduling**

Responder slot presence

- Possible values are:
  - 0x00: implicit scheduling, i.e., responder slots are not present;
  - 0x01: responder slots are present; therefore, M octets shall follow, specifying the assigned slot for each Responder DT-Anchor. 0x02-0xFF: RFU

uint8_t ***responderSlots**

All information provided in this document is subject to legal disclaimers.
© 2026 NXP B.V. All rights reserved.

Responder slot index assigned for responder transmissions

struct **phNotActivatedRounds_t**

#include <UwbApi_Types_Proprietary.h>

Structure for storing active round config rsp data in case of UWBAPI_STATUS_ERROR_ROUND_INDEX_NOT_ACTIVATED.

Public Members

uint8_t **noOfIndex**

No of index

uint8_t **indexList**[MAX_NO_OF_ACTIVE_RANGING_ROUND]

Index list

struct **phFwCrashLogInfo_t**

#include <UwbApi_Types_Proprietary.h>

Structure for storing Firmware Crash Info, allocate pLog buffer with max response.
length value and update the logLen filed

struct **UWB_Wlan_IndNtf_t**

#include <UwbApi_Types_Proprietary.h>

Structure for storing WiFiCoEx IND Ntf Context.

Public Members

uint8_t **UWB_Wlan_IndNtf_status**

UWB_WLAN_IND Status

uint32_t **UWB_Wlan_IndNtf_time_index**

Time Index where the GPIO change occured

uint32_t **UWB_Wlan_IndNtf_sessionHandle**

Session Handle to which UWB_WIFI_COEX_IND_NTF belongs

struct **Wlan_Uwb_IndNtf_t**

#include <UwbApi_Types_Proprietary.h>

Structure for storing WiFiCoEx Ind Error Notification.

Note:

This notification shall be sent when WIFI_COEX_FEATURE_CH5 config is set to 0x02 (Enable with Debug Verbose)

Public Members

uint8_t **Wlan_Uwb_IndNtf_status**

WLAN_UWB_IND_ERR Status

- 0x01: WLAN_UWB_IND_HIGH_AT_RR_START WLAN_UWB_IND GPIO is high at the start of the ranging round and ranging round shall be attempted.
- 0x02: WLAN_UWB_IND_HIGH_DURING_RR WLAN_UWB_IND GPIO changed to high during Ranging round

uint32_t **Wlan_Uwb_IndNtf_time_index**

Time Index where the GPIO change occured

struct **phSecureCalibParams_t**

#include <UwbApi_Types_Proprietary.h>

Structure lists out the Secure calibration command sturcture.

All information provided in this document is subject to legal disclaimers.
© 2026 NXP B.V. All rights reserved.

Public Members

uint8_t **channelId**

Channel ID Indicator

uint8_t **calibParamId**

Secure Calibration Param ID

uint16_t **length**

Secure Calibration Param Length

uint8_t ***calibrationValue**

Secure Calibration Param Value

uint32_t **configVersion**

Config Version value to be included

uint8_t **cryptoTag**[CRYPTO_TAG_LEN]

Crypto tag value for CMAC verification

struct **phSessionSetLocZone_t**

#include <UwbApi_Types_Proprietary.h>
Structure for storing fields of Session Set Localization Zone Command.

Public Members

uint32_t **setLocZone_SessionHandle**

Session Handle of the Session for which localization zone is estimated.

uint16_t **setLocZone_RangingBlockIndex**

Ranging Block Index for which localization zone is estimated.

eLocalizationZone_t **setLocZone_LocZone**

The estimated localization zone.

struct **sAoACoreConfigs_t**

#include <UwbApi_Types_Proprietary.h>
Structure lists out the fields for the PDAoA Configurations.

Public Members

uint8_t **key**

Key used to find the values against it NxpUwbConfig

const uint8_t ***pValue**

Value for the key

1.1.6 UWB OSAL APIs

1.1.6.1 API : UWB OSAL Interface

*group***grp_osal_uwb**

OSAL implementation related to thread, queue, memory, debug, random, semaphore and mutex functions.

Defines

PH_OSALUWB_MAX_TIMER

All information provided in this document is subject to legal disclaimers.
© 2026 NXP B.V. All rights reserved.

Note:
: API listed here encompasses Operating System Abstraction Layer interfaces which are used internal to the module. Include Files Macro Definitions Maximum Number of timers available with OSAL module

PH_OSALUWB_MAX_THREAD

Maximum Number of threads available with OSAL module

PH_OSALUWB_MSG_QUEUE_NOT_CREATED

Indicates whether thread has created its message queue or not

PH_OSALUWB_MAX_SEMAPHORE

Maximum Number of semaphores available with OSAL module

PH_OSALUWB_MAX_MUTEX

Maximum Number of Mutexes available with OSAL module

PH_OSALUWB_CONTINUABLE_EXCEPTION_TYPE

Continuable exception type

PH_OSALUWB_UNKNOWN_EXCEPTION_TYPE

Unknown exception type

PH_OSALUWB_RESET_VALUE

Value to reset variables of OSAL

PH_OSALUWB_VALUE_ONE

Value to reset variables of OSAL

TASK_NAME_MAX_LENGTH

struct **phOsalUwb_sOsalMutex_t**

#include <phOsalUwb_Internal.h>
Globals,Structure and Enumeration OSAL structure contains details of a Mutex

Public Members

QueueHandle_t **ObjectHandle**

Handle of the mutex object

struct **phOsalUwb_sOsalSemaphore_t**

#include <phOsalUwb_Internal.h>
OSAL structure contains details of a semaphore

Public Members

SemaphoreHandle_t **ObjectHandle**

Handle of the semaphore object

1.1.6.2 API : UWB OSAL Error Codes

*group***grp_osal_err**

UWB OSAL Error Codes
Include Files

Defines

PH_OSALUWB_SEMAPHORE_CREATION_ERROR

All information provided in this document is subject to legal disclaimers. © 2026 NXP B.V. All rights reserved.

A new semaphore could not be created due to a system error.

PH_OSALUWB_SEMAPHORE_PRODUCE_ERROR

The given semaphore could not be released due to a system error.

PH_OSALUWB_SEMAPHORE_CONSUME_ERROR

The given semaphore could not be consumed due to a system error.

PH_OSALUWB_SEMAPHORE_DELETE_ERROR

The given semaphore could not be deleted due to a system error.

PH_OSALUWB_MUTEX_CREATION_ERROR

A new mutex could not be created due to a system error.

PH_OSALUWB_MUTEX_LOCK_ERROR

The given Mutex could not be locked due to a system error.

PH_OSALUWB_MUTEX_UNLOCK_ERROR

The given Mutex could not be unlocked due to a system error.

PH_OSALUWB_MUTEX_DELETE_ERROR

The given mutex could not be deleted due to a system error.

PH_OSALUWB_TIMER_CREATE_ERROR

The Timer could not be created due to a system error

PH_OSALUWB_TIMER_START_ERROR

The Timer could not be started due to a system error or invalid handle

PH_OSALUWB_TIMER_STOP_ERROR

The Timer could not be stopped due to a system error or invalid handle

PH_OSALUWB_TIMER_DELETE_ERROR

The Timer could not be deleted due to a system error or invalid handle

PH_OSALUWB_TIMER_ID_INVALID

Invalid timer ID type.This ID used indicate timer creation is failed

PH_OSALUWB_TIMER_MSG

OSAL timer message .This message type will be posted to calling application thread.

PH_UWB_MAX_TIMER

1.1.6.3 API : UWB OSAL Thread Interface

*group***grp_osal_thread**

UWB OSAL Thread Interface
Include Files
Macro Definitions

Defines

UWBIOT_STACK_DEFINE(...)

PH_OSALUWB_THREAD_CREATION_ERROR

A new mutex could not be created due to a system error.

PH_OSALUWB_THREAD_SUSPEND_ERROR

All information provided in this document is subject to legal disclaimers.
© 2026 NXP B.V. All rights reserved.

The given thread could not be suspended due to a system error.

PH_OSALUWB_THREAD_RESUME_ERROR

The given thread could not be resumed due to a system error.

PH_OSALUWB_THREAD_DELETE_ERROR

The given thread could not be deleted due to a system error.

PH_OSALUWB_THREAD_SETPRIORITY_ERROR

The given thread could not be deleted due to a system error.

PH_OSALUWB_THREAD_JOIN_ERROR

The given thread could not be deleted due to a system error.

PH_OSALUWB_THREAD_ID_INVALID

Invalid timer ID type.This ID used indicate timer creation is failed

PH_UWB_MAX_THREAD

The maximum number of threads allowed in system.

Typedefs

typedef void (*__pphOsalUwb_ThreadFunction_t__)(void*)

Thread Function Pointer Declaration.

This points to the function to be invoked upon creation of thread. It is not the immediate thread procedure since the API of this function depends on the OS.

This function shall be called within the body of the thread procedure defined by the underlying, OS-depending OSAL implementation.

Functions

UWBSTATUS **phOsalUwb_CreateSemaphore**(*void **hSemaphore, uint8_t bInitialValue*)

Semaphore Creation. This function creates a semaphore in the underlying system.

Parameters

- **`hSemaphore`** – **[inout]** The handle: The caller has to prepare a void pointer where the handle of semaphore shall be returned.
- **`bInitialValue`** – **[in]** The initial value of the Semaphore.

Return values

- **`UWBSTATUS_SUCCESS`** – The operation was successful.
- **`UWBSTATUS_INVALID_PARAMETER`** – Parameter passed to the function is not Correct.
- **`UWBSTATUS_INSUFFICIENT_RESOURCES`** – All semaphores are occupied by other threads.
- **`PH_OSALUWB_SEMAPHORE_CREATION_ERROR`** – A new semaphore could not be created due to system error.
- **`UWBSTATUS_NOT_INITIALISED`** – Osal Module is not Initialized.

UWBSTATUS **phOsalUwb_ProduceSemaphore**(*void *hSemaphore*)

Semaphore-Produce (never waiting).

Increment the value of the semaphore. The purpose is to enable a waiting thread (“consumer”) to continue if it has been waiting because of the Semaphore value set to zero.

Parameters

`hSemaphore` – **[in]** The handle of the Semaphore.

All information provided in this document is subject to legal disclaimers.
© 2026 NXP B.V. All rights reserved.

Return values

- **UWBSTATUS_SUCCESS** – The operation was successful.
- **UWBSTATUS_INVALID_PARAMETER** – Parameter passed to the function is not Correct.
- **PH_OSALUWB_SEMAPHORE_PRODUCE_ERROR** – The semaphore cannot be produced due to a system error or invalid handle .
- **UWBSTATUS_NOT_INITIALISED** – Osal Module is not Initialized.

UWBSTATUS **phOsalUwb_ConsumeSemaphore_WithTimeout**(*void *hSemaphore, uint32_t delay*)

Semaphore Consumption (waiting if value is zero).

Decrement the value of the semaphore. When the internal value is non-zero, the function continues. If the value is zero however, the function blocks till the semaphore is released.

Parameters

- **hSemaphore** – **[in]** The handle of the Semaphore.
- **delay** – **[in]** time to wait for

Return values

- **UWBSTATUS_SUCCESS** – The operation was successful.
- **UWBSTATUS_INVALID_PARAMETER** – Parameter passed to the function is not Correct.
- **PH_OSALUWB_SEMAPHORE_CONSUME_ERROR** – The semaphore can not be consumed due to a system error or invalid handle .
- **UWBSTATUS_NOT_INITIALISED** – Osal Module is not Initialized.

UWBSTATUS **phOsalUwb_DeleteSemaphore**(*void **hSemaphore*)

Semaphore Deletion. This function deletes the Semaphore in the underlying OS.

Parameters

hSemaphore – **[in]** The handle of the Semaphore.

Return values

- **UWBSTATUS_SUCCESS** – The operation was successful.
- **UWBSTATUS_INVALID_PARAMETER** – Parameter passed to the function is not Correct.
- **PH_OSALUWB_SEMAPHORE_DELETE_ERROR** – The semaphore can not be deleted due to a system error or invalid handle .
- **UWBSTATUS_NOT_INITIALISED** – Osal Module is not Initialized.

UWBSTATUS **phOsalUwb_CreateMutex**(*void **hMutex*)

Mutex Creation. This function creates a Mutex in the underlying system.

Parameters

hMutex – **[inout]** The handle: The caller has to prepare a void pointer where the handle of mutex shall be returned.

All information provided in this document is subject to legal disclaimers.
© 2026 NXP B.V. All rights reserved.

Return values

- **UWBSTATUS_SUCCESS** – The operation was successful.
- **UWBSTATUS_INVALID_PARAMETER** – Parameter passed to the function is not Correct.
- **UWBSTATUS_INSUFFICIENT_RESOURCES** – All Mutexes are occupied by other threads.
- **PH_OSALUWB_MUTEX_CREATION_ERROR** – A new mutex could not be created due to system error.
- **UWBSTATUS_NOT_INITIALISED** – Osal Module is not Initialized.

UWBSTATUS **phOsalUwb_CreateBinSem**(*void **hBinSem*)

Bin Sem Creation. This function creates a Sem in the underlying system.

Parameters

hBinSem – **[out]** The handle: The caller has to prepare a void pointer where the handle of sem shall be returned.

Return values

- **UWBSTATUS_SUCCESS** – The operation was successful.
- **UWBSTATUS_INVALID_PARAMETER** – Parameter passed to the function is not Correct.
- **UWBSTATUS_INSUFFICIENT_RESOURCES** – All sem are occupied by other threads.
- **PH_OSALUWB_MUTEX_CREATION_ERROR** – A new sem could not be created due to system error.
- **UWBSTATUS_NOT_INITIALISED** – Osal Module is not Initialized.

UWBSTATUS **phOsalUwb_LockMutex**(*void *hMutex*)

Mutex Locking. This function locks a mutex used for handling critical section of code. However the function blocks till the Mutex is available to be occupied.

Parameters

hMutex – **[in]** The handle of the Mutex.

Return values

- **UWBSTATUS_SUCCESS** – The operation was successful.
- **UWBSTATUS_INVALID_PARAMETER** – Parameter passed to the function is not Correct.
- **PH_OSALUWB_MUTEX_LOCK_ERROR** – The mutex cannot be locked due to a system error or invalid handle.
- **UWBSTATUS_NOT_INITIALISED** – Osal Module is not Initialized.

UWBSTATUS **phOsalUwb_UnlockMutex**(*void *hMutex*)

Mutex Unlocking. This function unlocks a mutex after updating critical section of code.

Parameters

hMutex – **[in]** The handle of the Mutex.

All information provided in this document is subject to legal disclaimers.
© 2026 NXP B.V. All rights reserved.

Return values

- **UWBSTATUS_SUCCESS** – The operation was successful.
- **UWBSTATUS_INVALID_PARAMETER** – Parameter passed to the function is not Correct.
- **PH_OSALUWB_MUTEX_UNLOCK_ERROR** – The mutex cannot be locked due to a system error or invalid handle.
- **UWBSTATUS_NOT_INITIALISED** – Osal Module is not Initialized.

UWBSTATUS **phOsalUwb_DeleteMutex**(*void **hMutex*)

Mutex Deletion. This function deletes a Mutex in the underlying system.

Parameters

hMutex – **[in]** The handle of the Mutex.

Return values

- **UWBSTATUS_SUCCESS** – The operation was successful.
- **PH_OSALUWB_MUTEX_DELETE_ERROR** – The mutex cannot be deleted due to a system error or invalid handle.
- **UWBSTATUS_NOT_INITIALISED** – Osal Module is not Initialized.

void **phOsalUwb_GetTickCount**(*unsigned long *dwTickCount*)

Get tick count since scheduler has started.

Parameters

dwTickCount – **[in]** The address of the variable to which tickcount value will be assigned.

Return values

Returns – Tick count

UWBSTATUS **phOsalUwb_Thread_Create**(*void **hThread, pphOsalUwb_ThreadFunction_t pThreadFunction, void *pParam*)

Thread Creation.

This function creates a thread in the underlying system. To delete the created thread use the phOsalUwb_Thread_Delete function.

Parameters

- **hThread** – **[inout]** The Thread handle: The caller has to prepare a void pointer that need not to be initialized. The value (content) of the pointer is set by the function.
- **pThreadFunction** – **[in]** Pointer to a function within the implementation that shall be called by the Thread procedure. This represents the Thread main function. When this function exits the thread exits.
- **pParam** – **[in]** A pointer to a user-defined location the thread function receives.

Return values

- **UWBSTATUS_SUCCESS** – The operation was successful.
- **UWBSTATUS_INSUFFICIENT_RESOURCES** – At least one parameter value is invalid.
- **PH_OSALUWB_THREAD_CREATION_ERROR** – A new Thread could not be created due to system error.
- **UWBSTATUS_NOT_INITIALISED** – Osal Module is not Initialized.

UWBSTATUS **phOsalUwb_Thread_Delete**(*xTaskHandle hThread*)

Terminates the thread.

All information provided in this document is subject to legal disclaimers.
© 2026 NXP B.V. All rights reserved.

This function Terminates the thread passed as a handle.

Parameters

hThread – **[in]** The handle of the system object.

Return values

- **UWBSTATUS_SUCCESS** – The operation was successful.
- **UWBSTATUS_INVALID_PARAMETER** – At least one parameter value is invalid.
- **PH_OSALUWB_THREAD_DELETE_ERROR** – Thread could not be deleted due to system error.
- **UWBSTATUS_NOT_INITIALISED** – Osal Module is not Initialized.

xTaskHandle **phOsalUwb_GetTaskHandle**(*void*)

Create Event.

Return values

void – Task handle as per underlying implementation.

void **phOsalUwb_TaskResume**(*xTaskHandle xTaskToResume*)

This API allows to resume the current thread execution.

Note:

This function executes successfully without OSAL module Initialization.

Parameters

xTaskToResume – **[in]** Task to resume.

Return values

None –

void **phOsalUwb_TaskSuspend**(*xTaskHandle xTaskToSuspend*)

This API allows to suspend the current thread execution.

Note:

This function executes successfully without OSAL module Initialization.

Parameters

xTaskToSuspend – **[in]** Task to suspend.

Return values

None –

void **phOsalUwb_TaskStartScheduler**(*void*)

This API allows to start the scheduler.

Note:

This function executes successfully without OSAL module Initialization.

Return values

None –

void **phOsalUwb_Thread_Context_Switch**(*void*)

Context switch task

This function calls freertos TaskYIELD() to request context switch to another task.

Return values

None –

void **phOsalUwb_Thread_Join**(*xTaskHandle hThread*)

All information provided in this document is subject to legal disclaimers.
© 2026 NXP B.V. All rights reserved.

To join the thread This Function suspends execution of the calling thread until the target thread is terminated, in native system with pthread_t, does nothing on RTOS base system.

Parameters

`hThread` – **[in]** The handle of thread.

Return values

`None` –

struct **phOsalUwb_sOsalThreadHandle_t**

#include <phOsalUwb_Thread.h>

Thread handle Declaration. OSAL Handle structure containing details of a thread.

Public Members

pphOsalUwb_ThreadFunction_t **ThreadFunction**

Thread function to be invoked

void ***Params**

Parameters to the thread function

TaskHandle_t **ObjectHandle**

Handle of the Task object

struct **phOsalUwb_ThreadCreationParams_t**

#include <phOsalUwb_Thread.h>

Parameters to create a thread/task OS Independent way.

Public Members

uint16_t **stackdepth**

stackdepth in stackwidth units

char **taskname**[configMAX_TASK_NAME_LEN]

Name of task

void ***pContext**

Context passed to thread at creation

uint16_t **priority**

Priority of this thread

1.1.6.4 API : UWB OSAL Memory Management Interface

*group***grp_osal_mm**

Globals,Structure and Enumeration

Defines

MAX_DELAY

Functions

void ***phOsalUwb_GetMemory**(*uint32_t dwSize*)

Allocates some memory.

Note:

This function executes successfully without OSAL module initialization.

All information provided in this document is subject to legal disclaimers.

© 2026 NXP B.V. All rights reserved.

Parameters

`dwSize` – **[in]** Size, in uint32_t, to be allocated

Return values

- `NON-NULL` – value: The memory is successfully allocated ; the return value is a pointer to the allocated memory location
- `NULL` – The operation is not successful.

void **phOsalUwb_FreeMemory**(*void *pMem*)

This API allows to free already allocated memory.

Note:

This function executes successfully without OSAL module Initialization.

Parameters

`pMem` – **[in]** Pointer to the memory block to be deallocated

void **phOsalUwb_Delay**(*uint32_t dwDelay*)

This API allows to delay the current thread execution.

Note:

This function executes successfully without OSAL module Initialization.

Parameters

`dwDelay` – **[in]** Duration in milliseconds for which thread execution to be halted.

int32_t **phOsalUwb_MemCompare**(*const void *pDest, const void *pSrc, uint32_t dwSize*)

Compares the values stored in the source memory with the values stored in the destination memory.

Note:

This function executes successfully without OSAL module Initialization.

Parameters

- `pDest` – **[in]** Pointer to the Destination string.
- `pSrc` – **[in]** Pointer to the Source string.
- `dwSize` – **[in]** Number of bytes to be compared.

Return values

- `Zero` – value: The comparison is successful, Both the memory areas contain identical values.
- `Non-Zero` – Value: The comparison failed, both the memory areas are non-identical.

void **phOsalUwb_SetMemory**(*void *pMem, uint8_t bVal, uint32_t dwSize*)

Sets the given value in the memory locations.

Note:

This function executes successfully without OSAL module Initialization.

Parameters

- `pMem` – **[in]** Pointer to the memory block to be set to a value
- `bVal` – **[in]** Value to be set
- `dwSize` – **[in]** Number of bytes to be set.

void **phOsalUwb_MemCopy**(*void *pDest, const void *pSrc, uint32_t dwSize*)

Copies the values stored in the source memory to the values stored in the destination memory.

Note:

This function executes successfully without OSAL module Initialization.

All information provided in this document is subject to legal disclaimers.
© 2026 NXP B.V. All rights reserved.

Parameters

- `pDest` – **[in]** Pointer to the Destination Memory
- `pSrc` – **[in]** Pointer to the Source Memory
- `dwSize` – **[in]** Number of bytes to be copied.

1.1.6.5 API : UWB OSAL Queue Interface

*group***grp_osal_queue**

UWB OSAL Queue Interface

Functions

intptr_t **phOsalUwb_msgget**(*uint32_t queueLength*)

Allocates message queue.

Parameters

`queueLength` – **[in]** Length of the queue ignored for linux

Return values

`(int)` – value of pQueue if successful -1, if failed to allocate memory or to init mutex

void **phOsalUwb_msgrelease**(*intptr_t msqid*)

Releases message queue.

Parameters

`msqid` – **[in]** message queue handle

Return values

`None` –

int **phOsalUwb_msgctl**(*intptr_t msqid, int cmd, void *buf*)

Destroys message queue, needed only for posix.

Parameters

- `msqid` – **[in]** message queue handle
- `cmd` – **[in]** ignored, included only for Linux queue API compatibility
- `buf` – **[in]** ignored, included only for Linux queue API compatibility

Return values

- `0` – if successful
- `-1` – if invalid handle is passed

UWBSTATUS **phOsalUwb_msgsnd**(*intptr_t msqid, phLibUwb_Message_t *msg, unsigned long waittimeout*)

Sends a message to the queue. The message will be added at the end of the queue as appropriate for FIFO policy.

Parameters

- `msqid` – **[in]** message queue handle
- `msg` – **[in]** message to be sent
- `waittimeout` – **[in]** timeout in ms

All information provided in this document is subject to legal disclaimers.
© 2026 NXP B.V. All rights reserved.

Return values

- **`UWBSTATUS_SUCCESS`** – if successful
- **`UWBSTATUS_FAILED`** – if invalid parameter passed or failed to allocate memory

UWBSTATUS **phOsalUwb_msgrcv**(*intptr_t msqid, phLibUwb_Message_t *msg, unsigned long waittimeout*)

Gets the oldest message from the queue. If the queue is empty the function waits (blocks on a mutex) until a message is posted to the queue with phOsalUwb_msgsnd.

Parameters

- **`msqid`** – **[in]** message queue handle
- **`msg`** – **[in]** container for the message to be received
- **`waittimeout`** – **[in]** max wait time for task in ms

Return values

- **`UWBSTATUS_SUCCESS`** – if successful
- **`UWBSTATUS_FAILED`** – if invalid parameter passed

int **phOsalUwb_queueSpacesAvailable**(*intptr_t msqid*)

Gets the space available in queue.

Parameters

`msqid` – **[in]** message queue handle

Return values

- **`-1`** – if invalid parameter passed
- **`others`** – space available if successful

UWBSTATUS **phOsalUwb_msgreset**(*intptr_t msqid*)

Required for PCTT Demo APP Which needs queue reset in case interface is stuck.

Parameters

`msqid` – **[in]** message queue handle

Return values

`UWBSTATUS_SUCCESS` – if successful

1.1.6.6 API : UWB OSAL Timer Interface

*group***grp_osal_timer**

UWB OSAL Timer Interface

Globals,Structure and Enumeration

Functions

uint32_t **phOsalUwb_Timer_Create**(*bool bAutoReload*)

Creates a timer which shall call back the specified function when the timer expires Fails if OSAL module is not initialized or timers are already occupied.

Parameters

`bAutoReload` – **[in]** (not used in Posix) If bAutoReload is set to TRUE then the timer will expire repeatedly with a frequency set by the xTimerPeriodInTicks parameter. Timer restarts automatically. If bAutoReload is set to FALSE then the timer will be a one-shot timer and enter the dormant state after it expires.

All information provided in this document is subject to legal disclaimers.

© 2026 NXP B.V. All rights reserved.

Return values

TimerId – TimerId value of PH_OSALUWB_TIMER_ID_INVALID indicates that timer is not created -

UWBSTATUS **phOsalUwb_Timer_Start**(*uint32_t dwTimerId, uint32_t dwRegTimeCnt, pphOsalUwb_TimerCallbck_t pApplication_callback, void *pContext*)

Creates a timer which shall call back the specified function when the timer expires Starts the requested, already created, timer If the timer is already running, timer stops and restarts with the new timeout value and new callback function in case any.

Parameters

- **dwTimerId** – **[in]** valid timer ID obtained during timer creation
- **dwRegTimeCnt** – **[in]** requested timeout in milliseconds
- **pApplication_callback** – **[in]** application callback interface to be called when timer expires
- **pContext** – **[in]** caller context, to be passed to the application callback function

Return values

- **UWBSTATUS_SUCCESS** – the operation was successful
- **UWBSTATUS_NOT_INITIALISED** – OSAL Module is not initialized
- **UWBSTATUS_INVALID_PARAMETER** – invalid parameter passed to the function
- **PH_OSALUWB_TIMER_START_ERROR** – timer could not be created due to system error

UWBSTATUS **phOsalUwb_Timer_Stop**(*uint32_t dwTimerId*)

Stops already started timer, allows to stop running timer. In case timer is stopped, timer callback will not be notified any more.

Parameters

dwTimerId – **[in]** valid timer ID obtained during timer creation

Return values

- **UWBSTATUS_SUCCESS** – the operation was successful
- **UWBSTATUS_NOT_INITIALISED** – OSAL Module is not initialized
- **UWBSTATUS_INVALID_PARAMETER** – invalid parameter passed to the function
- **PH_OSALUWB_TIMER_STOP_ERROR** – timer could not be stopped due to system error

UWBSTATUS **phOsalUwb_Timer_Delete**(*uint32_t dwTimerId*)

Deletes previously created timer Allows to delete previously created timer. In case timer is running, it is first stopped and then deleted.

Parameters

dwTimerId – **[in]** valid timer ID obtained during timer creation

Return values

- **UWBSTATUS_SUCCESS** – the operation was successful
- **UWBSTATUS_NOT_INITIALISED** – OSAL Module is not initialized
- **UWBSTATUS_INVALID_PARAMETER** – invalid parameter passed to the function
- **PH_OSALUWB_TIMER_DELETE_ERROR** – timer could not be deleted due to system error

uint32_t **phUtilUwb_CheckForAvailableTimer**(*void*)

Find an available timer id.

All information provided in this document is subject to legal disclaimers.
© 2026 NXP B.V. All rights reserved.

Return values

`Available` – timer id

UWBSTATUS **phOsalUwb_CheckTimerPresence**(*void *pObjectHandle*)

Checks the requested timer is present or not.

Parameters

`pObjectHandle` – **[in]** timer context

Return values

`UWBSTATUS_SUCCESS` – if found

1.1.7 UWB Helper APIs

*group***uwb_utils**

Helper APIs exposed by UWB MW

Functions

void **parseRangingNtf**(*uint8_t *p, uint16_t len, phRangingData_t *pRngData*)

Extracts Ranging Params from the given byte array and updates the structure phRangingData_t.

Parameters

- **`p`** – Pointer to byte array containing range notification
- **`len`** – Length of input array `p`
- **`pRngData`** – Pointer to phRangingData_t structure to be populated

void **parseRadarNtf**(*uint8_t *p, uint16_t len, phUwbRadarNtf_t *pRadarNtf, uint8_t *pRadarNtfBuff*)

Extracts Radar Params from the given byte array and updates structure phUwbRadarNtf_t and pRadarNtfBuff if passed.

Parameters

- **`p`** – Pointer to byte array containing radar notification
- **`len`** – Length of input array `p`
- **`pRadarNtf`** – Pointer to phUwbRadarNtf_t structure to be populated
- **`pRadarNtfBuff`** – Pointer to output buffer to be populated with cirdata

void **parseCsaRangingNtf**(*uint8_t *p, uint16_t len, phCsaRangingData_t *pCsaRangingData*)

Extracts CSA Ranging Params from the given byte array and updates structure phCsaRangingData_t.

Parameters

- **`p`** – Pointer to byte array containing CSA ranging notification
- **`len`** – Length of input array `p`
- **`pCsaRangingData`** – Pointer to phCsaRangingData_t structure to be populated

void **parseDataRcvNtf**(*uint8_t *p, uint16_t len, phUwbRcvDataPkt_t *pRcvDataPkt*)

Extracts Data notification from the given byte array and updates structure phUwbRcvDataPkt_t.

Parameters

- **`p`** – Pointer to byte array containing data receive notification
- **`len`** – Length of input array `p`
- **`pRcvDataPkt`** – Pointer to phUwbRcvDataPkt_t structure to be populated

void **parseLogicalDataRcvNtf**(*uint8_t *p, uint16_t len, phLogicalLinkDataPkt_t *pRcvDataPkt*)

All information provided in this document is subject to legal disclaimers.
© 2026 NXP B.V. All rights reserved.

Extracts Data notification from the given byte array and updates structure phLogicalLinkDataPkt_t.

Parameters

- **p** – Pointer to byte array containing data receive notification
- **len** – Length of input array p
- **pRcvDataPkt** – Pointer to phLogicalLinkDataPkt_t structure to be populated

void **parseNewRoleChangeRcvNtf**(*uint8_t *p, uint16_t len, phNewRoleChangeNtf_t *pNewRole*)

Extracts session Role Change notification from the given byte array and updates structure phNewRoleChangeNtf_t.

Parameters

- **p** – Pointer to byte array containing data receive notification
- **len** – Length of input array p
- **pNewRole** – Pointer to phNewRoleChangeNtf_t structure to be populated

1.1.8 APIs for Porting

*group***uwb_uwbs**

This layer depends on uwb_bus_board.h and uwb_bus_interface.h
The implementation of all these APIs would be UWBD and Transport specific. e.g. If PnP mode of transport is used, RTCSync pins of SR1XX need be bothered about.
These APIs are HANDSHAKE and Protocol Aware with the UWBS.

- It menas waiting for any PIN before any operation
- It means reading “header” first, then then reading the rest of the frame later.

Variables

uwb_bus_board_ctx_t **busCtx**

Lower lever bus specific context SPI/PNP/SOCKET
This structure is defined by Implementation layer, and not by the common UWB Library

uwb_uwbs_tml_mode_t **mode**

tml interface read write mode

void ***mSyncMutex**

This mutex is used to make tml interface read and write operations mutually exclusive

int **noOfBytesWritten**

*group***uwb_uwbs_tml_ctx**

Enums

enum **uwb_uwbs_tml_mode_t**

Mode of operation of the TML Layer
This functional mode selection simplifies handling of low level transport protocol.
Values:

enumerator **kUWB_UWBS_TML_MODE_UCI**

Defaut Mode

enumerator **kUWB_UWBS_TML_MODE_SWUP**

FW Download mode for SR040

All information provided in this document is subject to legal disclaimers.
© 2026 NXP B.V. All rights reserved.

enumerator **kUWB_UWBS_TML_MODE_HBCI**

FW Donwnload mode for SR1XXT

enumerator **kUWB_UWBS_TML_MODE_HDLL**

FW Download mode/protocol for SR2XXT

struct **uwb_uwbs_tml_ctx_t**

#include <uwb_uwbs_tml_interface.h>
Context for the Transport Layer.
This allows management of data / layer information.

*group***uwb_uwbs_tml_data**

Defines

SWITCH_PROTOCOL_SWUP

SWITCH_PROTOCOL_UCI

Functions

UWBStatus_t **uwb_uwbs_tml_init**(*uwb_uwbs_tml_ctx_t *pCtx*)

Initailize it with some sane values

Parameters

`pCtx` – The context

UWBStatus_t **uwb_uwbs_tml_deinit**(*uwb_uwbs_tml_ctx_t *pCtx*)

De-Iniailize the context and free up references

Parameters

`pCtx` – The context

UWBStatus_t **uwb_uwbs_tml_setmode**(*uwb_uwbs_tml_ctx_t *pCtx, uwb_uwbs_tml_mode_t mode*)

Initailize it with some sane values

Parameters

- **`pCtx`** – The context
- **`mode`** – See uwb_uwbs_tml_mode_t

UWBStatus_t **uwb_uwbs_tml_data_tx**(*uwb_uwbs_tml_ctx_t *pCtx, uint8_t *pBuf, size_t bufLen*)

Transmit a data frame

Parameters

- **`pCtx`** – The context
- **`pBuf`** – **[in]** The data that we want to transmit
- **`bufLen`** – **[in]** The data length

Returns

Status of Transmit

UWBStatus_t **uwb_uwbs_tml_data_rx**(*uwb_uwbs_tml_ctx_t *pCtx, uint8_t *pBuf, size_t *pBufLen*)

Receive a data frame

All information provided in this document is subject to legal disclaimers.
© 2026 NXP B.V. All rights reserved.

Parameters

- **`pCtx`** – The context
- **`pBuf`** – **[out]** The pointer where we copy received data
- **`pBufLen`** – **[inout]** Input: The max length that we can copy. Output: actual length read.

Returns

Status of Receive

UWBStatus_t **uwb_uwbs_tml_helios_get_hdll_edl_ntf**(*uwb_uwbs_tml_ctx_t *pCtx, uint8_t *pRxBuf, size_t *pRx BufLen*)

Transmit a data frame which read hdll and edl notification.

Parameters

- **`pCtx`** – The context
- **`pRxBuf`** – **[out]** The pointer where we copy received data
- **`pRxBufLen`** – **[in]** No of bytes to read

Returns

Status of Tx/Rx

UWBStatus_t **uwb_uwbs_tml_reset**(*uwb_uwbs_tml_ctx_t *pCtx*)

Reset uwbs tml interface

Parameters

`pCtx` – The context

Returns

Status of reset

All information provided in this document is subject to legal disclaimers.
© 2026 NXP B.V. All rights reserved.

Legal information

Definitions

Draft — A draft status on a document indicates that the content is still under internal review and subject to formal approval, which may result in modifications or additions. NXP Semiconductors does not give any representations or warranties as to the accuracy or completeness of information included in a draft version of a document and shall have no liability for the consequences of use of such information.

Disclaimers

Limited warranty and liability — Information in this document is believed to be accurate and reliable. However, NXP Semiconductors does not give any representations or warranties, expressed or implied, as to the accuracy or completeness of such information and shall have no liability for the consequences of use of such information. NXP Semiconductors takes no responsibility for the content in this document if provided by an information source outside of NXP Semiconductors.

In no event shall NXP Semiconductors be liable for any indirect, incidental, punitive, special or consequential damages (including - without limitation - lost profits, lost savings, business interruption, costs related to the removal or replacement of any products or rework charges) whether or not such damages are based on tort (including negligence), warranty, breach of contract or any other legal theory.

Notwithstanding any damages that customer might incur for any reason whatsoever, NXP Semiconductors' aggregate and cumulative liability towards customer for the products described herein shall be limited in accordance with the Terms and conditions of commercial sale of NXP Semiconductors.

Right to make changes — NXP Semiconductors reserves the right to make changes to information published in this document, including without limitation specifications and product descriptions, at any time and without notice. This document supersedes and replaces all information supplied prior to the publication hereof.

Suitability for use — NXP Semiconductors products are not designed, authorized or warranted to be suitable for use in life support, life-critical or safety-critical systems or equipment, nor in applications where failure or malfunction of an NXP Semiconductors product can reasonably be expected to result in personal injury, death or severe property or environmental damage. NXP Semiconductors and its suppliers accept no liability for inclusion and/or use of NXP Semiconductors products in such equipment or applications and therefore such inclusion and/or use is at the customer's own risk.

Applications — Applications that are described herein for any of these products are for illustrative purposes only. NXP Semiconductors makes no representation or warranty that such applications will be suitable for the specified use without further testing or modification.

Customers are responsible for the design and operation of their applications and products using NXP Semiconductors products, and NXP Semiconductors accepts no liability for any assistance with applications or customer product design. It is customer's sole responsibility to determine whether the NXP Semiconductors product is suitable and fit for the customer's applications and products planned, as well as for the planned application and use of customer's third party customer(s). Customers should provide appropriate design and operating safeguards to minimize the risks associated with their applications and products.

NXP Semiconductors does not accept any liability related to any default, damage, costs or problem which is based on any weakness or default in the customer's applications or products, or the application or use by customer's third party customer(s). Customer is responsible for doing all necessary testing for the customer's applications and products using NXP Semiconductors products in order to avoid a default of the applications and the products or of the application or use by customer's third party customer(s). NXP does not accept any liability in this respect.

Terms and conditions of commercial sale — NXP Semiconductors products are sold subject to the general terms and conditions of commercial sale, as published at https://www.nxp.com/profile/terms, unless otherwise agreed in a valid written individual agreement. In case an individual agreement is concluded only the terms and conditions of the respective agreement shall apply. NXP Semiconductors hereby expressly objects to applying the customer's general terms and conditions with regard to the purchase of NXP Semiconductors products by customer.

Export control — This document as well as the item(s) described herein may be subject to export control regulations. Export might require a prior authorization from competent authorities.

Suitability for use in non-automotive qualified products — Unless this document expressly states that this specific NXP Semiconductors product is automotive qualified, the product is not suitable for automotive use. It is neither qualified nor tested in accordance with automotive testing or application requirements. NXP Semiconductors accepts no liability for inclusion and/or use of non-automotive qualified products in automotive equipment or applications.

In the event that customer uses the product for design-in and use in automotive applications to automotive specifications and standards, customer (a) shall use the product without NXP Semiconductors' warranty of the product for such automotive applications, use and specifications, and (b) whenever customer uses the product for automotive applications beyond NXP Semiconductors' specifications such use shall be solely at customer's own risk, and (c) customer fully indemnifies NXP Semiconductors for any liability, damages or failed product claims resulting from customer design and use of the product for automotive applications beyond NXP Semiconductors' standard warranty and NXP Semiconductors' product specifications.

HTML publications — An HTML version, if available, of this document is provided as a courtesy. Definitive information is contained in the applicable document in PDF format. If there is a discrepancy between the HTML document and the PDF document, the PDF document has priority.

Translations — A non-English (translated) version of a document, including the legal information in that document, is for reference only. The English version shall prevail in case of any discrepancy between the translated and English versions.

Security — Customer understands that all NXP products may be subject to unidentified vulnerabilities or may support established security standards or specifications with known limitations. Customer is responsible for the design and operation of its applications and products throughout their lifecycles to reduce the effect of these vulnerabilities on customer's applications and products. Customer's responsibility also extends to other open and/or proprietary technologies supported by NXP products for use in customer's applications. NXP accepts no liability for any vulnerability. Customer should regularly check security updates from NXP and follow up appropriately.

Customer shall select products with security features that best meet rules, regulations, and standards of the intended application and make the ultimate design decisions regarding its products and is solely responsible for compliance with all legal, regulatory, and security related requirements concerning its products, regardless of any information or support that may be provided by NXP.

NXP has a Product Security Incident Response Team (PSIRT) (reachable at PSIRT@nxp.com) that manages the investigation, reporting, and solution release to security vulnerabilities of NXP products.

NXP B.V. — NXP B.V. is not an operating company and it does not distribute or sell products.

Trademarks

Notice: All referenced brands, product names, service names, and trademarks are the property of their respective owners.

NXP — wordmark and logo are trademarks of NXP B.V.

© 2026 NXP B.V. All rights reserved.

Contents

1 UWBIoT API Reference 1
1.1 APIs : UWB IoT and SR250 1
1.1.1 UWB API Status Codes 1
1.1.2 UWB API Consts 6
1.1.3 UWB API Data Types 6
1.1.4 UWB Functional APIs 45
1.1.5 UWB Functional APIs (SR1XX/SR2XX Specific) 61
1.1.6 UWB OSAL APIs 117
1.1.7 UWB Helper APIs 130
1.1.8 APIs for Porting 131
Legal information 134

Please be aware that important notices concerning this document and the product(s) described herein, have been included in section 'Legal information'.

© 2026 NXP B.V. All rights reserved.

Date of release: 16 March 2026
Document identifier: UWBIOT_API_REFERENCE_GUIDE